

Harry Marsh



Darts Game

Contents Page

 Analysis	6
Outline of the project	6
Stakeholders	6
How the project is solvable by computational methods	7
Thinking Abstractly	7
Thinking Ahead	9
Preconditions	9
Constraints	10
Edge Cases & Considerations	10
Thinking Procedurally	11
Thinking Logically	15
Thinking Concurrently	16
Conclusion	16
Technical Implementation	17
Programming Language and Tools	17
AI Difficulty and Opponent Behaviour	17
Project Timeline - Gantt Chart	19
Testing and Debugging	20
Testing Strategy	20
Performance Optimization	20
User Interface & Design	20
UI Structure & Justification	20
Visual & Usability Considerations	21
Research into the project	21
Interview #1 - 19/02/25	21
Interview #1 evaluation	24
Game Review: PDC World Championship Darts - Pro Tour	25
PDC World Championship Darts Review	29
Game Review: Darts of Fury	32
Darts of Fury Review	36
Game Preference Survey	39
Survey Review	39
Interview #2 - 27/02/25	43
Interview #2 Evaluation	45
Features of the proposed solution	49
Limitations of the solution	52
Proposed Requirements	54

Hardware Requirements	54
Software Requirements	56
Success Criteria	56
 Design	61
Decomposition of the problem	61
Explanation of my Decomposition Diagram	62
Conclusion	66
Program Structure	66
Algorithms for the Solution	66
Startup Screen Algorithm	67
Gamertag Validation Algorithm	69
Achievement Unlocking Algorithm	71
Game Flow Algorithm	72
Dart Throwing Mechanics Algorithm	74
AI Decision-making Algorithm	77
Score Calculation Algorithm	79
Score Updating Algorithm	80
Game End Algorithm	81
Tournament Progression Algorithm	83
Leaderboard Update Algorithm	85
Conclusion	87
Entity Relationship Diagram	87
Usability Features	88
Loading Screen	88
Main Menu	89
Instructions	89
Leaderboard	90
Settings	90
Pause Screen	91
Game Over	91
Achievements	92
Playing the game	92
Main Variables and Data Structures in the Solution	94
Test Data	101
During Development / Iterative Testing	101
Post Development Testing	103
 Development	109
Development Plan	109
Iterative Development Log	110
Week 1 - Iteration 1 - Main Menu Navigation	110

Week 2 - Iteration 2 - Gamertag Input & Validation	115
Week 3 - Iteration 3 - Scoreboard + Calculation	1
Week 4 - Iteration 4 - Dart Throwing System	1
Week 5 - Iteration 5 - Game Loop & Turn System	1
Week 6 - Iteration 6 - AI Opponent Game Logic	1
Week 7 - Iteration 7 - Leaderboard System	1
Week 8-11 - Iteration 8 - Tournament Mode	1
Week 12-13 - Iteration 9 - Achievements System	1
Week 14 - Iteration 10 - Pausing and Instructions	1
Week 15 - Iteration 11 - Sound Effects	1
Week 16 - Iteration 12 - Settings and Background Switching	1
Stakeholder Follow-up	1
Week 17 - Iteration 13 - QOL Improvements	1
 Evaluation	1
Post Developing Testing	1
Testing for Function and Robustness	1
Usability Testing	1
Evaluating the Success Criteria	1
How the Criteria has and hasn't been met	1
Unmet Criteria and Further Development	1
Evaluating Usability Features	1
Evidence of Usability Features	1
Main Menu	1
Instructions	1
Leaderboard	1
Settings	1
Game Over	1
Achievements	1
Pause Menu	1
Playing the Game	1
Unmet Features and Further Development	1
Improvements and Limitations	1
Maintenance Issues and Limitations	1
Improvements and Dealing with Limitations	1
 Final Look at the Game	1
 Credits and Attribution	1



Outline of the project

For my project I am planning to create a game that is similar to the popular and growing sport of darts, where the user's character will play against a computer opponent in a 16 player knockout tournament, or have the option to play against a friend on the same system in a match of darts with the classical scoring system of 501 similar to that of the real game. This scoring system will be shown on the screen to provide the user with an indicator of how well they are doing and the checkouts needed, and the scoring system will encourage competition in the game. There will be a menu screen where the user navigates around four different areas playing the game; the main player vs computer tournament area; a player vs player area; seeing the number of trophies they have won; and an area to see the rules of the game and how to navigate around the game menus. The main reason for this program is to connect the sport of darts and the gaming industry, and enable existing or new fans of the sport to get involved in a game based around darts.

Stakeholders

Darts is a game that demands precision, focus, and strategy, making it a popular pastime for individuals who enjoy skill-based challenges. Gaming is often used as a way for people to relax and enjoy themselves. Developing a digital darts game provides an opportunity to bring the excitement of the sport to those who already have an interest in darts but may not always have access to physical equipment or the time to visit a venue. This game aims to offer a convenient and immersive experience, allowing users to engage with the sport from the comfort of their own homes.

Since a stakeholder is anyone with an interest in a problem, the primary stakeholders for my project will be individuals with an interest in darts, whether they are casual players, members of local darts leagues, or fans of professional tournaments. These stakeholders represent a range of skill levels and motivations, ensuring the game appeals to both seasoned players and newcomers.

To ensure the game meets the needs and expectations of darts enthusiasts, I will be regularly consulting (Anonymous), a regular darts player and lover of the sport. His knowledge of the sport, gained through both watching and playing, will provide valuable feedback on gameplay mechanics, design elements, and features that appeal to the target audience. In addition to (Anonymous) insights, I plan to expand stakeholder involvement by conducting surveys or brief interviews with casual players and league members to gather a broader perspective on key aspects of the game. This will ensure the final product is well-rounded and incorporates diverse feedback.

Regular discussions with (Anonymous) and other stakeholders throughout the development process will guide the refinement of features, such as scoring systems, difficulty levels, and visual design. By maintaining this feedback loop, the game will be able to capture the essence of darts while offering an engaging and enjoyable experience for a wide audience.

How the project is solvable by computational methods

Thinking Abstractly

Abstraction is a key process in computational thinking, allowing developers to simplify complex problems by removing unnecessary details. By focusing on essential features, abstraction enables a stronger focus on key features. In the case of my project, abstraction is used to determine which elements of a darts

game are crucial for gameplay and which can be simplified or removed without making the user experience overly simplistic.

The following abstractions will make the project solvable within the given constraints:

1. The game will be designed in 2D instead of replicating the 3D aspects of real-life darts. This reduces complexity by eliminating the need for advanced 3D modeling, physics, and rendering, allowing more focus on gameplay mechanics like aiming, throwing, and scoring.
2. Only necessary audio elements will be included, such as the sound of the dart hitting the board, sounds and celebrations for wins and losses, and minimal crowd noise as background ambiance. This avoids the need for complex audio mixing whilst ensuring that sound design enhances gameplay.
3. In real-life darts matches, the crowd adds to the atmosphere, but visually representing a crowd in the game would require additional animations and processing power. Instead, the game will only simulate the presence of a crowd through audio, such as subtle cheers or gasps, which is much simpler and focuses attention on the gameplay.
4. Rather than rendering an entire player character, the game will only show the hand or the throwing mechanism used to aim and throw the dart. This abstraction reduces development time by focusing only on what is functionally necessary for gameplay while still providing a sense of interaction and immersion. The character may only be fully shown during a winning screen.
5. Realistic dart-throwing physics will be simplified to core components like direction and power. These elements will be calculated using straightforward power bars rather than complex simulations, making the game both efficient and fun to play.

By using these abstractions, the project becomes more achievable within the available time whilst maintaining an authentic darts gameplay experience. Abstraction ensures the game remains focused on its primary goal of delivering an enjoyable darts simulation.

Thinking Ahead

Thinking ahead is where you consider any user inputs, as well as outputs of a solution before you begin programming it. By doing this I will find out how I can implement these into my program to give me a successful solution. I also need to consider how I will make my program as efficient as possible, and one way to do this is through reusing code, through functions, procedures and objects, as well as external libraries, given that they are already tested and functional, such as my use of the pygame library.

Preconditions

Preconditions define what must be true before an action occurs. This ensures valid inputs and prevents errors in execution. Some preconditions are as follows:

Valid Inputs for User Actions:

- When a user aims a dart, the input must be a valid direction (e.g., within the dartboard's bounds).
- The power selection for a throw must be within the predefined range (e.g., 0–100% power).
- Menu navigation should only accept valid key presses (e.g., arrow keys/wasd, enter key).

Game Start Conditions:

- A match can only begin once at least one player has been assigned.
- If the AI is playing, it must have a difficulty level selected before the match starts.
- The tournament mode can only begin with a full bracket of players.

Constraints

Constraints are the rules that must be followed throughout the game to maintain correctness, such as the following:

Score Calculation Constraints:

- The score must decrement correctly from 501.
- Players must finish on a double (as per standard darts rules).
- If a player scores more than their remaining total, the score should not decrease (i.e., the throw is invalid).

AI Behavior Constraints:

- The AI should never exceed the game's logic (e.g., it cannot "cheat" by scoring impossible shots).
- Difficulty levels should scale appropriately
- Beginner AI should have randomised inaccuracy.
- Advanced AI should have calculated shot placement with minimal mistakes.

User Interface & Accessibility:

- The game must remain playable on a keyboard, with optional control customisation.
- Font sizes and colour contrast should ensure clear readability for all players.
- Sound effects should have an option to toggle volume or mute.

Edge Cases & Considerations

Anticipating unusual or unexpected scenarios can prevent major issues in gameplay.

Some edge cases to consider are as follows:

Handling Invalid User Input:

- If a user presses an invalid key, it should not affect gameplay.
- If a user attempts to navigate outside the menu bounds, the selection should remain on the last valid option.

Game Flow Interruptions:

- If the AI's turn is interrupted, it should correctly reset before the next throw.

Performance Optimization Techniques

Since the game includes real-time input tracking, AI decision-making, and animation rendering, it is important to optimise performance to maintain smooth gameplay.

- **AI Processing in Background Threads** - AI calculations will be performed asynchronously, ensuring that user inputs remain responsive even when the AI is making decisions.
- **Scoreboard and UI Updates Only on Change** - Instead of continuously refreshing the entire UI, only elements that change will be updated to reduce CPU usage.

By implementing these techniques, the game will maintain a stable 30+ FPS, even on lower-end hardware.

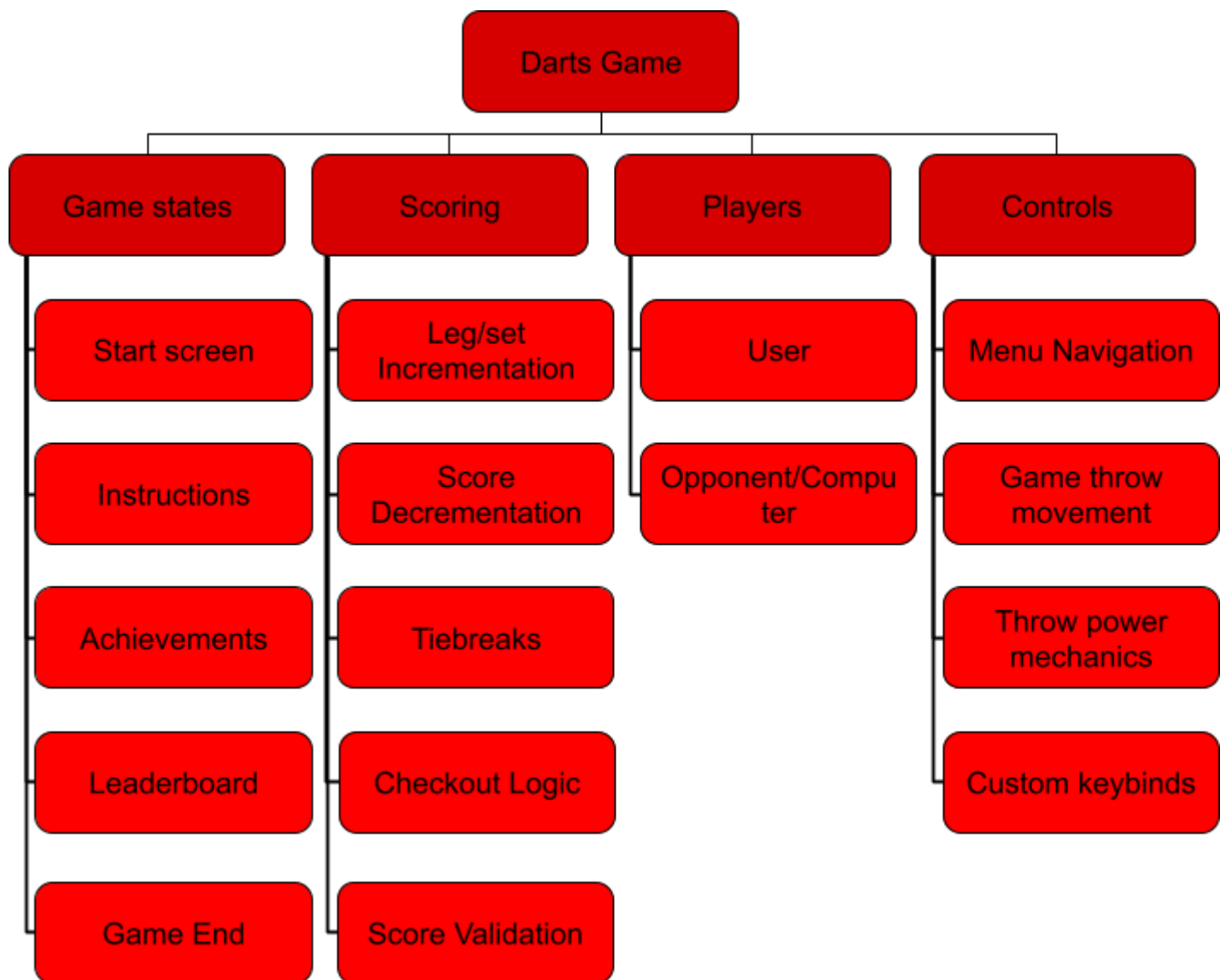
Thinking Procedurally

By applying decomposition, I will break down the overall problem into smaller, more manageable subproblems. This will enhance the efficiency of my solution, making it easier to develop, debug, and optimise. Each subproblem will correspond to a specific subroutine, ensuring a structured approach to coding.

To minimise repeated code, I will implement functions and procedures for key gameplay mechanics, such as:

- Score Calculation - A function will handle score tracking, including checking whether a player has won a leg or set.
- Throw Mechanics - A procedure will process the player's throw, determining where the dart lands based on input variables like power and direction.
- AI Opponent Behavior - A subroutine will control the AI's decision-making, adjusting difficulty and simulating throws.

Additionally, menu navigation and game controls will be structured into subroutines to keep the code organized and efficient. By structuring the game in this way, I ensure that the final product is scalable, maintainable, and responsive.



Game states

There will be a main menu which will have different options on it to play the game, instructions and rules for the game, a leaderboard menu and an achievements section. Pressing one of these will result in moving to that page in the game.

Scoring

The scoring will be decremented from 501 by points and upon “checking out” or reaching 0, legs and sets will be incremented. The scoring will result in a sense of progression and at the end of the game achievement upon winning. At the end of the game, there will be an animation and in the case of a tie, logic will be used to implement a tiebreaker. This will be researched further through stakeholders and surveys.

Players

This will decide the way that the user will be playing the game, and in turn how the opponent or computer will need to respond in order to try and win the game.

Controls

There are different controls that need to be considered within the game such as the navigation of the menu screens which will result in the changing game states to “play” or “leaderboard” etc, and how this will happen. For example, a mouse click or pressing keys, the movement and power of the darts in the game which will be with

mouse inputs or keys dependent on what my stakeholder and stakeholder group would prefer, and also whether or not to allow custom keybinds to make the game more user friendly.

Thinking Logically

Thinking logically involves identifying situations in your game where a decision must be made and considering the possible outcomes. This approach will help me to analyse how to handle various aspects of my game and the different results based on decisions made:

- **Menu Navigation:** The game will check which button is pressed and navigate accordingly, for example:

IF "Start Game" is selected → Load game scene

IF "Settings" is selected → Open settings menu

- **Scoring System:** When a player throws a dart, the game will check where it lands and subtract the correct amount from the total score. Logical conditions such as IF score reaches 0 → Player wins the leg will ensure the game follows official rules.
- **AI Decision Making:** The AI must decide where to aim and how accurate it should be. In the case of using different AI difficulties, it could look something like this:
 - IF AI skill = Beginner → 40% chance of missing target
 - IF AI skill = Advanced → 90% accuracy on checkouts

This logical thinking allows me to map out how user actions (like selecting menu options or scoring) affect the flow of the game, ensuring smooth gameplay and appropriate responses.

Thinking Concurrently

Thinking concurrently involves allowing the game to handle multiple tasks simultaneously, which helps make the game more efficient and responsive. For a darts game, this could include several things, such as:

- Continuously updating the score as players throw darts, without interrupting the flow of the game.
- Managing player turns and keeping track of their actions in real-time, so that as soon as a player throws a dart, the game responds instantly, updating the score and showing the result of the throw, all whilst maintaining an uninterrupted game experience.

By thinking concurrently, I ensure that the game can perform several tasks at once, making it more fluid and enjoyable for the player.

Conclusion

A computational approach is ideal for this project because it allows for real-time decision-making, dynamic score tracking, and AI-controlled opponents, all of which are fundamental to my game. The game requires a consistent and structured rule system (e.g., 501 scoring, checking out, AI decision-making), which can be implemented effectively through logical algorithms and data structures.

Additionally, a computational solution enables customisation and interactivity, as players can control their throws, and track their performance over time. This would be very hard to do in a non-digital format. The ability to process user inputs and dynamically adjust outputs (e.g. real-time leaderboards) further highlights why this problem is good to be solved using a computational approach.

Technical Implementation

Programming Language and Tools

The game will be developed using Python with Pygame due to the following reasons:

- Python is beginner-friendly, has clear syntax, and provides strong community support, making development faster and debugging easier, especially within the given time frame for the project.
- Pygame is a lightweight library designed for 2D game development, offering built-in support for graphics, input handling, and sound effects, making it ideal for this project.
- It allows cross-platform compatibility, meaning the game can run on Windows, macOS, and Linux without major modifications.

Alternative tools such as Unity (C#) and Godot (GDScript) were considered but ruled out due to their steep learning curve and additional complexity in rendering, which would be too unrealistic to produce within my given time frame for the project.

AI Difficulty and Opponent Behaviour

To create engaging gameplay, I plan to have AI opponents which have adjustable difficulty levels as follows:

Level	Behaviour
Beginner AI	Random errors and inconsistent accuracy.
Intermediate AI	More strategic but still makes occasional mistakes.

Advanced AI	Highly accurate with minimal errors.
Adaptive AI	Adjusts accuracy based on player performance.

AI Shot Calculation and Decision-Making

The Idea is that AI determines its throw based on ideal target positioning (so using x and y values for the board) and adding random error factors known as deviation, but is subject to change. The final dart position is calculated as:

$$X_{\text{final}} = X_{\text{target}} + (E_x \times R)$$

$$Y_{\text{final}} = Y_{\text{target}} + (E_y \times R)$$

where:

- $X_{\text{target}}, Y_{\text{target}}$ = Best target on the dartboard
- E_x, E_y = Error factor based on difficulty level (deviation)
- R = Random number between -1 and 1 (adds variation)

How Difficulty Affects Accuracy:

- Beginner AI: Large E_x, E_y values (big errors, frequent misses).
- Intermediate AI: Moderate E_x, E_y (some mistakes).
- Advanced AI: Small E_x, E_y (very accurate but not perfect).

Example:

An Intermediate AI aiming for Triple 20 ($x=50, y=100$) with $E_x=10$ and $E_y=10$ might land at (55,105), making it slightly off-target.

This ensures that lower difficulty AIs make more mistakes, while higher difficulties feel more competitive but still human-like.

Project Timeline - Gantt Chart

To manage the project efficiently, I have structured the development into four key phases, each with a planned length of time, which may vary based on issues that may arise:

Phase	Weeks	
Design and Research	1-4	Conduct stakeholder interviews, analyse competitor games, create UI templates, and finalise project scope.
Development Phase	5-10	Implement core game mechanics (throwing, scoring), AI logic, and UI elements.
Testing & Debugging	11-14	Conduct playtesting, optimise performance, and refine AI behaviour.
Evaluation & Feedback	15-16	Gather user feedback, analyse results, and finalise documentation.

This timeline ensures that each phase has clear objectives, preventing rushed development and ensuring a high-quality final product.

Contingency

If a particular stage of development, such as the AI development, takes longer than expected, other areas of development, such as the UI improvements, will be deprioritised to balance workload, or a change in approach may be required.

High-risk areas

The AI system and real-time scoring updates will require additional testing to ensure accuracy and fairness.

Testing and Debugging

Testing Strategy

The game will undergo two testing phases to ensure stability and gameplay balance:

- Testing for function and robustness → Check dart throw accuracy, score tracking, AI decision-making, etc
- Usability Testing → Qualitative feedback, check usability features met

Performance Optimization

- Ensure FPS stability by optimizing animations and AI calculations.
- Prevent AI lag by running calculations in background threads.

User Interface & Design

UI Structure & Justification

The game's interface will be minimalist yet functional, ensuring clarity and ease of use:

- **Main Menu:** Navigate between Play vs AI, Tournament Mode, Trophies, and Settings.
- **Gameplay Screen:** Displays dartboard, power bar, score, current turn, and checkout suggestions.
- **Tournament Bracket:** Shows progression in knockout format.
- **Settings Menu:** Adjust AI difficulty and sound control

Visual & Usability Considerations

- Neutral backgrounds maintain focus on gameplay.
- Subtle sound cues enhance immersion without being distracting.

Research into the project

Interview #1 - 19/02/25

I interviewed my stakeholder, (Anonymous), to gather some information regarding basic requirements in the game, and opinions on different features that could be implemented. I have focused the interview around 4 main areas: game aesthetics, scoring and achievements, controls, and audio. The aim is to find a more in-depth answer to each question so that I can gather key insights from an avid darts player and gamer as to what would best fit the game. The questions have been transcribed below (Harry , (Anonymous)):

Game aesthetics:

1. *Would you prefer more of a simplistic colour theme and background, or more colourful and unique?*
 - Personally, I prefer a more simplistic and classical look, because then I can focus attention on the gameplay instead of being distracted by the bright colours, however others may say otherwise.
2. *Would you rather a more gameplay focussed screen, simply displaying the dart board and a background, or have a split-screen such as in this image (show image) where you can also see the players and the crowd?*

- I think that simply having the dartboard displaying means that users can focus on their gameplay rather than half the screen being used to just show their character.



Scoring and Achievements:

1. *Do you think having a scoring system would benefit the darts game?*
 - 100%. I think that the game has a sense of objectivity with the inclusion of a scoring system and makes the game more enjoyable.
2. *Would you prefer the scoring to follow that of a normal darts match (501 with legs and sets) or have a more unique scoring system?*
 - A scoring system which is like the sport of darts would enable new players and darts enjoyers to understand the concept, however could be more confusing at first, but I think it is the better option, as it will also cater to those who already enjoy darts.

3. *Would you want trophies for winning an in-game tournament for example?*
 - Yes for sure, because it adds more objectivity to the game and adds another dynamic to the game, as players would want to get as many trophies as possible, meaning the game would be played more

Controls:

1. *What do you think the main game controls should be*
 - Personally, I like to use arrows keys to control the movement of my characters, or in this case darts, however some players may prefer to use a mouse movement or joystick, depending on the platform.

Audio:

1. *Do you think the game should have audio?*
 - Yes definitely, as it adds some immersiveness into the game and makes it more enjoyable to play in my opinion
2. *Do you think players should be able to hear the crowd, or simply just the audio of darts being thrown and landing on the board?*
 - I think that there should be enough audio to make it enjoyable but not too much to be distracting, so just having the audio of darts being thrown and landing would be good, but maybe at times you could hear the crowd cheer.

Interview #1 evaluation

Following on from my interview with (Anonymous), there are some features and key ideas I have pulled out that I could consider or implement into the game, which follow the same categories as used in the interview. Given that (Anonymous) is an avid darts player and enthusiast himself, I believe that he is representative of my stakeholder group and his ideas would be beneficial to the development of my game. Nevertheless, I would still like to gain some more insight from a broader audience.

Game aesthetics:

- No split-screen to see the players and crowd, solely focus on the gameplay and main components that matter towards it
- Prefers a more simplistic look, however survey research could give more insight into wider preference amongst players.

Scoring and achievements:

- Implement scoring in the style of a real-life darts match.
- Some sort of trophy or reward system to encourage players to play more

Controls:

- Preference for arrow keys to control the dart movement, however could use survey research to widen the perspective.

Audio:

- Simple audio cues to attract users to the game, so that the game is not boring, however not too much to distract users from the focus of gameplay.

Game Review: PDC World Championship Darts - Pro Tour

PDC World Championship Darts Pro Tour is a popular darts game developed by Rebellion Developments which can be played on PlayStation 3, Xbox 360, and Nintendo Wii.

- In the game you can either play as a well known darts professional or create a custom character to play as, in a variety of different game modes, such as career mode, party games, exhibition mode or 'play it your way', or simply practice, which provides both the option to emulate a real darts match in career mode, or to have a more fun and unique experience with party games.
- The controls vary depending on the device you are playing on. On Xbox, you use the left joystick to move the dart and the right joystick or 'A' button to throw the dart, whereas on a Wii controller, you point the controller at the screen to aim the darts and use the 'A' or 'B' buttons to throw the darts.
- In PDC World Championship Darts Pro Tour you can play against a computer AI, locally against another player in split-screen format, or against people on a different device.
- The game is presented from a third-person perspective, with players viewing the darts and throwing mechanics in a 3D environment, but the gameplay itself, such as the dartboard and UI, is mainly 2D.

Given the time constraints on the project and complexity of creating 3-dimensional graphics, I will be designing my game in 2 dimensions, as per the 'limitations of my project' section.

Main menu



The navigation menu is split into 8 different sections that include 5 different game modes, a custom character creation, online access, and a settings tab. There is a main home screen when you load onto the game that displays a navigation menu and a darts player and board outlined in the background which are clear for the user to navigate the menu. Links to Xbox Live, a way to play online against other users.

Career mode tournament selection



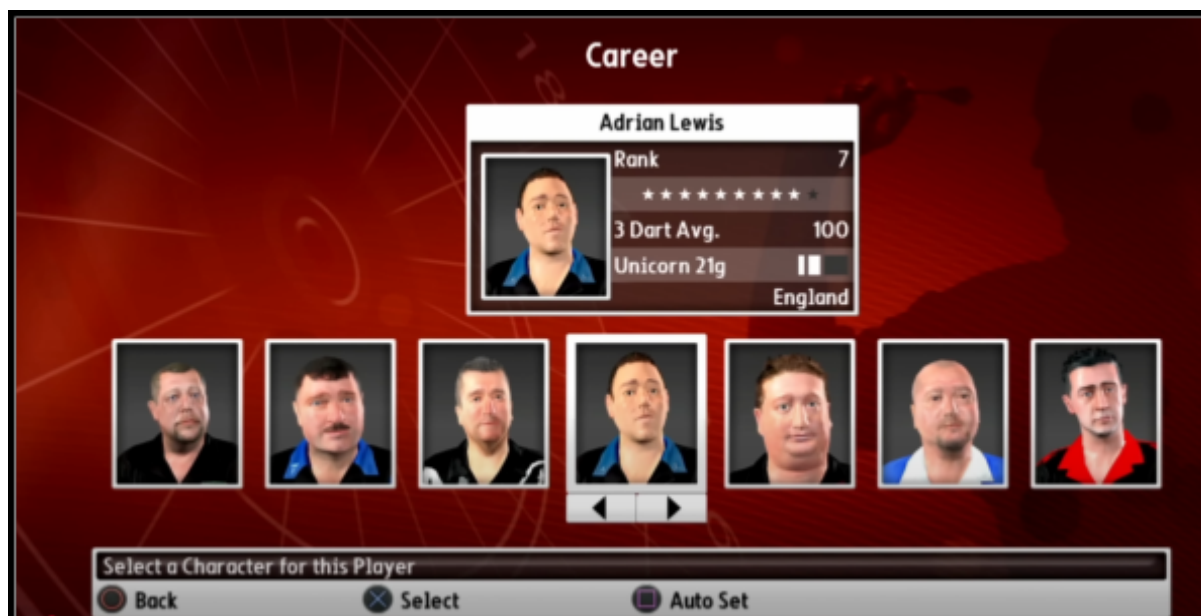
There are 7 different tournaments available in the career mode that are the same as those used in the sport of darts, making the game feel more immersive and representative of the sport. The career mode offers more progression by the option to go to the next season after the tournaments for the current season have finished, making the game more progressive and enjoyable to continue playing. Each tournament has its own entry requirement in order to play the tournament, which makes the game more realistic like it would be in the sport of darts, and also gives the game an element of progression. There is a display of your character's stats based on the tournaments throughout the season, where the tournament money is shown and the ranking based on the money made, which then can be used to enter other tournaments. Adds immersiveness.

Gameplay



The darts and dartboard look realistic and the movement looks logical. There is a power bar to add an element of difficulty to the game instead of it being point and click. The gameplay is split-screened with one side showing the board and the darts position and power, and the other side showing your player throwing the darts, your opponent, and a crowd, making the game feel more immersive and realistic. In the career mode, you can play in different places, just like that of real darts tournaments, such as at Ally Pally or Butlins Minehead. There is a scoring system which is the same as that of a real darts match, displaying the player names, score remaining and total “games” or legs and sets won.

Character selection



The character selection (not custom character) has a wide range of professional dart players that you can choose to play as in your career. This allows for less restriction within the game and users can play as their favourite player, making it more enjoyable. Each of the characters have their own unique characteristics and stats, whether it be their ranking, the type and weight of dart they use or their 3 dart average, which is a representation of their skill level.

PDC World Championship Darts Review

Feature from the Game	Opinion	Explanation of feature
Real Darts Scoring	Like	The game uses the official scoring system for darts, making the game feel authentic and true to the sport.
Sound of darts	Like	The sound of the darts hitting the board is

		realistic, adding to the immersion of the gameplay. It mimics the feel of an actual darts match, which enhances the experience.
Sound of crowd	Like	The crowd sound effects, such as cheering and reactions, help create the atmosphere of a real tournament. It adds excitement and makes the game feel more alive.
Avatar customisation	Like	Players can customize their avatars, giving players the chance to make a character like themselves.
Licensed Events	Like	The game features real darts events, such as the World Darts Championship, adding authenticity. Playing in these iconic tournaments against real-life pros makes the game feel official and exciting.
Player names	Like	The inclusion of actual professional darts players' names, like Phil Taylor

		and Raymond van Barneveld, adds realism to the game. It's great for fans who want to play as or against their favorite pros.
Limited gameplay depth	Dislike	Despite a career mode, the game lacks a lot of depth. The gameplay can become repetitive after a while, with limited ways to improve.
Splitscreen of board and crowd	Dislike	The split-screen view that separates the dartboard and the crowd can be distracting. It takes away from the focus on the actual gameplay and doesn't allow for an immersive view of the actual gameplay
Limited AI difficulty	Dislike	The AI difficulty can be too easy or too predictable, especially on lower levels. Once players get a grip on the mechanics, the game becomes less challenging. A more varied or dynamic AI

		would have made the game more engaging long-term.
--	--	---

I would like my project solution to include many of the aspects of PDC World Championship Darts that I liked, particularly the real-darts scoring, and darts audio, which my stakeholder also said he liked, as these will improve my game and make it more immersive and enjoyable to play for the average user. Features like the split-screen crowd are very distracting whilst playing the game so I will not be implementing them.

Game Review: Darts of Fury

Darts of Fury is an exciting and fast-paced darts game developed by Yakuza Studios, available on mobile platforms like iOS and Android. It offers players the chance to compete in various darts tournaments or challenge friends in online multiplayer modes.

- In the game, players can choose to play as one of several unique characters, each with their own abilities and stats.
- Players can also compete in various game modes such as Career Mode, Tournaments, Quick Play, or Multiplayer Mode, which allows for both online play and local challenges.

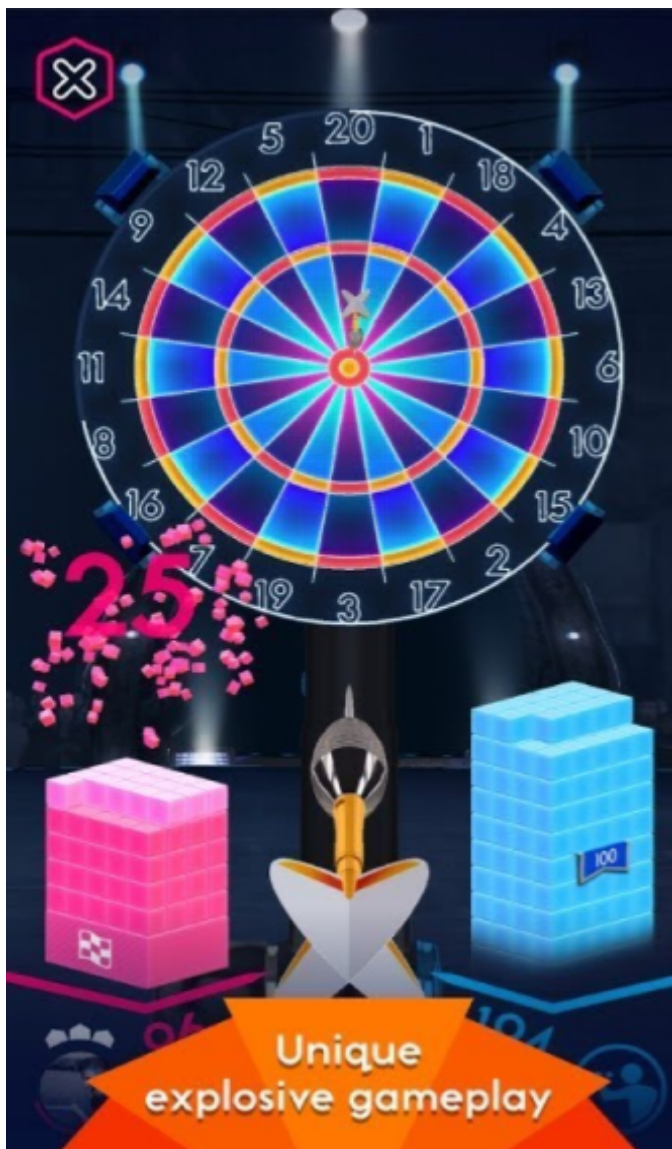
The game focuses on both strategic gameplay and fast-paced action, making it accessible to casual players while still offering depth for experienced darts fans.

- The controls are simple and intuitive. Players use touch screen controls to aim and throw darts by swiping and tapping the screen. The precision required to aim and release the dart adds an engaging challenge, and the game's visual feedback helps guide players in perfecting their throws.

- Darts of Fury presents the action in a dynamic 2D art style, with vibrant, colorful environments and detailed animations. The game's interface is clean and easy to navigate, with a focus on the dartboard and player stats.
- Players can compete against a computer-controlled AI, challenge a friend locally, or go head-to-head with online opponents.

Darts of Fury maintains its focus on high-quality 2D visuals, ensuring that the gameplay experience remains accessible, with smooth controls and engaging content for all players.

Gameplay



The gameplay is filled with bright colours which makes the game very appealing to a younger audience too. It also contains special game modes which make the game more enjoyable and replayable.

League Career Mode



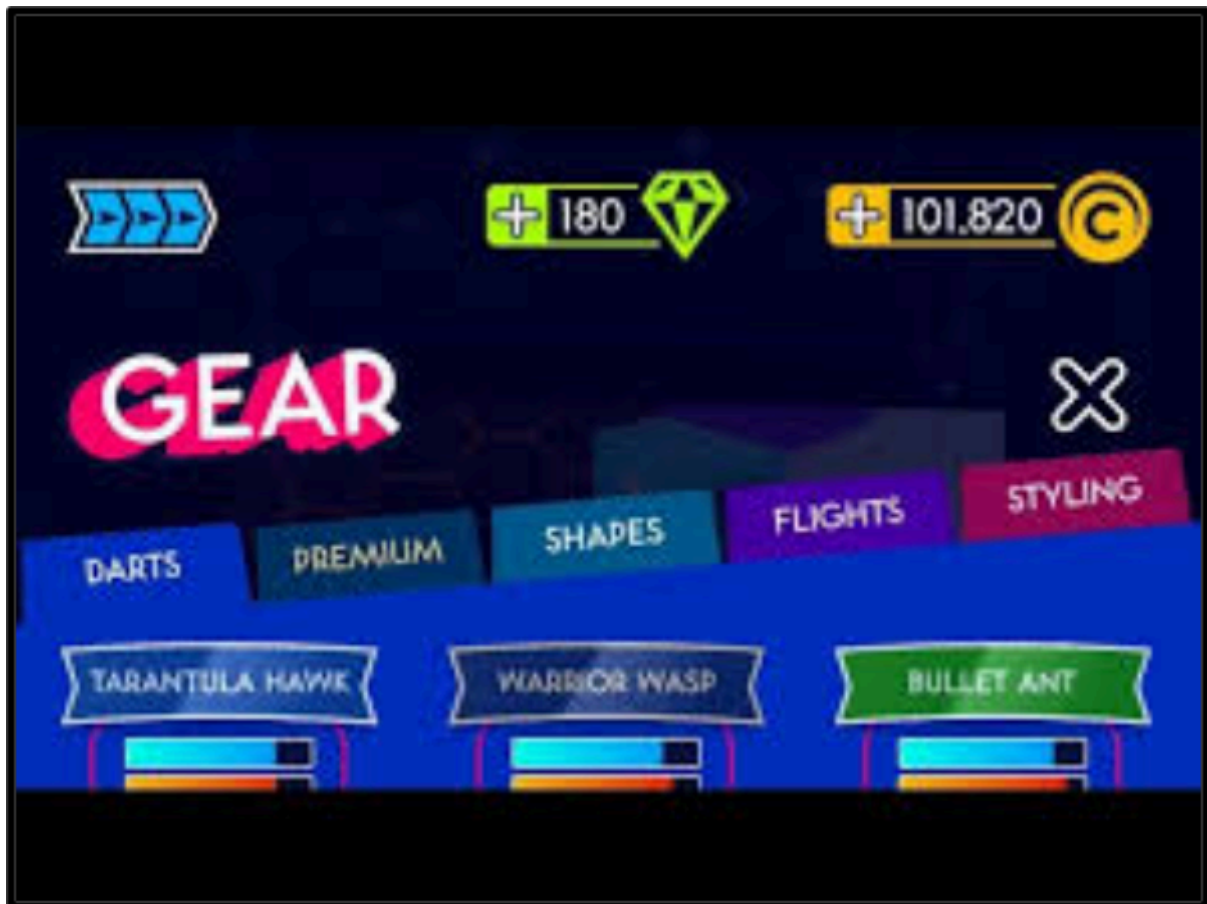
The game displays how much XP is required to reach the next league, which encourages people to play more to get to that higher league constantly. Each game gives you a prize of coins, depending on your league, which can be used to spend in the customisable shop as shown below. This adds another feature which may cater to collectionists or casual players who want to get some good looking cosmetics. Display showing your current league that you are in, and where you are in a “ladder” styled format in comparison to your next league.

Challenge Screen



There is a challenge screen that shows the opponent you are playing, adding more immersiveness and competitiveness to the game. The screen also display the name of the league you are in. The game mode being played is also displayed. The level of each player is displayed, showing an element of progression within the game as you want to get to higher levels.

Collectables and customisation



Gems are the premium currency which can be obtained through special events, achievements, or in-app purchases, and can be used for high end purchases and special purchases. Coins can be earned by playing matches and levelling up, and are used to upgrade and purchase new cosmetics, as well as enter certain game modes. Upgrades and purchases of new gear range from new darts and flights, all the way to the styling of the darts themselves. The gear shop adds another element of enjoyment to casuals or those who want to keep playing just to get their desired cosmetics.

Darts of Fury Review

Feature from the Game	Opinion	Explanation of feature
-----------------------	---------	------------------------

Colourful and stylised design	Like	The game has a vibrant and modern look, with bold colors and creative designs that make it visually appealing and attractive to play
Intuitive controls	Like	The controls are simple and easy to pick up, making it accessible for both casual players and newcomers to darts games.
Online multiplayer	Like	The game offers online matches, allowing players to compete against others worldwide, adding replayability and social interaction.
Diverse gamemodes	Like	Features a variety of modes, including tournaments and challenges, keeping the gameplay fresh and exciting.
Power-ups and special abilities	Like	The game includes fun, arcade-style power-ups, adding extra excitement and strategy to each match.

Limited customisation	Dislike	Customization options for players and avatars are somewhat limited, reducing personalization in the game.
Repetitive gameplay	Dislike	While fun with different game modes, the gameplay can feel repetitive after a while, especially if you're looking for deeper mechanics or variety.
Lack of realistic darts implementation	Dislike	The game takes a more arcade approach, so it lacks the realism of official darts simulations, which might disappoint fans of the game.
Basic AI	Dislike	The AI doesn't present much of a challenge on higher levels, making single-player matches less engaging over time.

I would also like my project solution to consider many of the aspects of this game, Darts of Fury, that I liked, particularly the colourful design of the game, of which I will be surveying a larger audience within my stakeholder target group, to find their opinions. I believe that having a colourful but still balanced design will improve my game and make it more captivating and enjoyable to play for the average user, whilst also not being distracting from the gameplay. Features like the lack of

realistic darts, I don't like, however I would like to gain insight from my stakeholder group.

Game Preference Survey

Following my discussion with my stakeholder, (Anonymous), as well as conducting research into both games, PDC World Championship Darts and Darts of Fury, I created a survey to research more into my problem to get some insight from my stakeholder group to whether they would enjoy the idea of a game themed around darts and given that the primary stakeholders for my project will be individuals with an interest in darts, I believed I would be able to gain some valuable information as to what they thought would expect to be within the game and what they would like to see implemented. I also sent the survey to video game enthusiasts as I thought they would be most familiar with the general design of games and could give good feedback regarding quality design.

Survey Review

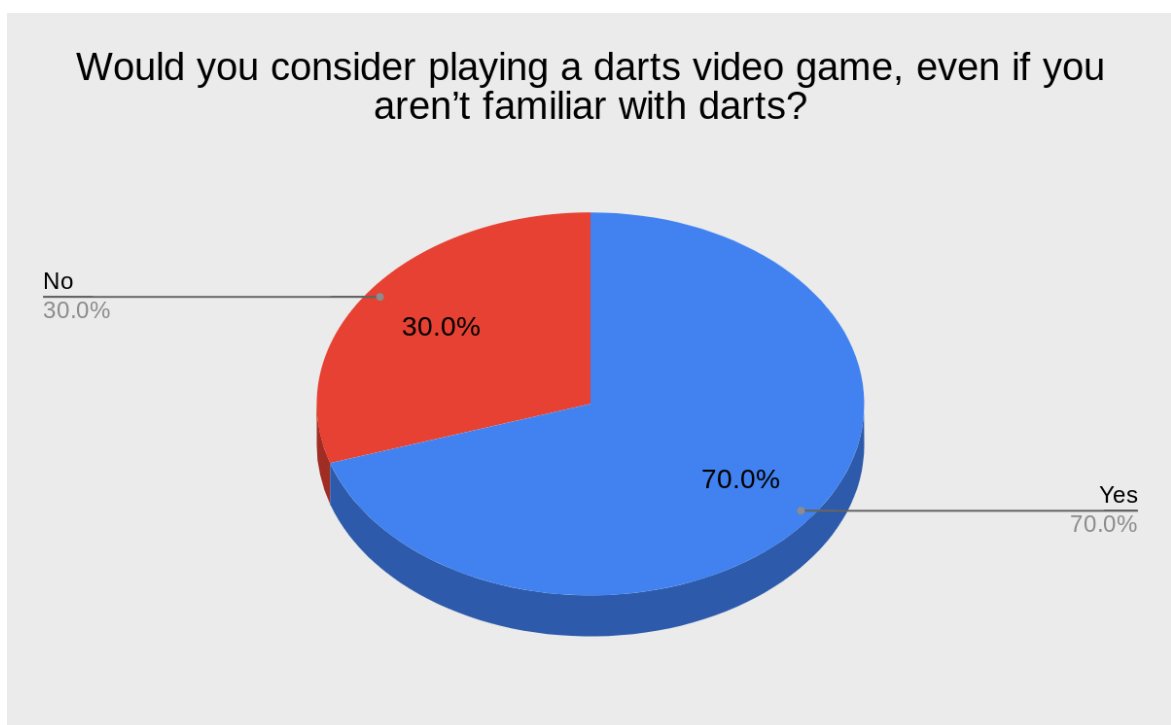
Q. How often do you play darts (Physical or digital)?

I found that 40% of my respondents to the survey do not currently play darts at all, however most of these were not part of my stakeholder group. Most of those who do play darts, said that they do so frequently or very often.

Q. How often do you play video games?

Only 10% of the survey respondents never play video games and of those who do play video games, they reported that most of them play 3-4 hours per week, with some playing more than 6 hours. This means that there is a large market for video games, especially people within my stakeholder group.

Q. Would you consider playing a darts video game, even if you aren't familiar with darts?



70% of my respondents stated that they would consider playing a video game based on darts, even if they are not familiar with darts, meaning that there is a large market for a game of my type, and many people, both in my stakeholder group and general gamers that I surveyed, would be likely to play my game.

Q. What style of gameplay would interest you the most?

60% of respondents said that they would prefer playing a realistic darts simulation game, 25% said they would prefer an arcade-styled game and 15% would prefer a mix of both. This indicates to me that the majority of people would prefer a more realistic simulation game, however a mix of both could be implemented.

Q. What game modes would you be most likely to play? (Select all that apply)

90% of my respondents would be most likely to play a single player vs AI gamemode and/or a career mode in which you can progress and do tournaments, 20% of the respondents of the survey said they would prefer to play a local multiplayer or minigame mode.

Q. Which control method do you think would feel most intuitive in a darts video game?

60% said they would most like a keyboard control method, 30% said mouse-based aiming and clicking, and only a small majority said a joystick/controller, making keyboard controls the primary focus currently, possibly with an option to switch control method.

Q. What type of darts game would you prefer to play?

The spread of responses from this question was fairly split between both a mix of simple and minimalistic design, as well as a bright and colourful design, so possibly I have to consider a midpoint between the two.

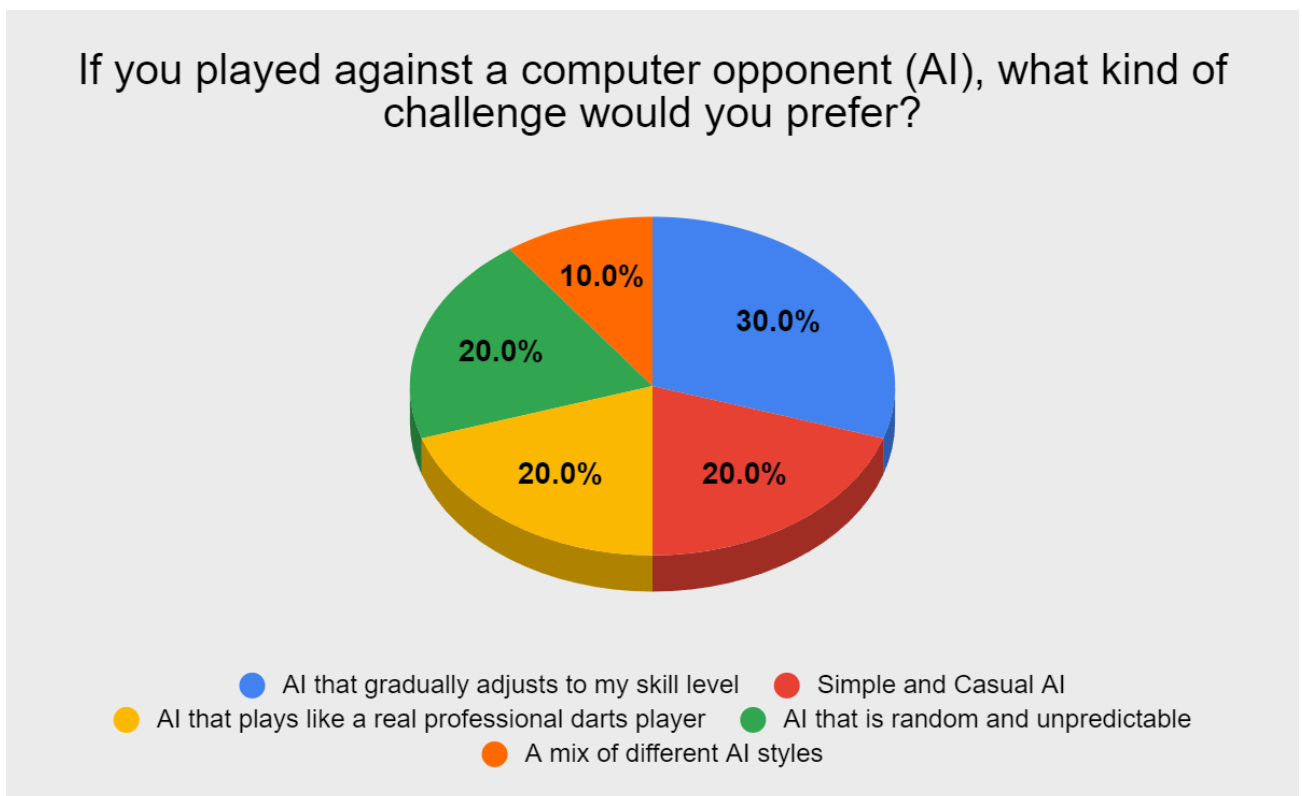
Q. How important is sound design to you in a game?

Over 65% of the respondents to my survey stated that sound design in a game is nice to have, with the remaining respondents saying that it is either not important, or essential. Given the majority of people said that sound is nice to have, I could consider implementing subtle audio to keep them engaged.

Q. Would you enjoy a trophy/achievement system to track progress?

75% said that they would enjoy a trophy or achievement within the game, which I will consider implementing, as this will give the game another feature for both casuals and completionists to work towards.

Q. If you played against a computer opponent (AI), what kind of challenge would you prefer?



The survey responses were evenly mixed, with some respondents wanting a simple and casual AI, some wanting a random and unpredictable AI, a professional AI, an adaptive AI and some wanting a mix of all of them. For this reason, I will likely consider implementing a range of difficulties to cater to all skill levels.

Q. Additional Comments?

Some respondents gave an additional comment at the end of the survey, including: “Looking forward to seeing how AI difficulty is balanced!”, “Make sure the UI is not too cluttered.”, “I think the crowd noise should be optional.”, and “Realism is key, keep it true to darts.”. From these comments I can also consider other things such as a volume bar/mute volume button within the settings and UI clutter.

Interview #2 - 27/02/25

After gathering insight from my stakeholder group, as well as a variety of gaming enthusiasts, about their preferences, I have turned my focus to my primary stakeholder, through an interview with (Anonymous), as transcribed below, in order to find out his overall opinion on specific features that could be implemented into the game, that have specifically been highlighted in my survey or extensive research into two other games revolving around darts, as well as some additional questions. I have split the questions into 2 categories: General gameplay and navigation, and additional features. (Harry, (Anonymous))

General Gameplay and Navigation

1. *How many legs/sets would you like to be won or lost in order to end the game?*
 - Possibly 2 or 3 legs per set and then 2 or 3 sets to take the match.
2. *How would you prefer to navigate around game menus?*
 - Using arrow keys or maybe left-click on the mouse.
3. *Should there be a variety of game modes and difficulties, to cater to different players?*
 - I think that this would be a good feature to add, however within the given time frame, could be difficult to achieve and require some complex programming.
4. *Would navigation around the menus using keybinds such as “s” for settings be beneficial?*
 - Yes, however it could become confusing if it is not clear or if there are too many keybinds.
5. *Would you like celebrations or animations for every leg that is completed?*
 - I think that a celebration or animation after every single leg that is played could become annoying to the player, so no.
6. *Would you like celebrations/animations for every match completed?*
 - No, maybe not a celebration, but a popup that shows a match or leg is won, as this would add to the immersive experience and add a sense of

accomplishment or defeat after each match, whilst being relatively easy to implement.

7. *Would you like there to be a tiebreaker in the game?*

- I think keeping the game simple is more beneficial, simply first to 2 or 3 sets, a tiebreaker is not needed.

Additional Features

1. *Should there be a leaderboard in the menu?*

- Yes, this would encourage competitive players to climb the leaderboards and become the very best by getting the most trophies or match wins total.

2. *Should each player have a gamertag (In-game name)?*

- Yes, it would make it clear to the player who they are playing as on the scoreboard and would also stand out from other players on the leaderboard.

3. *What would you want the achievements or trophies to look like?*

- Simple trophy cups or shields, similar to that of real darts, but could also have different designs depending on the event or difficulty.

Interview #2 Evaluation

Following this interview with (Anonymous), I used his opinions to consider the problem at hand. I created a list of features that resonate with his views towards the game design.

Gameplay and Navigation

- Navigation using arrow keys, including keybinds possibly but not too many, potential mouse navigation.
- Game length being 2 or 3 legs within 2 or 3 legs with the first player reaching this becoming the winner, and depending on how long the match is upon testing, could be altered.
- Popup animations for each leg, set, or match won or lost.

Extra Features

- A gamertag that can be displayed on the scoreboard as well as leaderboard in the menus.
- A trophy or achievement menu where all of the players' tournament wins are documented in his career.
- A leaderboard displaying the most trophies won or matches total won.

I also made a basic sketch of what the game may look like, including both a menu screen, and a gameplay screen:

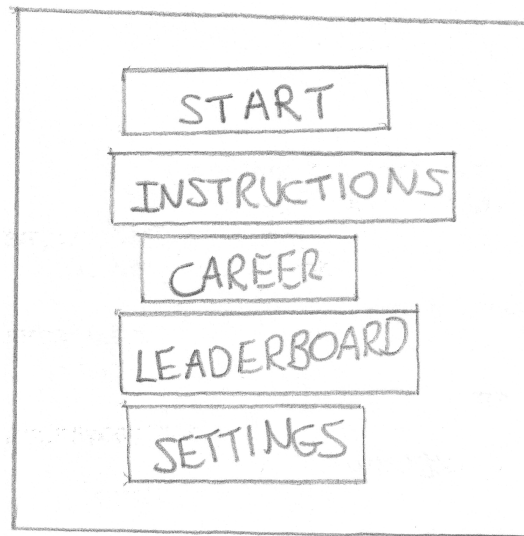


Figure 1. Main menu initial draft

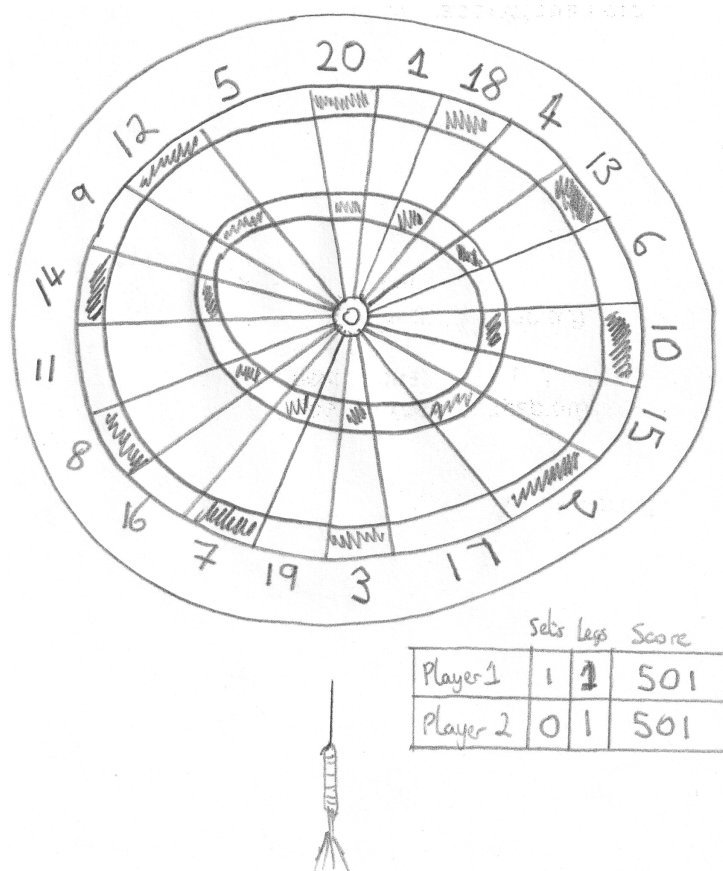


Figure 2. Gameplay ideas

LEADERBOARD

MATCH WINS
TOURNAMENTS WON

Player 1	10
Player 3	7
Player 2	0

Figure 3. Leaderboard sketch



Figure 4. Achievements screen

Main menu screen example, which links to other pages in the game. There is a leaderboard idea, with two options, displaying total matches won, or overall tournaments won. Proposed gameplay screen design, with a simple board and dart design and a display scoreboard. This is the proposed achievements screen, with the number in the middle indicating the number of times the player has won it, and the achievement name for what it was for at the base of the trophy.

Features of the proposed solution

After thorough research on both PDC World Championships Darts and Darts of Fury, as well as my interviews with my stakeholder (Anonymous) and a survey, I have devised a proposed feature list below, with justification as to why I believe they are needed for my project.

Feature	Description & Justification
Main menu	The game will have a structured menu allowing users to navigate between different game modes, settings, achievements, and leaderboards. This ensures a clear and user-friendly experience.
Scoring Decrementation and Leg/Set Incrementation	The game will use the official darts scoring system, decrementing from 501 with proper validation for checkouts. This ensures an authentic experience

	for darts players. Upon reaching 0, a leg will be given to the player, and sets will be played as follows: 2 legs → 1 set 2 sets → win the match This format means that it is not too long or short of a format, remaining enjoyable to players.
Scoreboard and Live updating	Displays player scores, throws, and required checkouts dynamically. Ensures players can track progress and strategy without many external calculations.
AI Opponent (Multiple Difficulty Levels)	AI opponents simulate real darts players with varying skill levels: beginner (randomised errors), intermediate (strategic mistakes) and advanced (high accuracy). This ensures long-term engagement for all skill levels.
Gamertag System & Player Identification	Each player has a custom name displayed on leaderboards, improving clarity within the game
Keyboard-Based Controls (With Custom Keybinds)	Players can customise their controls for aiming and throwing, improving accessibility and comfort.
Leg/Set Win Screens	When a leg or set is won, the game displays a transition screen with animations to enhance immersion.

Tournament Mode (16-Player Knockout)	A structured knockout-style tournament mode where the player competes against AI opponents. Winning matches progresses them through rounds until they reach the final. This adds long-term gameplay value. The tournament mode follows a 16-player knockout structure, ensuring a balanced progression system while keeping the experience engaging. • A smaller tournament (e.g., 8 players) would result in fewer matches, limiting competitive depth. • A larger tournament (e.g., 32 players) would make matches take too long, reducing accessibility for casual players. • 16 players allow for four rounds (First Round, Quarter-finals, Semi-finals, Final), maintaining challenge while preventing fatigue.
Leaderboard & Tournament Tracking	A leaderboard tracks match wins and trophies earned in tournaments. Competitive players are encouraged to play more and improve their ranking.
Achievements & Trophy System	Players unlock trophies for winning

	tournaments or achieving high checkouts. This encourages replayability.
Basic Sound Effects for Immersion	Includes dart impact sounds, and a sound for leg or set wins. These small touches enhance the gaming atmosphere.
Customisable Backgrounds & Dartboards	Players can choose from different background themes This adds aesthetic variety without affecting gameplay.
Game Settings & Accessibility Options	Includes volume control, key remapping, and difficulty adjustments to cater to different player preferences and accessibility needs.

Limitations of the solution

Similarly to the proposed features of my project, I have devised a table with the limitations of the solution, with justification as to why they were not chosen to be included in the game.

Feature	Description & Justification	Why this is a limitation
Adaptive AI	The AI has preset difficulty levels but does	Implementing machine learning AI requires large

	not dynamically adjust based on player performance.	datasets and real-time calculations, which exceed the scope of this project, and would be too complex to implement in the time that I have.
Complex audio and visuals	The game does not include live commentary from commentators, or excessive crowd noise.	The time constraints on the project mean adding these features would not be a good use of time. Audio can also become too distracting.
Use of a mouse as a control method	The game does not use mouse inputs for aiming or general controls, relying instead on keyboard control.	I will be developing the game in Python using Pygame. Mouse use in Python is much more complex than arrow keys and would be difficult to do within the time constraints
Realistic dart physics and movements	Instead of using complex physics (gravity, spin, air resistance, dart wobble), the game uses a power bar and aim-based system.	Simulating true dart physics requires advanced physics engines, which would be too complex to implement.
3D Graphics	The game is developed in 2D using Pygame, with a	3D modeling and physics would significantly

	simplified dart-throwing animation.	increase development time, due to the complexity.
Online Multiplayer	Players can only compete on the same system (alternating turns).	Implementing real-time networking and matchmaking is beyond my project's technical scope.

Proposed Requirements

Hardware Requirements

Hardware Requirement	Minimum Requirement	Recommended Requirement	Justification
Processor (CPU)	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5	A modern processor ensures smooth execution of the Python program and AI calculations.
RAM	4GB	8GB	More RAM allows for smoother gameplay, especially when handling AI logic

			and real-time updates.
Graphics Card (GPU)	Integrated graphics (Intel HD 4000 or equivalent)	Dedicated GPU (NVIDIA GTX 1050 / AMD Radeon RX 560)	Pygame is lightweight, but a dedicated GPU helps render visuals more smoothly.
Storage	500MB free space	1GB free space	Python and Pygame require little storage, but extra space is needed for assets and updates.
Display	1280 x 720 resolution monitor	1920 x 1080 resolution monitor	A higher resolution allows for a better gaming experience and clearer UI.
Input Devices	Keyboard	Keyboard and mouse	The game is designed for keyboard input, but in future may support mouse control.

Software Requirements

Software Requirement	Version/Details	Justification
Operating System (OS)	Windows 7 (64-bit)	Python and Pygame can run on Windows 7, but Windows 10+ is recommended for better support and performance.
	macOS 10.13 (High Sierra)	The minimum version that supports modern Python versions and Pygame without compatibility issues.
	Ubuntu 18.04 LTS (or equivalent)	Earlier versions may lack necessary dependencies, but 18.04 ensures good Python 3 and Pygame compatibility.

Success Criteria

Number	Requirement	Justification	Reference	
1	Startup Screen			
	1.1	The game must have a startup screen with options to proceed.	The user will be able to navigate around to get to their desired game state. This provides a clear entry point.	PDC World Championship Darts research
	1.2	Users must be able to navigate menus using wasd keys	Provides Intuitive navigation for all users.	Usability Features, Survey - Preferred Controls
	1.3	A back button must be included in submenus.	Ensures users can return to previous menus without restarting the game.	Usability Features, Research into other games
2	Gameplay Mechanics			
	2.1	Players must be able to aim the darts using a controlled input method.	Allows for precision in gameplay.	Interview - Stakeholder Feedback
	2.2	A power meter must determine the strength of the dart throw.	Introduces skill-based mechanics.	Research into other games
	2.3	The dart must follow a logical path based on user input such as where	Ensures realistic gameplay.	Thinking Logically, Research into

		they aim, and the amount of power.		other games
	2.4	User should be able to pause game during gameplay	Allows users to take a break if needed	Research into games
3	Scoring			
	3.1	The game must follow official 501 darts scoring, decrementing scores correctly.	Ensures accuracy and realism.	Research into other games, Stakeholder Interview
	3.2	The player must finish on a double to win the game.	Follows official darts ruling and ensures accuracy.	Research into other games
	3.3	A scoreboard must display the current score for the leg, as well as number of legs and sets won for each player.	Enhances user experience by providing necessary feedback.	Research into other games - PDC World Championship Darts, Stakeholder Interview.
4	AI Opponent and Difficulty Levels			
	4.1	The AI opponent must follow logical decision-making when throwing darts.	Ensures fair and competitive gameplay.	Research into other games
	4.2	The game must include multiple AI difficulty levels	Allows players to choose a level of challenge	Interview - Stakeholder

		(Beginner, Intermediate, Advanced).	that suits them, encouraging replayability.	Feedback
	4.3	The AI should make more frequent errors at lower difficulty levels.	Simulates realistic AI behaviour based on difficulty.	Thinking Logically, Research into other games
5	Game Flow and Win Conditions			
	5.1	The game must detect when a valid checkout has been achieved.	Ensures accurate match results and fair gameplay.	Research into other games
	5.2	A victory screen must be displayed when a player wins.	Provides clear feedback on match results and adds an element of achievement.	Interview - Stakeholder Feedback
6	Achievement and Leaderboard System			
	6.1	Players must earn trophies for completing particular achievements or winning tournaments.	Encourages long-term engagement and adds competitiveness.	Interview - Stakeholder Feedback
	6.2	A leaderboard must display the top players based on different categories such as total match wins or tournament wins.	Adds competitiveness to the game, encouraging players to keep playing to become the best.	Interview - Stakeholder Feedback

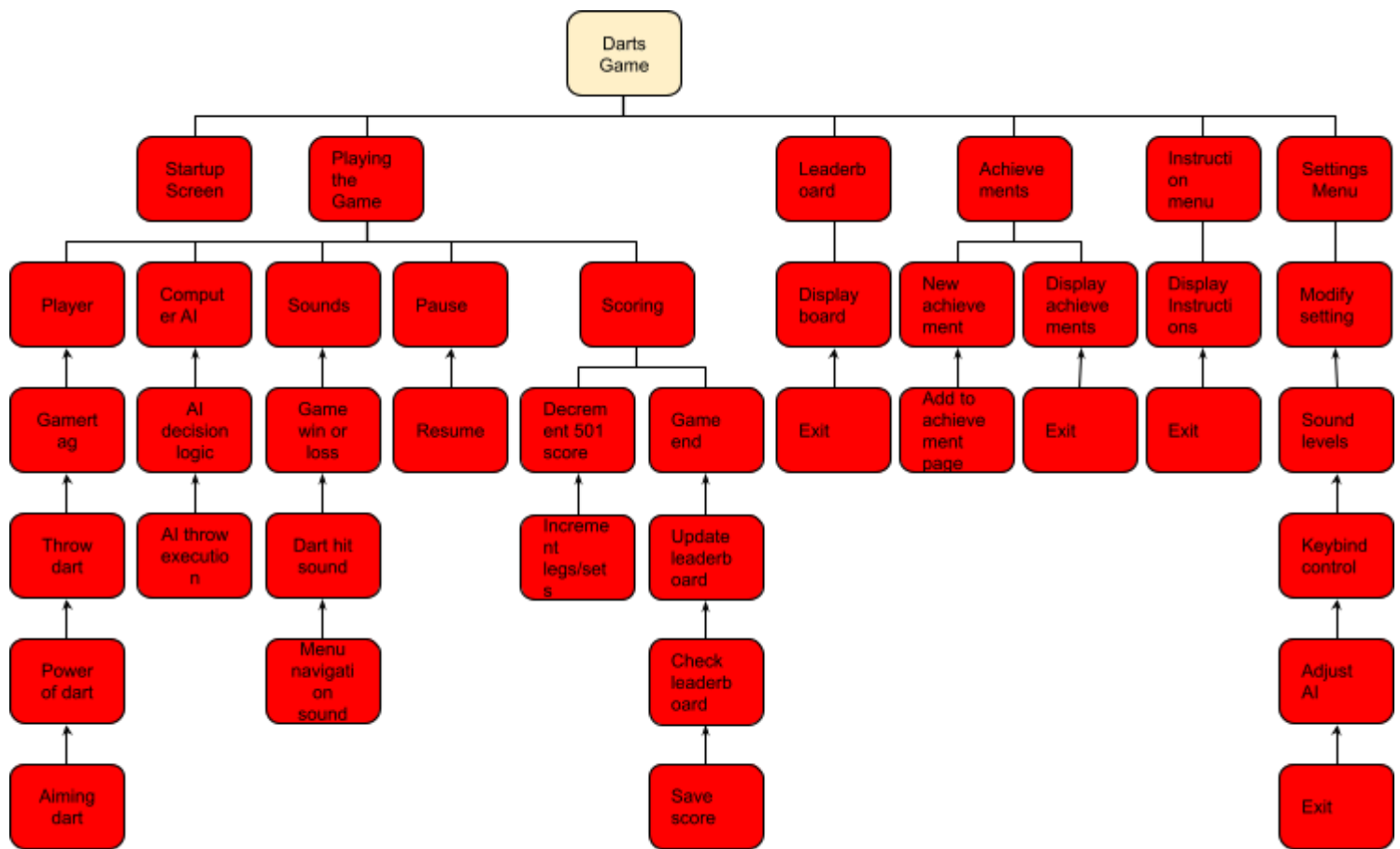
	6.3	Each player must have an individual gamertag that will be displayed on the leaderboard.	Individualises each player and gives players' a sense of achievement that is unique to them.	Interview - Stakeholder Feedback
7	Audio Feedback			
	7.1	Sound effects must be included for dart hits, menu navigation and game events.	Improves game immersion.	Interview - Stakeholder Feedback, Survey Feedback
8	Game Settings and Customisation			
	8.1	Players must be able to adjust audio settings.	Ensures accessibility for different player preferences.	Interview - Stakeholder Feedback, Research into other games
	8.2	AI difficulty must be adjustable	Provides a tailored experience for different skill levels.	Survey - Difficulty Preferences
	8.3	Background can be changed	Provides wider experience and customisation options	Stakeholder Interview
	8.4	Keybinds can be changed	Provides wider accessibility and option with how you play the	Research into games

			game	
9	Performance and Stability			
	9.1	The game must be run at a minimum of 30 FPS	Ensures smooth gameplay on supported hardware.	Hardware Requirements, Testing Plan
	9.2	The game must not crash during standard gameplay.	Provides a reliable user experience.	Testing Plan



Decomposition of the problem

In order to approach the problem better, I have broken it down into designated subproblems using modularisation. This creates different modules with subroutines inside them, so that when I program my solution, I can do so efficiently, in a hierarchical fashion which will enable me to better manage my time, and see how they link to one another to create a final solution.



Explanation of my Decomposition Diagram

There are six main categories of design that must be considered in order to make up the final solution: Playing the Game, Leaderboard, Achievements, Instructions Menu, and Settings Menu. Each category has been broken down into smaller subsections to define the functionality and processes of the game. The choice of these subcategories comes down to the different aspects of the game, for example, users should be able to view a leaderboard for their scores, see how to play, customise their experience through settings, and also play the game, which is why it makes sense to split my problem down into these individual categories and the further categories below that. This allows me to focus on just a single aspect of the game at a time, making it less stressful and time-consuming for me to program and design, and will then enable me to combine them together to form a complete solution.

Startup Screen

- This will display the main menu to the users, with the ability to see each menu option to proceed with by using arrow keys and the enter key.

Playing the Game

- This section outlines the core gameplay mechanics of the game itself.

Player

- Players can enter a gamertag to personalise their experience and save progress.
- Players can control the dart through aiming, selecting power, and throwing.

Computer AI

- The AI opponent uses decision-making logic to select throws based on difficulty level.
- AI executes throws according to programmed logic to simulate a real opponent.

Sounds

- The game includes sound effects such as:
 - Dart hits sound when a dart lands on the board.
 - Game win/loss sound to indicate match results.

Pause

- Players can pause the game at any time.

- The option to resume allows the player to continue from where they left off.

Scoring

- The game follows the 501 scoring system, where scores decrement with each throw.
- Incrementing legs/sets tracks the overall match progression.
- Game End
 - The system updates scores when a match ends.
 - The leaderboard is updated with new rankings.
 - Players can check their position on the leaderboard of wins.

Leaderboard

The Leaderboard tracks player performance and displays rankings.

Display Board

- Shows a list of top-ranking players.

Check Leaderboard

- Players can see their position among others.

Save Score

- Scores are recorded and stored for future reference.

Exit

- Players can exit back to the main menu.

Achievements

Achievements reward players for reaching milestones and excelling in the game.

New Achievement

- Players unlock achievements for feats such as tournament wins, perfect scoring legs, or other secret features.

Add to Achievement Page

- Newly unlocked achievements are stored in a collection.

Display Achievements

- Players can view unlocked and locked achievements.

Exit

- Players can return to the main menu.

Instruction Menu

This section provides guidance on gameplay and mechanics.

Display Instructions

- Shows an overview of the game's controls and mechanics.

Exit

- Allows players to return to the main menu.

Settings Menu

Players can customise various game settings to improve their experience.

Modify Settings

- Players can change different game settings.

Sound Levels

- Adjust sound effects volume.

Keybinds Control

- Customise controls for accessibility and preference.

Adjust AI

- Modify AI difficulty to make the game easier or more challenging.

Exit

- Return to the main menu.

Conclusion

This decomposition diagram breaks down the Darts Game into logical categories, ensuring clear functionality and efficient development. Each section contributes to the overall gameplay, making the game structured, interactive, and engaging for players.

Program Structure

The solution to my problem will be written using a functional and procedural programming paradigm, but also with a bit of object oriented programming for certain aspects such as my player class to track player variables. This is due to its maintainability when it comes to debugging and testing code, and as I am using pygame in python, it is much simpler to use a function approach than an object based approach in most areas, but by using objects for my player class, it enables me to better reuse code.

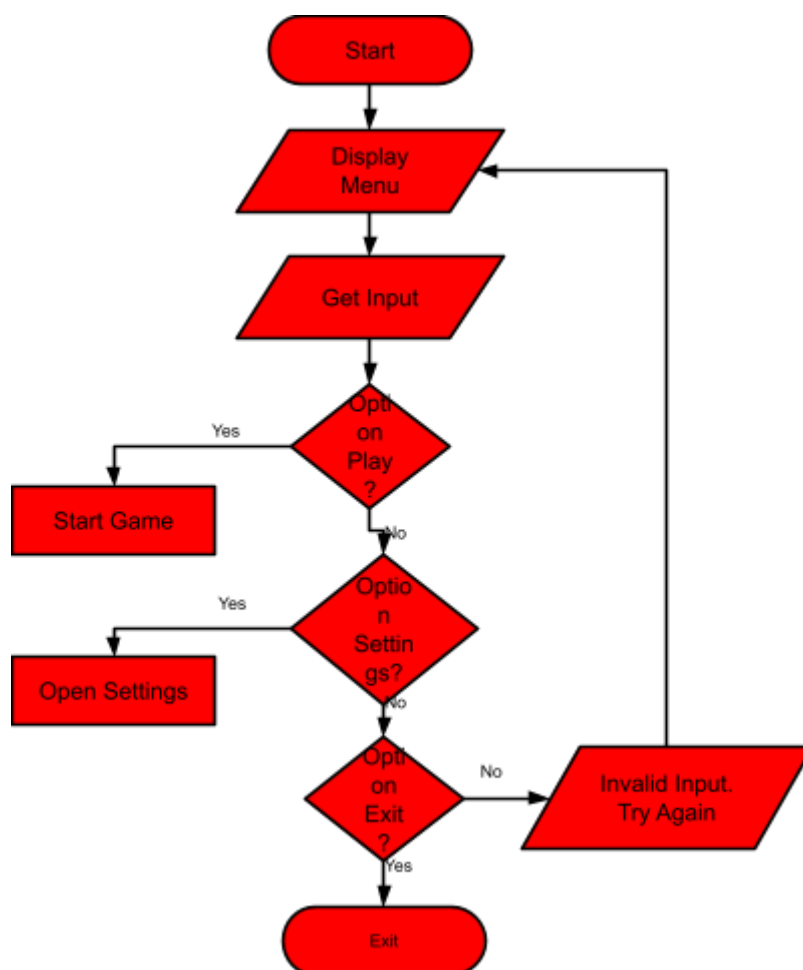
Algorithms for the Solution

To develop my solution effectively, I need an outline of the different features in the game and how they will work. I have provided a more indepth design of my game

by using flowcharts to show a brief overview of each key subprogram from my decomposition diagram, as well as 'code blocks' to write sample code.

The following algorithms outline the core functionality of the Darts Game. Each algorithm is written in python-styled pseudocode and has explanatory notes about it.

Startup Screen Algorithm



The main menu allows players to choose between different game modes and settings. This is implemented using a state-based approach. The arrow keys allow the player to navigate through menu options, and pressing Enter selects an option and takes you to the designated screen, whether this may be the gameplay,

leaderboard or settings, not all of which are within the sample algorithm. The highlighted option cycles through when pressing Up or Down. This is important to ensure that users can effectively switch between game states and explore different options.

None

```
FUCNTION main_menu()

    selected_option = 0 # Tracks which menu option is highlighted

    menu_options = ["Play", "Settings", "Exit"]

    WHILE menu_active

        FOR event in events

            IF event.type == Quit THEN

                return # Exit the game

            ELIF event.type == KEYDOWN THEN

                IF event.key == DOWN THEN

                    selected_option = (selected_option + 1) %
len(menu_options) # Move down

                ELIF event.key == UP THEN

                    selected_option = (selected_option - 1) %
len(menu_options) # Move up

                ELIF event.key == RETURN THEN

                    IF selected_option == 0 THEN

                        start_game() # Start a new game
```

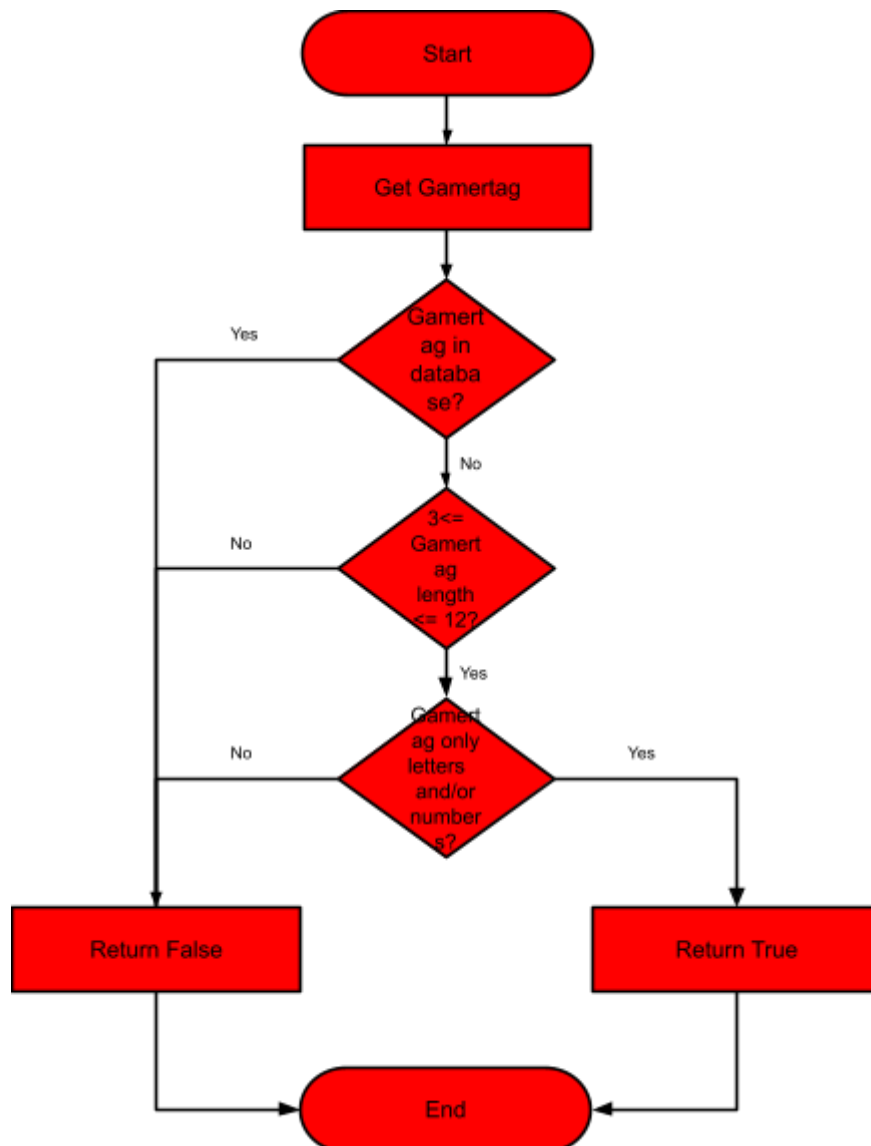
```
        ELIF selected_option == 1:

            open_settings() # Open settings menu

        ELIF selected_option == 2:

            return # Exit menu
```

Gamertag Validation Algorithm



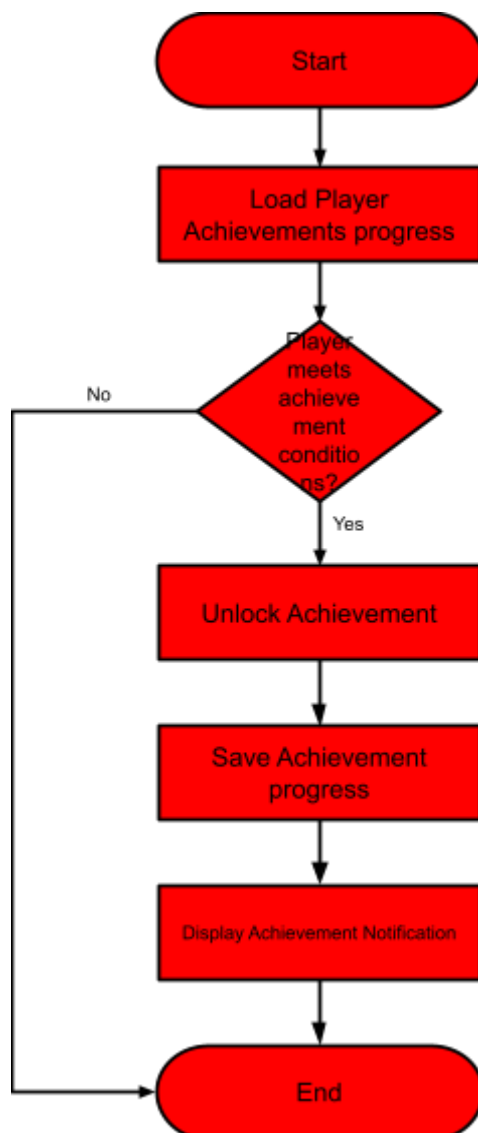
Ensures that the entered gamertag meets length and character requirements, and not already existing. Checks that the gamertag is between 3 and 12 characters long and only contains letters and numbers. This is particularly needed in order to make sure that no 2 users have the exact same gamertag so that each user has uniqueness and to avoid complications with the data storage.

None

```
FUNCTION validate_gamertag(gamertag)
```

```
IF gamertag not in database THEN  
  
return 3 <= len(gamertag) <= 12 AND gamertag.isalnum()  
  
return False
```

Achievement Unlocking Algorithm



Tracks achievements and updates them if the player meets criteria for achievements, unlocking them when achieved. This algorithm adds an element of achievement and reward for playing the game and completing set challenges, encouraging longevity of the game.

None

```
PROCEDURE check_achievements(player)

    achievements = {

        "Score 1000 Points": player.score >= 1000,

        "Win 10 Matches": player.matches_won >= 10,

        "Checkout 170": player.highest_checkout == 170

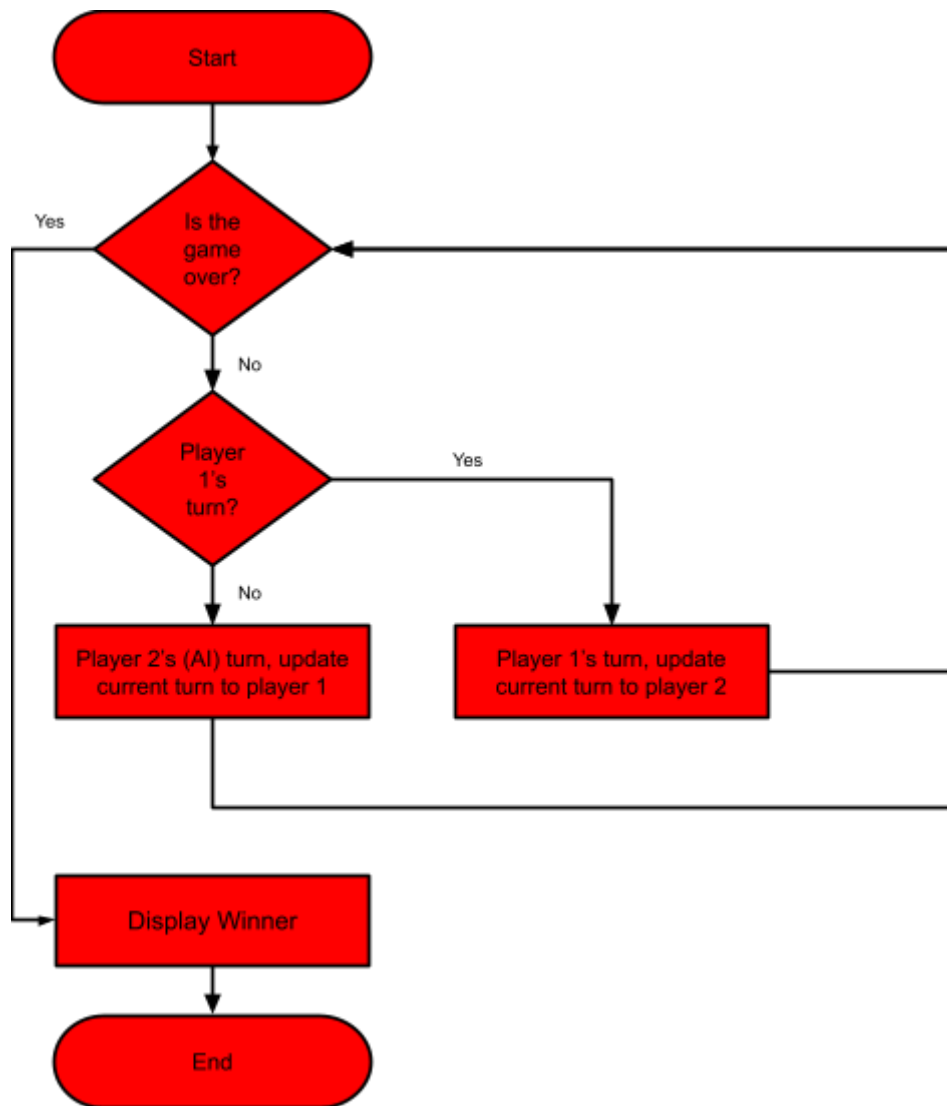
    }

    for achievement, condition in achievements.items() DO

        IF condition THEN

            unlock_achievement(player, achievement)
```

Game Flow Algorithm



This algorithm ensures that turns are handled correctly between players and the AI, tracking game progression and determining the winner. This ensures fair and structured turn management, alternating between the human player and AI while checking if the game is complete.

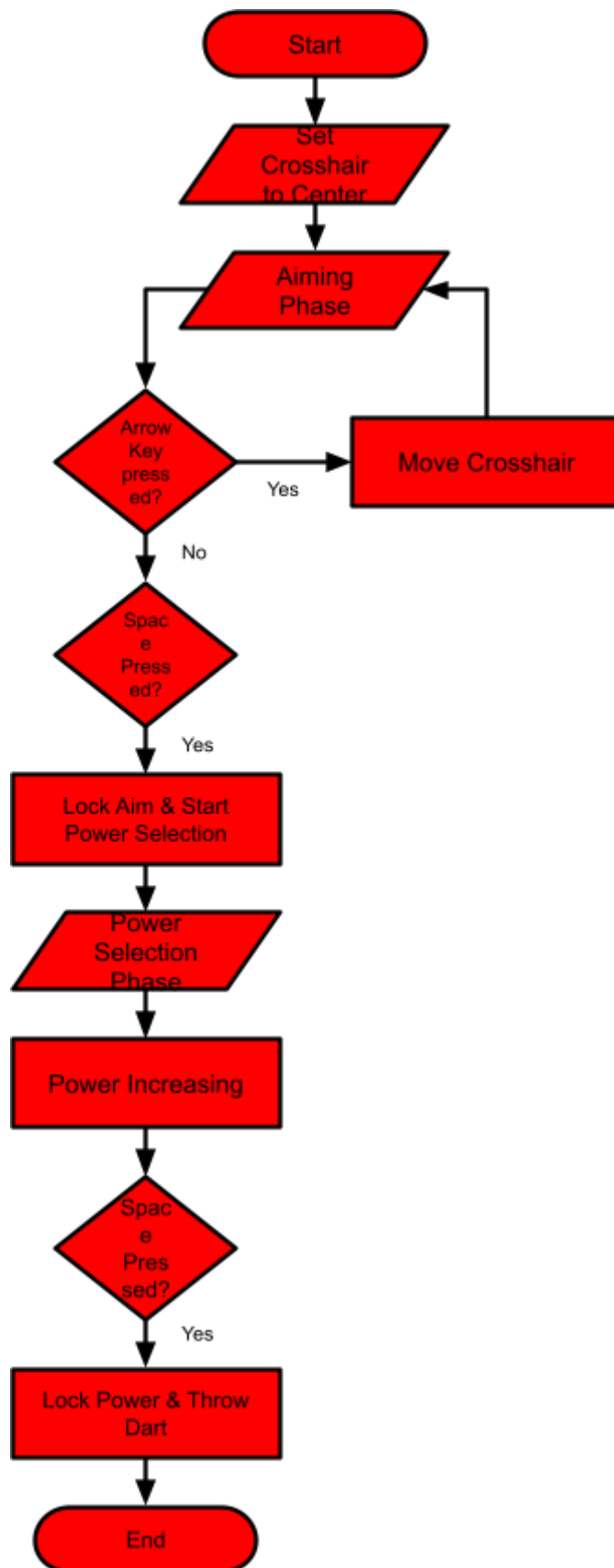
None

```
PROCEDURE handle_turns(player1, player2)
```

```
    WHILE game_not_over() DO
```

```
    IF current_turn = player1 THEN  
        CALL player1_throw()  
        current_turn ← player2  
    ELSE  
        CALL AI_throw()  
        current_turn ← player1  
    ENDIF  
ENDWHILE  
CALL display_winner()  
END PROCEDURE
```

Dart Throwing Mechanics Algorithm



Implements a crosshair-based aiming system using keyboard controls and power selection. Players move a crosshair over the dartboard using the arrow keys. Pressing space locks the aim and starts a power selection phase, where the power cycles and is confirmed with another space press. The power will be determined in a more sophisticated algorithm with the target selection which will be further researched. This algorithm is needed so that the user can decide where they want to throw the dart and then be able to throw it, handled by a separate algorithm called within this one.

None

```
PROCEDURE aim_and_throw_dart():

    crosshair_x, crosshair_y = BOARD_CENTER_X, BOARD_CENTER_Y # Start
    at the center

    power = 0

    aiming = True

    power_selecting = False

    WHILE aiming == True DO

        FOR event in events DO

            IF event.type == KEYDOWN THEN

                IF event.key == LEFT THEN

                    crosshair_x -= CROSSHAIR_SPEED # Move left

                ELIF event.key == RIGHT THEN

                    crosshair_x += CROSSHAIR_SPEED # Move right

                ELIF event.key == UP THEN
```

```

        crosshair_y -= CROSSHAIR_SPEED # Move up

    ELIF event.key == DOWN THEN

        crosshair_y += CROSSHAIR_SPEED # Move down

    ELIF event.key == SPACE THEN

        aiming = False # Lock aim and start power
selection

    power_selecting = True

    WHILE power_selecting == True DO

        FOR event in events DO

            IF event.type == KEYDOWN AND event.key == SPACE THEN

                power_selecting = False # Lock power and throw dart

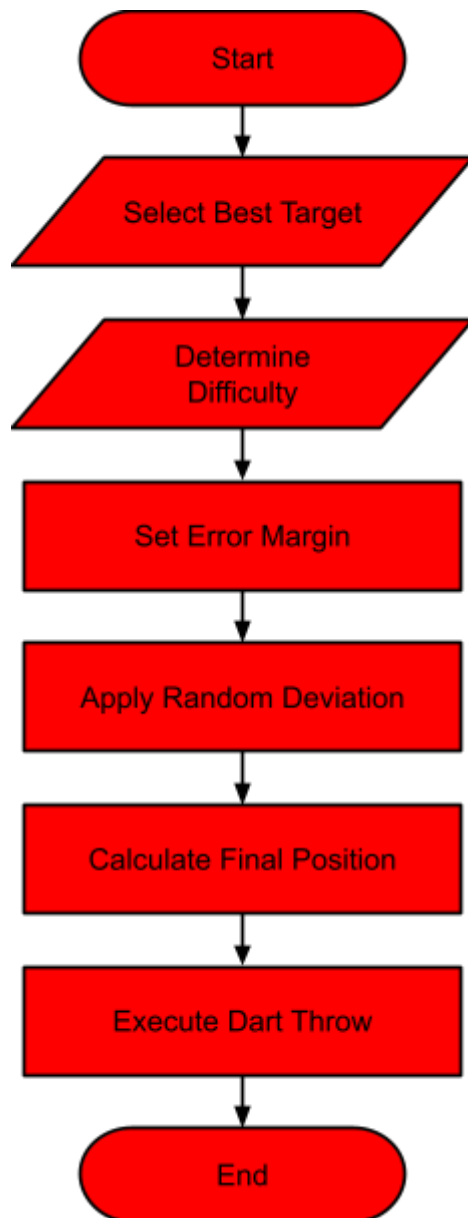
            power = (power + 1) % 100 # Cycle power 1-100 continuously

            time.delay(50)

        throw_dart(power, crosshair_x, crosshair_y)

```

AI Decision-making Algorithm



The AI opponent determines its throws based on predefined difficulty levels. The AI selects a target on the board and applies a difficulty-based deviation amount, with harder difficulties having smaller deviations. This ensures that users can play against AI's of different skill levels to cater to their own experience, allowing for an enjoyable experience for all players.

None

```
FUNCTION ai_throw(difficulty)

    target_x, target_y = select_best_target()

    error_factor = {

        "Beginner": 15,

        "Intermediate": 8,

        "Advanced": 3

    }[difficulty]

    deviation_x = random.uniform(-error_factor, error_factor)

    deviation_y = random.uniform(-error_factor, error_factor)

    final_x = target_x + deviation_x

    final_y = target_y + deviation_y

    execute_dart_throw(final_x, final_y)
```

Score Calculation Algorithm

This algorithm calculates the score for a player's throw, ensuring that it follows the official darts rules. This ensures that scores are calculated correctly based on where the dart lands, following standard scoring rules.

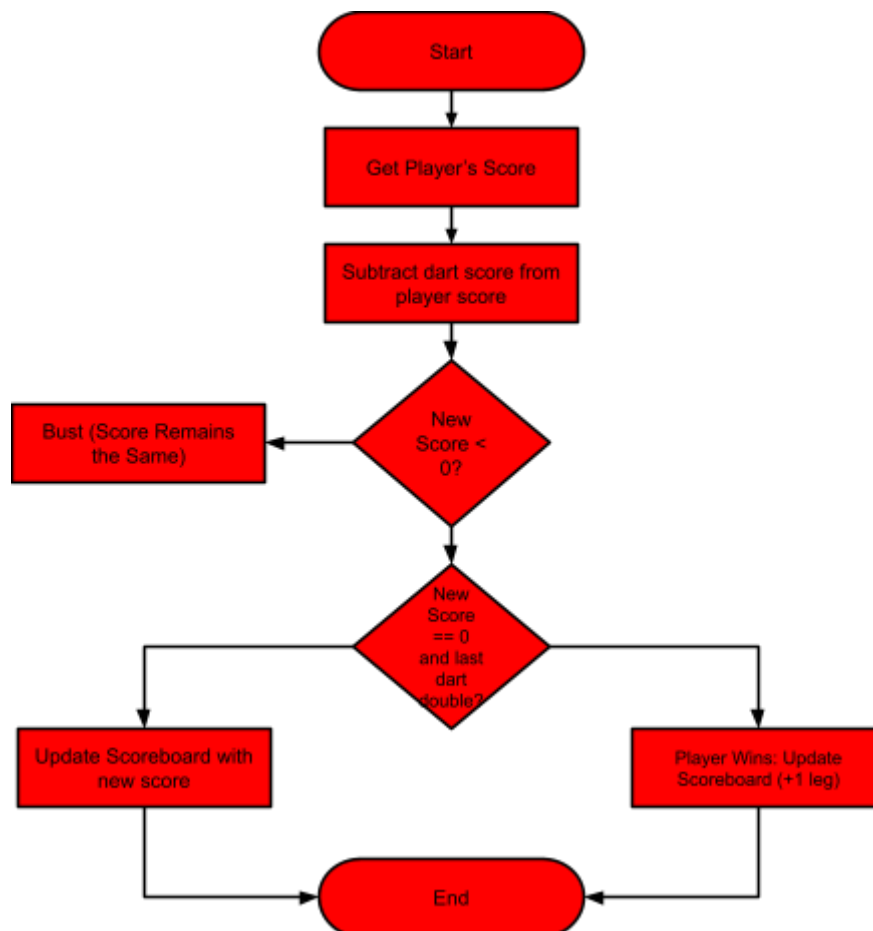
None

```
FUNCTION calculate_score(hit_section, multiplier)
```

```
    base_score = dartboard_values[hit_section]
```

```
    return base_score * multiplier
```

Score Updating Algorithm



The game follows standard darts rules, decrementing from 501 and requiring a double-out to win. The player's score is reduced by the dart's value. If they go

below zero, their score stays the same (bust rule). To win, they must land exactly on zero with a double. The algorithm ensures scores are correctly updated to ensure fairness.

None

```
FUNCTION update_score(player, dart_score)

    new_score = player.score - dart_score

    IF new_score < 0 THEN

        return # Bust rule applied, score remains the same

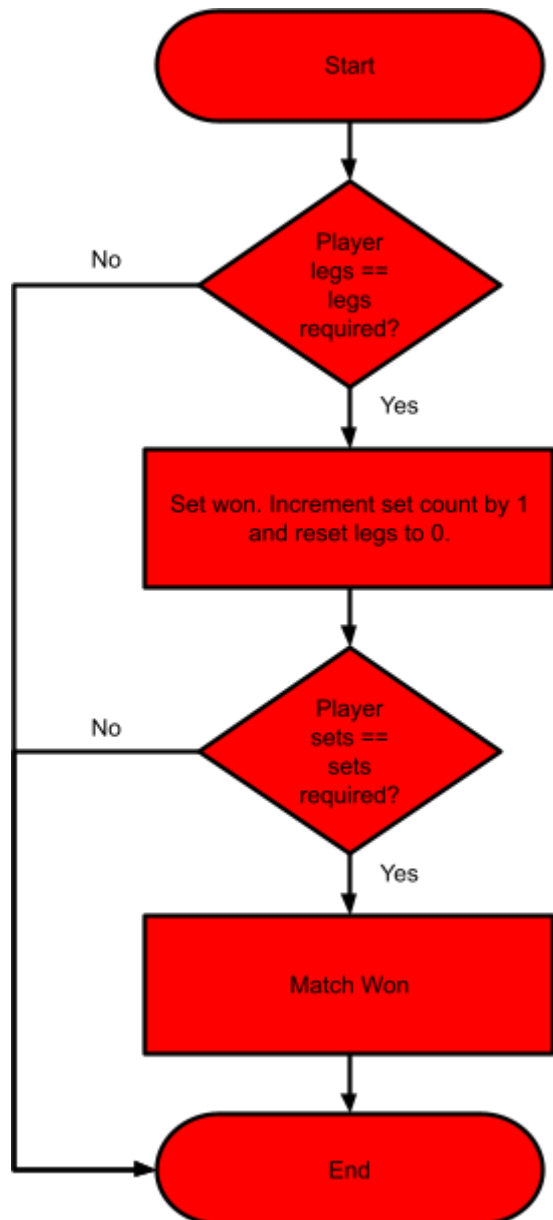
    IF new_score == 0 and last_throw_was_double() THEN

        declare_winner(player)

    ELSE DO

        player.score = new_score
```

Game End Algorithm



Implements a system where a player must win the required number of legs to win a set, and the required number of sets to win the match.

None

```
FUNCTION    check_match_winner(player,    opponent,    legs_required,
           sets_required)
```

```
    IF player.legs_won >= legs_required THEN
```

```
    player.sets_won += 1

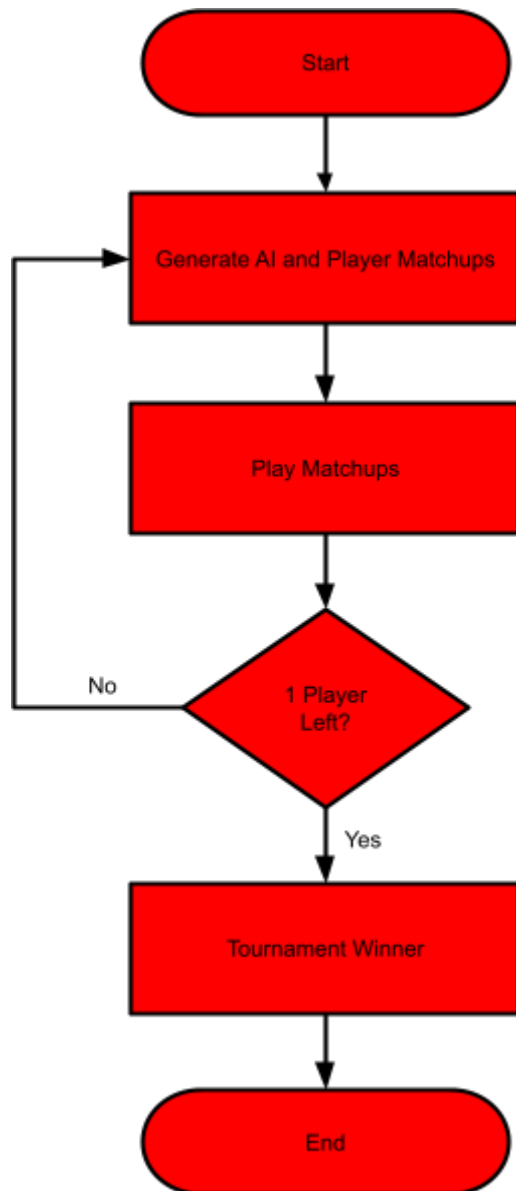
    player.legs_won = 0
    opponent.legs_won = 0

    IF player.sets_won >= sets_required THEN

        return True # Player wins match

    return False # Match continues
```

Tournament Progression Algorithm



In tournament mode, the player competes against AI opponents in a knockout-style format. The tournament consists of AI-controlled opponents, and the player competes through knockout rounds, until one player is left.

None

```
PROCEDURE run_tournament(player)
```

```
    ai_opponents = generate_ai_opponents()
```

```

players = [player] + ai_opponents # Player and AI competitors

WHILE len(players) > 1 DO
    winners = []

    FOR i in range(0, len(players), 2) DO
        winner = play_match(players[i], players[i+1]) # Simulate
match
        winners.append(winner)

    players = winners # Move winners to the next round

declare_winner(players[0])

```

Leaderboard Update Algorithm



Tracks and updates player achievements, enhancing competitiveness. Adds the player's score to the leaderboard, whether this is total matches won or achievements gained, sorts it from highest to lowest, and saves the updated leaderboard. Lambda refers to an anonymous function and is just used to put the scores in order.

None

```
PROCEDURE update_leaderboard(player_name, player_score)

    leaderboard = load_leaderboard()

    leaderboard.append((player_name, player_score))

    leaderboard.sort(key=lambda x: x[1], reverse=True)
```

```
save_leaderboard(leaderboard)
```

Conclusion

These algorithms collectively ensure smooth gameplay, balanced AI difficulty, and a structured game flow. Their modular design allows for easy adjustments and scalability, supporting future enhancements and refinements. The feature of having an adaptive AI is still being researched further into, hence why it has not been included amongst the algorithms yet. Not all of the algorithms are final and will need adjusting based on some additional research when implementing my solution in pygame, however they provide insight into how the different modules can be broken down into sub-problems and then be recombined to form a complete solution, as demonstrated by my decomposition using a modular approach.

Entity Relationship Diagram

In order to store permanent data within the game, I will need to use either a file or a database. I am going to use a JSON file most likely as it is easy to maintain and use alongside my python code. Below is an entity-relationship diagram representing how each user and their unique gamertag links to their score. It is a one-to-many relationship, as each user can have multiple overall scores, showing their total matches one, total tournaments won, or total achievements unlocked, all of which will be stored in the file.



Usability Features

After my last interview with (Anonymous), where we came up with a rough outline of the design and presentation of each game state, I have created virtual designs of my ideas for each game state, which I have annotated with justifications of the features in them and why they are beneficial to the user who plays the game.

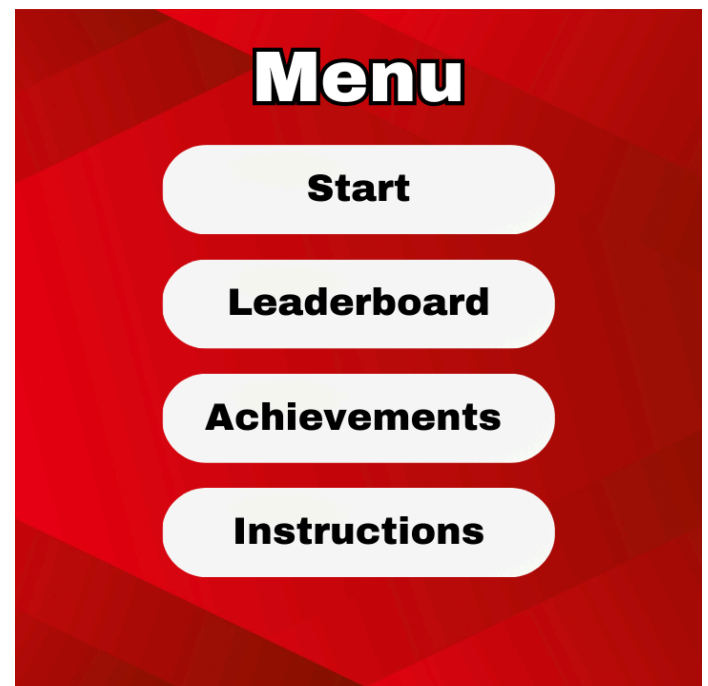
Loading Screen

- The game title is shown at the centre in a clear and easy to understand font.
- Darts to emphasise the darts theme.
- A simplistic theme with the colour red being the main colour palette throughout the game in order to not over complicate the game.



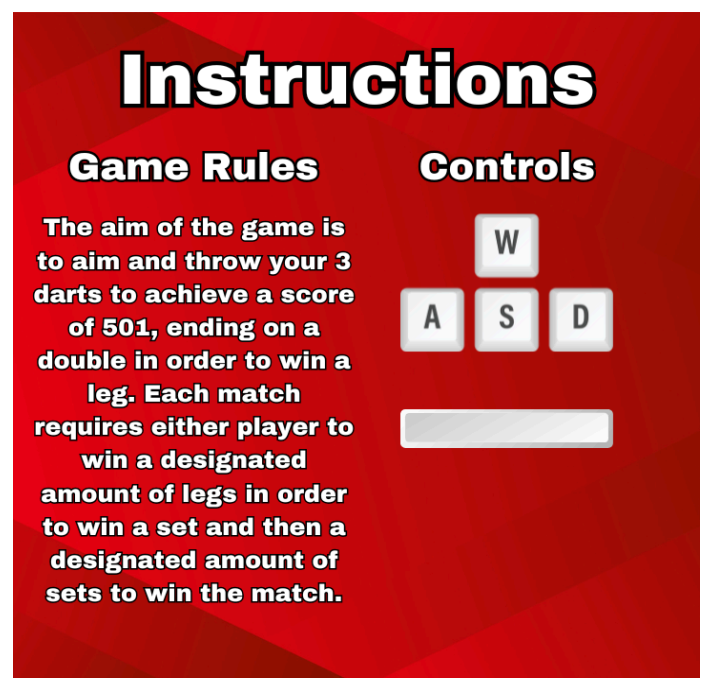
Main Menu

- There are four options on the screen which are separated clearly in a large and clear font to enable the user to quickly see where they would like to navigate. However, I may choose to add a settings or exit option at a later point.
- The user will use WASD by default to select which game state they would like to select and then press space to go to that selected game state. These can be changed in the settings.



Instructions

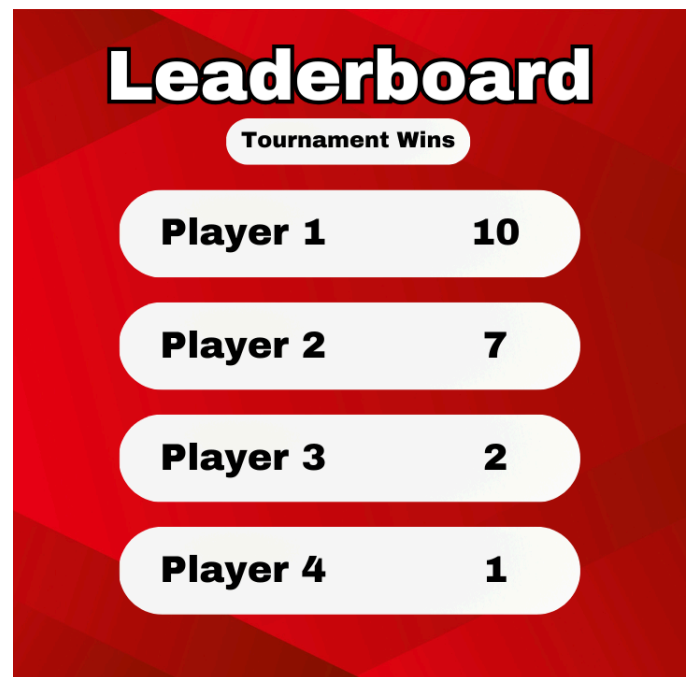
- The title shows the user which game state they are in, in a clear font that is easy to read and understand.



- The menu is split into two sections to provide clarity, game rules and controls.
- The right side has the default navigation controls for movement around the menus.
- The left has the game rules whilst playing the actual game, and may also include gameplay controls for better clarity.

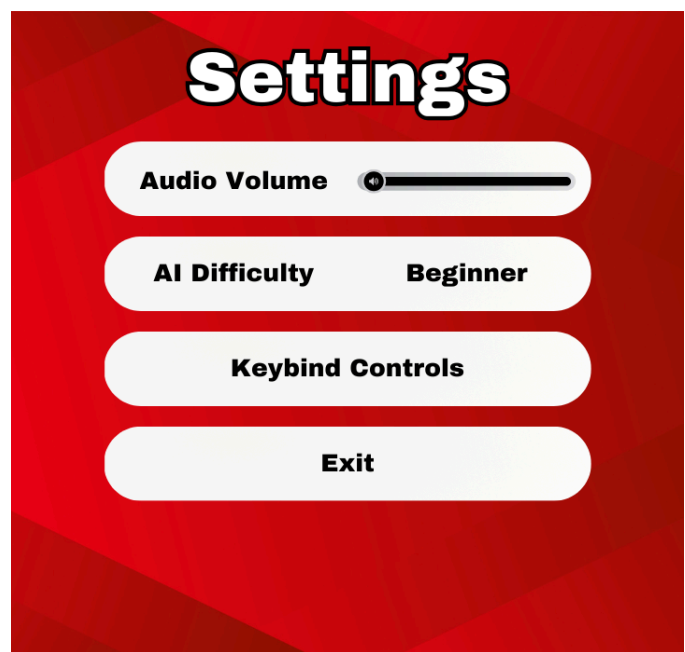
Leaderboard

- The title shows the user which game state they are in, in a clear font that is easy to read and understand.
- Each position will be ranked by the players score so it follows the usual conventions for leaderboards.
- The display will contain both the gamertag, to identify the user and their score, to show how many “points” they have, depending on the category of ranking.



Settings

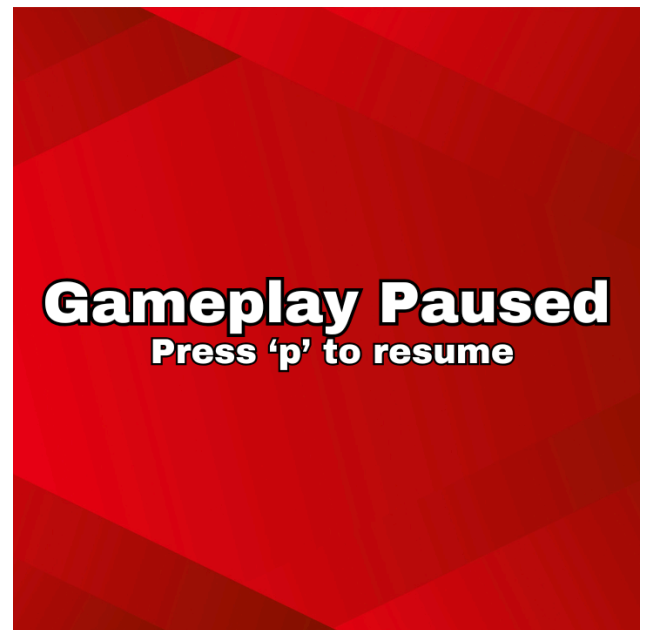
- The title shows the user which game state they are in, in a clear font that is easy to read and understand.



- This page allows the user to customise particular settings to fit their needs, whether this may be volume, the AI difficulty, or their controls.
- More settings may be added during development and testing in order to ensure different options for users.

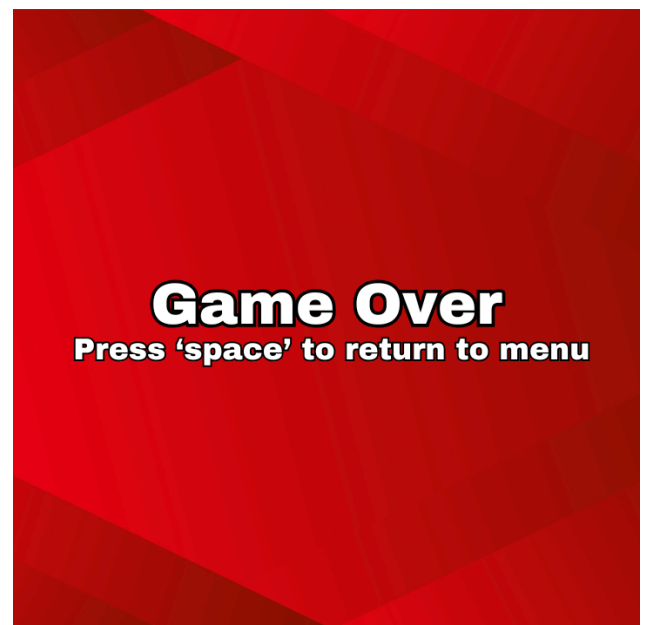
Pause Screen

- There is a big heading in the centre of the screen to show the user that they have paused the game.
- The only instruction on this screen will be to resume the game so it enables the user to navigate easily, clearly stating how to resume gameplay.



Game Over

- There are two messages on the screen at this point to signify that the user has reached the end of the game, and how to return to the menu.
- The limited instructions eliminate any clutter on the screen and make it clear



to the user that the game is over and they can return to the menu.

Achievements

- The achievement trophies/rewards are gold once achieved and grey when not yet achieved.
- Each achievement will provide the user a point on the achievements leaderboard to create a sense of competition.
- The user can use 'a' and 'd' to navigate through the achievements.
- The name of the achievement is shown below the reward/trophy, and for win based achievements, the number of times that the tournament has been won is shown in the centre of the reward.



Playing the game

Scoring box

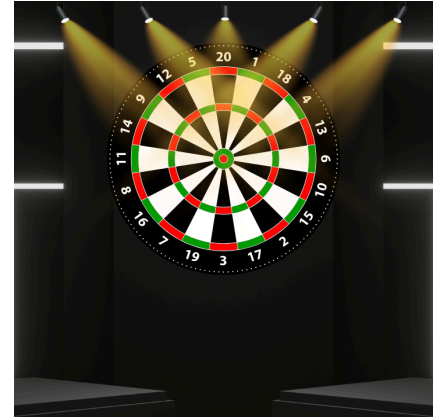
This will be placed in the bottom right, much like in real darting events, as it is not in the way of the dartboard but still easy to look at and check the score during the game. It will be simplified into legs, sets, and scores remaining in the current leg, so the user can quickly see the different scores and how they are doing to tell whether they are about to win or lose the game. The design can be shown below.

Final - First to 3 Sets		Sets	Legs
Player 1	1	1	367
Player 2	0	1	501

Backgrounds

There are 3 different game backgrounds which depict different dart venues, so that the user can individualise the game and have some customisation to keep the game feeling fresh. There will be clear instructions for what to press in order to switch between each background which will help the user navigate the game more easily.

The mockup designs for each background are shown below, but may undergo tweaks for aesthetic purposes during development.



Main Variables and Data Structures in the Solution

Method	Name	Data Type	Explanation	Justification
Menu				
Variable	event.key	string	Stores the key pressed by the user	This is needed so that the user can navigate the menus

Gamertag				
Variable	gamertag	string	Stores the user's gamertag	This will allow for scores to be saved to the leaderboard file.
Function	validate_gamertag		This will check the length of the username and its characters	This will stop invalid input being entered into the system and ensures gamertag is entered correctly
Procedure	save_gamertag		This will save the gamertag to the file	This ensures that the gamertag is existing when a score is added
Pick Background				
Image	background_1	.jpeg	These are the image files for the 3 background options that the user can select	This will allow the player to individualise the game to whatever they like best
Image	background_2	.jpeg		
Image	background_3	.jpeg		
Function	get_background		This will get the correct background based on user input	This ensures that the user can select their desired background for playing
AI				

Variable	ai_difficulty	string	Stores the current AI difficulty level	This affects how the AI performs and aims
Variable	player2.name	string	Randomly generated AI name stored as the second player's name	This adds immersion and variety to the AI opponent so you aren't constantly playing "Opponent"
Function	ai_select_target		Determines where the AI should aim based on current score and difficulty. Includes logic to finish on a double	This helps to ensure that the AI follows proper darts logic and makes the game more engaging
Dart Movement				
Procedure	animate_dart_throw		This will ensure that the dart is constantly moving on a fixed path	This will enable the user to move their dart to aim at their desired location
Variable	running	boolean	This will keep track if the game is running or not	This will ensure that the player can move their dart whilst the game is running

Variable	key	string	This will store the key input by the user	This ensures that the player can move and throw their darts
Function	simulate_dart_throw		This will allow the user to throw their dart wherever they are currently aiming, based on power	This ensures that the user can throw their darts at the board
Variable	power	integer	This stores the power which varies when preparing to throw a dart	This will ensure variation when throwing darts to add an element of skill and add deviation.
Dart Score				
Function	get_dart_score		This will get the score of the dart based on where it lands on the board	This ensures that the score is accurately updated according to the score achieved
Variable	score	integer	This will store the score that the user achieves with their darts	This allows correct calculation of score thrown and remaining
Variable	base_score	integer	This will store the	This helps with

			score before any multiplier is added for a triple or double	calculation and calibration of the scoring.
Pause				
Variable	paused	boolean	This stores the current state of the pause menu	This ensures that the computer can keep track whether the program is paused
Function	pause_game		This will pause the game	This ensures that the game stops wherever it is and changes to the pause screen
Scoring				
Function	simulate_dart_throw		This function also will help to decrement the scores according to the user's dart score	This ensure accurate calculation and scoring of the darts
Variable	p.sets	integer	This will hold the number of legs that a player has won	This means that the sets score can increment and a winner can be checked for
Variable	p.legs	integer	This will be used	This ensures that

			increment the legs count based on who wins the leg and also holds the number of legs a player has won	the legs are accurately updated based on whoever wins and can be used to check if a set has been won
Procedure	Display_text_popup or display_text_and_flash_popup		Displays text popup if a player wins a leg or set, or flash alongside it	This adds an extra visual aspect to the game and makes it clear who wins a leg and when it happens
Variable	turn_start_score	integer	Stores the score of the player at the start of their turn. Exists for both players	This allows for scores to be reset if a bust occurs and for better tracking.
Leaderboard				
Procedure	display_leaderboard		This will display the leaderboard	This means the user can see if they rank in the top 10 players
Achievements				
Procedure	display_achievements		This will display the achievement state to the user	This will allow the user to view their achievements and see if they have

				unlocked any more
Image	gold_achievement	.jpeg	This is the image for an achievement that has been unlocked	This provides a visual to the user whether they are doing well in the achievements side of the game and provides competition
Image	grey_achievement	.jpeg	This is the image for an achievement that has not yet been unlocked	
Instructions				
Procedure	display_instructions		This will swap to the screen to display the instructions to the user	This informs the user the rules of the game and also how they work
Settings				
Procedure	display_settings		This will display the settings to the user	This ensures that the user can view the setting option and can change them
Variable	setting_selected	string	This will determine the state of a setting, whether it may be on or off, or a	This ensures that the user can change their experience to fit their needs/liking

			particularly difficulty setting	
--	--	--	------------------------------------	--

Test Data

Following the decomposition of my problem, I determined the different aspects of my game which would require data to be input, either through user input, or a function/procedure and I have devised test tables so that I can test the inputs and outputs to ensure that the program is robust and correctly functioning.

During Development / Iterative Testing

Test Data	Type	Justification	Expected Output
Verify Gamertag			
'helloworld'	valid	This will check whether the user's gamertag that they input is between 3-12 characters and contains only letters and/or numbers	Gamertag Accepted
'87654321891375'	valid		Gamertag Accepted
'liv'	boundary		Gamertag Accepted
'hi'	invalid		Invalid Gamertag

Test Data	Type	Justification	Expected Output
get_background			
1	boundary	This will check that the input entered is between 1 and 3 inclusive, in order to select a game background	Design Selected
2	valid		Design Selected
'hi'	erroneous		Re-ask Question
123	invalid		Re-ask Question
Dart movement			
Up arrow key	valid	This will ensure that the movement of the dart will only happen if a correct key is pressed	Crosshair moves up
'hi'	invalid		Nothing changes
123	erroneous		Nothing changes
Pause game			
esc	valid	This will ensure that the pause screen is only displayed if the user presses 'esc'	Pause screen opened
a	invalid		Nothing changes
pp	valid		Pause screen is opened and then closed again
123	erroneous		Nothing changes
Menu Navigation			
Up arrow key	valid	This will ensure	The highlighted

Test Data	Type	Justification	Expected Output
		game states are only changed if the correct keys are pressed (unless mapped otherwise)	option is now the one above
Down arrow key	valid		The highlighted option is now the one below
Right arrow key	invalid		Nothing changes
'enter'	valid		The option currently highlighted will be selected and displayed
123	erroneous		Nothing changes

Post Development Testing

Number	Success Criteria	Input	Expected Output
1.1	Menu screen is displayed		Menu screen is displayed upon launch
1.2.1	Pressing 'up' highlights option above	'w'	Game moves highlight option up
1.2.2	Pressing 'down' highlights option below	's'	Game moves highlight

			option down
1.2.3	Pressing 'enter' selects highlighted option	'enter'	Games moves to selected option e.g. settings screen
1.3	Pressing back button returns to main menu	'return'	Game returns to main menu
2.1.1	Pressing 'up' moves dart up	Up arrow key	Dart aim moved up one speed unit
2.1.2	Pressing 'left' moves dart left	Left arrow key	Dart aim moved left one speed unit
2.1.3	Pressing 'down' moves dart down	Down arrow key	Dart aim moved down one speed unit
2.1.4	Pressing 'right' moves dart right	Right arrow key	Dart aim moved right one speed unit
2.2.1	Pressing 'space' starts power selection	'space'	Dart navigation locked and power can be selected
2.2.2	Power bar is shown		Power bar shown on screen
2.2.3	Releasing 'space' confirms power and throws dart	'space'	Power is locked and dart is thrown
2.3.1	Dart moves on screen and lands in correct position as per user input and logical		Dart moves on the screen after is has been thrown and lands

	pathing		in a logical position
2.4.1	User can pause gameplay by pressing 'esc'	'esc'	Gameplay is paused
2.4.2	Pause screen displayed		Pause screen is displayed and visible to the user
2.4.2	Pressing resume unpauses gameplay	Click resume button	Gameplay is resumed and pause screen no longer visible
3.1.1	Score must correctly update		Score in the scoreboard will correctly update based on dart scores
3.1.2	Score box must be designed		Score box can be seen on the screen
3.2.1	Players must win a leg by finishing on a double	A double e.g. double 10 etc	Game only ends if a player ends on a double, else bust or score correctly updated
3.2.2	Score correctly updated upon leg completion	1	1 is added to the leg count of the winner of the current leg
3.3.1	Score box displayed on screen in bottom right		Score box is in the bottom right handle corner of the display
3.3.2	Leg count displayed		Leg count is visible

	correctly		and correct in the scoreboard
3.3.3	Set count displayed correctly		Set count is visible and correct in the scoreboard
3.3.4	Player names displayed		Scoreboard displayed gamertags of both players
4.1.1	AI Computer moves dart across board		Dart moves across the board towards target
4.1.2	AI follows basic decision-making		Easy AI more likely to miss targets, Advanced AI more precise
4.1.3	AI Computer has a name		AI player gamertag displayed on the scoreboard
4.2.1	User can select AI difficulty in settings		AI difficulty can be changed by the user in settings and updated correctly
4.2.2	Default AI difficulty is Beginner		Default AI upon game loading is Beginner
4.3.1	Beginner AI misses more targets		Less frequently hitting high scores or big checkouts, and misses more numbers

4.3.2	Advanced AI misses less targets		More frequently hitting high scores or big checkouts, and doesn't miss as many numbers
5.1.1	Game must check if a checkout has been achieved		If a checkout has been achieved, game must end/leg won
5.2.1	Victory/Game over screen displayed when a user wins		After a game has been completed, game over screen displayed to the user
6.1.1	Achievements displayed are grey until unlocked	Trophy unlocked variable is false	Achievement displayed grey
6.1.2	Achievements displayed are gold/yellow once unlocked	Trophy unlocked variable is passed as true	Achievement displayed yellow/gold
6.1.3	Achievements appearance has been designed		Trophies can be seen on the screen
6.2.1	Leaderboard displays correctly		Leaderboard can be seen on screen and is correctly ordered
6.2.2	User can switch between	Left or right	Leaderboard switched

	different leaderboards	arrow keys	upon user input
6.3.1	Each gamertag is unique	String of characters	No gamertag is the same
6.3.2	Gamertags are displayed on the leaderboard		Gamertags are visible on the leaderboard screen next to scores
7.1.1	Sound upon dart hitting the board		Sound played when dart hits board
7.1.2	Sound upon selecting and navigating menu options		Sound played when menu is navigated
7.1.3	Sound upon game completion		Sound played when game is over
8.1.1	Settings menu correctly displayed		Settings menu displayed to user
8.1.2	User can change audio levels	Left or right arrow keys	Audio goes up or down depending on user input
8.2.1	AI difficulty can be adjusted in settings	Left or right arrow keys	Difficulty increases or decreases based on user input
8.3.1	Background can be selected	Left or right arrow keys	Background selected
8.3.2	Background correctly displayed and updated		Background displayed to user in game
9.1.1	Game runs at a minimum		Game runs at 30+

	of 30 FPS		FPS
9.2.1	Game runs smoothly and does not crash		Game runs smoothly without stutters and does not crash

Development

Development Plan

Following the design of my game, I have decided to first create a plan for development, which is split into 12 main iterations. By doing this, it enables me to best manage my time when developing the game, which ensures I can finish within the given time frame, whilst maintaining the high quality and accuracy I desire, and making sure I develop one module at a time. Each week I will work on a new iteration and the following week, will be also used to test the iteration from the previous week as well, to ensure the game is running smoothly or in case any updates are needed to the code.

Iteration	Module	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12
1	Main Menu Navigation												
2	Gamertag Input & Validation												
3	Scoreboard + Calculation												
4	Dart Throwing System												
5	Game Loop & Turn System												
6	AI Opponent Game Logic												
7	Leaderboard System												
8	Tournament Mode												
9	Achievements System												
10	Pausing + Instructions												
11	Sound Effects												
12	Settings + Background Switching												

Iterative Development Log

This section documents the iterative development process of my game, following the development plan I created. Each iteration builds on the previous and includes the date of which I first started the iteration, testing and refinements, as well as a brief description, justification of the choices made, and code evidence of the iteration. The length of time for each iteration will vary as some iterations are more difficult than others. I will attempt to use a modularised approach so that each iteration can be used as a building block for the next iteration, however this may vary at some points where iterations are needed alongside each other to develop a product.

Week 1 - Iteration 1 - Main Menu Navigation

Description

I built a basic menu with 5 options: Start, Settings, Leaderboard, Achievements, and Exit. I programmed the menu to enable arrow key navigation, using enter to select an option. The highlighted option should be red and when enter is clicked on the highlighted option, a message should be displayed in the output console to indicate success. When enter is pressed on exit, the game should exit correctly.

Justification

Creating a main menu is the first essential thing needed in a game to provide the user with UI interaction. It is the base for branching into all other features such as starting a game, checking the leaderboard, or exiting the game. However I have decided to make sure the UI is not too cluttered as highlighted as a comment in my survey conducted in the analysis section.

Code Evidence

Python

```
import pygame
from menus import main_menu
import sys

def main():
    # Initialise pygame
    pygame.init()

    # Create the game window (720x720)
    screen = pygame.display.set_mode((720, 720))
    pygame.display.set_caption("Precision Darts")

    # Call into the main menu loop
    while True:
        result = main_menu(screen)
        if result == "EXIT":
            pygame.quit()
            break

    # Run the program only if this file is executed directly
    if __name__ == "__main__":
        try:
            main()
        except SystemExit:
            pygame.quit()
            sys.exit()
```

Python

#menus.py

```
import pygame
```

```
import sys
```

```
import os
```

```
# Define common colours used in the menu
```

```
BLACK = (0, 0, 0)
```

```
RED = (255, 0, 0)
```

```
FONT_NAME = "freesansbold.ttf"
```

```
# Set the path to the menu background image
```

```
BG_PATH = os.path.join("assets", "main_menu.png")
```

```
# Function to draw a single menu option with optional highlight
```

```
def draw_text(surface, text, size, x, y, selected=False):
```

```
    font = pygame.font.Font(FONT_NAME, size) # Load font
```

```
    color = RED if selected else BLACK # Use highlight color if  
selected
```

```
    text_surface = font.render(text, True, color) # Render Text
```

```
    text_rect = text_surface.get_rect(center=(x, y)) # Center it at the  
given position
```

```
    surface.blit(text_surface, text_rect) #Draw it to the screen
```

```
# Main menu loop function
```

```
def main_menu(screen):
```

```
    clock = pygame.time.Clock()
```

```
    options = ["Start", "Leaderboard", "Achievements", "Settings",  
"Exit"]
```

```
    selected_index = 0 # Starts with first choice selected
```

```
# Load and scale the menu background image to fit the screen
```

```
background = pygame.image.load(BG_PATH)
```

```

background = pygame.transform.scale(background, (720, 720))

# Game loop for the menu screen
while True:
    screen.blit(background, (0, 0))

    # Handle events (like key presses or closing the window)
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()

        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_UP:
                selected_index = (selected_index - 1) %
len(options) # Move up in menu
            elif event.key == pygame.K_DOWN:
                selected_index = (selected_index + 1) %
len(options) # Move down in menu
            elif event.key == pygame.K_RETURN:
                selected = options[selected_index] # Get selected
option

                # Handle option logic
                if selected == "Exit":
                    pygame.quit()
                    sys.exit()
                else:
                    print(f"Selected: {selected}") # Placeholder
for future screens

    # Draw all menu options with highlights
    for i, option in enumerate(options):

```

```
draw_text(screen, option, 40, 360, 165 + i * 120,  
selected=(i == selected_index))  
  
pygame.display.flip() # Update the Display  
clock.tick(60) # Limit to 60 FPS
```

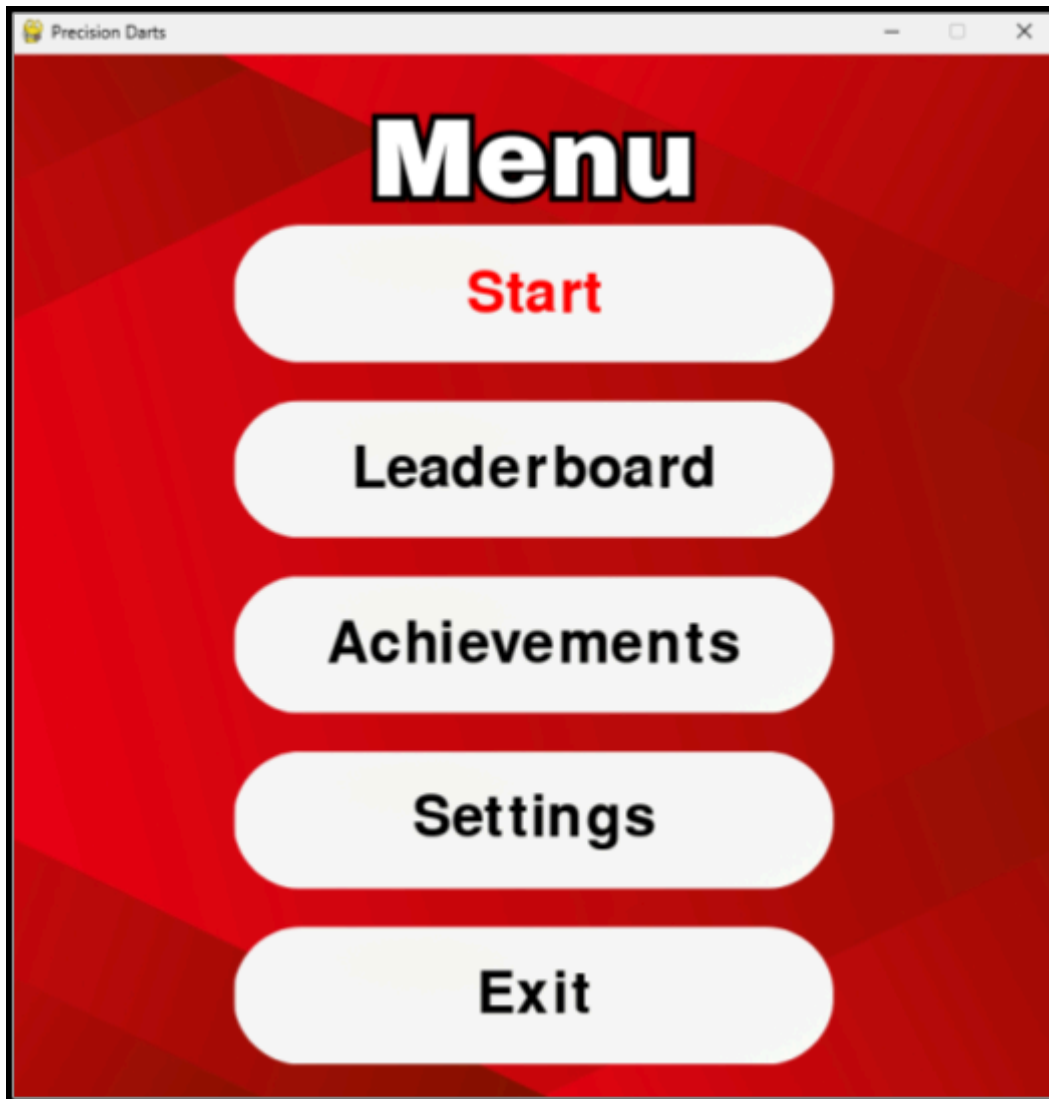
Test Summary

- **TC01:** Launching the game → menu appears on background ✓
- **TC02:** Press up or down arrow key → highlighted option moves up or down respectively ✓
- **TC03:** Press Enter on the exit option → game exits correctly ✓
- **TC04:** Press Enter on another option → message prints to indicate selected option ✓

Issues and Fixes

I faced a simple but critical error at first, as my assets for the game were in a completely different place to my game and so when I tried to run the game, the assets were unable to be found. To fix this, I simply just created a structure of folders with my assets in and put them in the same place as my program to simplify things down.

First Look at the Menu



The game name is displayed in the top UI bar. Options are displayed and the highlighted option is in red to highlight to the user more easily.

Week 2 - Iteration 2 - Gamertag Input & Validation

Description

Players input a gamertag, which is validated (3-12 characters, alphanumeric, and unique). If one of the criteria is not met, an error message is displayed until the user correctly enters a valid gamertag. This is then saved to a json file full of gamertags so that it can be stored across the database to ensure no gamertag is reused.

Justification

Validating a player's gamertag ensures that the user identity is unique and traceable across gameplay, leaderboard tracking, and achievements. Having a gamertag also makes each player's experience unique as highlighted by (Anonymous) in my stakeholder interview and It is a good aspect to have in the game.

Code Evidence

Python

```
#gamertag_screen() in menus.py

# Gamertag Login / Registration Screen

def gamertag_screen(screen):

    global current_gamertag

    clock = pygame.time.Clock()

    font = pygame.font.Font("freesansbold.ttf", 32)

    input_box = pygame.Rect(210, 360, 300, 50)

    gamertag = ""

    password = ""

    error_msg = ""

    entering_password = False
```

```

login_mode = False

accounts = get_all_gamertags()

# Load background or fallback
bg_path = os.path.join("assets", "blank_background.png")

try:
    background = pygame.image.load(bg_path)
    background = pygame.transform.scale(background, (720, 720))
except Exception:
    background = pygame.Surface((720, 720))
    background.fill((30, 30, 30))

while True:
    screen.blit(background, (0, 0))

    # Display prompt depending on stage
    prompt_text = "Enter your gamertag:" if not entering_password
else "Enter your password:"

    prompt = font.render(prompt_text, True, (255, 255, 255))
    screen.blit(prompt, (180, 280))

```

```

# Mask password input

display_value = "*" * len(password) if entering_password else
gamertag

txt_surface = font.render(display_value, True, (255, 255, 255))

screen.blit(txt_surface, (input_box.x + 10, input_box.y + 10))

pygame.draw.rect(screen, (255, 255, 255), input_box, 2)


# Show error messages (orange text)

if error_msg:

    error_surface = font.render(error_msg, True, (255, 165, 0))

    screen.blit(error_surface, (100, 440))


pygame.display.flip()


# Handle events

for event in pygame.event.get():

    if event.type == pygame.QUIT:

        return None

    if event.type == pygame.KEYDOWN:

        if event.key == pygame.K_RETURN:

            if not entering_password:

                # If gamertag exists -> login mode

```

```

        if gamertag.lower() in accounts:

            entering_password = True

            login_mode = True

            error_msg = ""

        else:

            # Else -> validate new gamertag

            valid, message =
validate_gamertag(gamertag)

            if valid:

                entering_password = True

                login_mode = False

                error_msg = ""

            else:

                error_msg = message

        else:

            if login_mode:

                # Authenticate login attempt

                if accounts.get(gamertag.lower(), "") ==
hash_password(password):

                    current_gamertag = gamertag

                    return gamertag

                error_msg = "Incorrect password."

                password = ""

```

```

        else:

            # Register new account

            save_gamertag(gamertag, password)

            current_gamertag = gamertag

            return gamertag

    elif event.key == pygame.K_BACKSPACE:

        if entering_password:

            password = password[:-1]

        else:

            gamertag = gamertag[:-1]

    elif event.unicode and event.unicode.isprintable():

        if entering_password and len(password) < 12:

            password += event.unicode

        elif not entering_password and len(gamertag) < 12:

            gamertag += event.unicode

    clock.tick(60)

```

Python

#Adjustment to option selection in main_menu()

```

if selected == "Exit":
    pygame.quit()
    sys.exit()
elif selected == "Start":
    player_name = gamertag_screen(screen)
    print(f"Welcome, {player_name}")

```

Python

player.py

```

import re
from leaderboard_data import load_leaderboard, save_leaderboard,
    hash_password

# Validation of gamertag
def validate_gamertag(name, for_registration=True):
    # Validates a new gamertag.
    # Length must be between 3 and 12 characters
    # Only alphanumeric characters allowed
    # Must not already exist (if registering)

    if not (3 <= len(name) <= 12):
        return False, "Gamertag must be 3-12 characters."
    if not re.match(r"^[A-Za-z0-9]+$", name):
        return False, "Only letters and numbers allowed."
    accounts = get_all_gamertags()
    if for_registration and name.lower() in accounts:
        return False, "Gamertag already taken."
    return True, "Valid."

```

```

# Get Existing Gamertags
def get_all_gamertags():
    # Returns dictionary of all gamertags and their password hashes

    lb = load_leaderboard()
    return {name.lower(): data.get("password_hash", "") for name, data
in lb.items()}

# Saving new gamertag
def save_gamertag(name, password):
    # Saves a new gamertag to the leaderboard file.
    # The password is hashed before storage.
    # Default player stats are set to zero.

    lb = load_leaderboard()
    lb[name] = {
        "password_hash": hash_password(password),
        "legs_won_total": 0,
        "tournaments_won": 0,
        "achievements_unlocked": 0
    }
    save_leaderboard(lb)

```

Test Summary

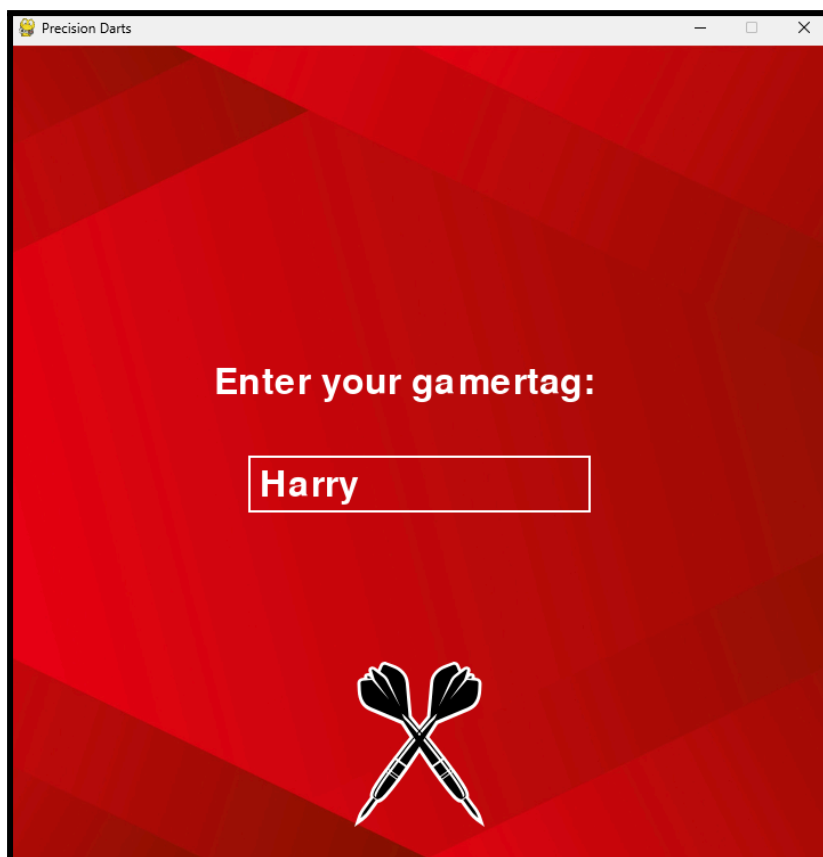
- **TC01:** Reject short name "hm" → Error message ✓
- **TC02:** Reject long name averylonginvalidname123 → Error ✓
- **TC03:** Reject "hm!" → Special character not allowed ✓

- **TC04:** Accept “harry” → Valid ✓
- **TC05:** Re-enter “harry” again → Already taken ✓

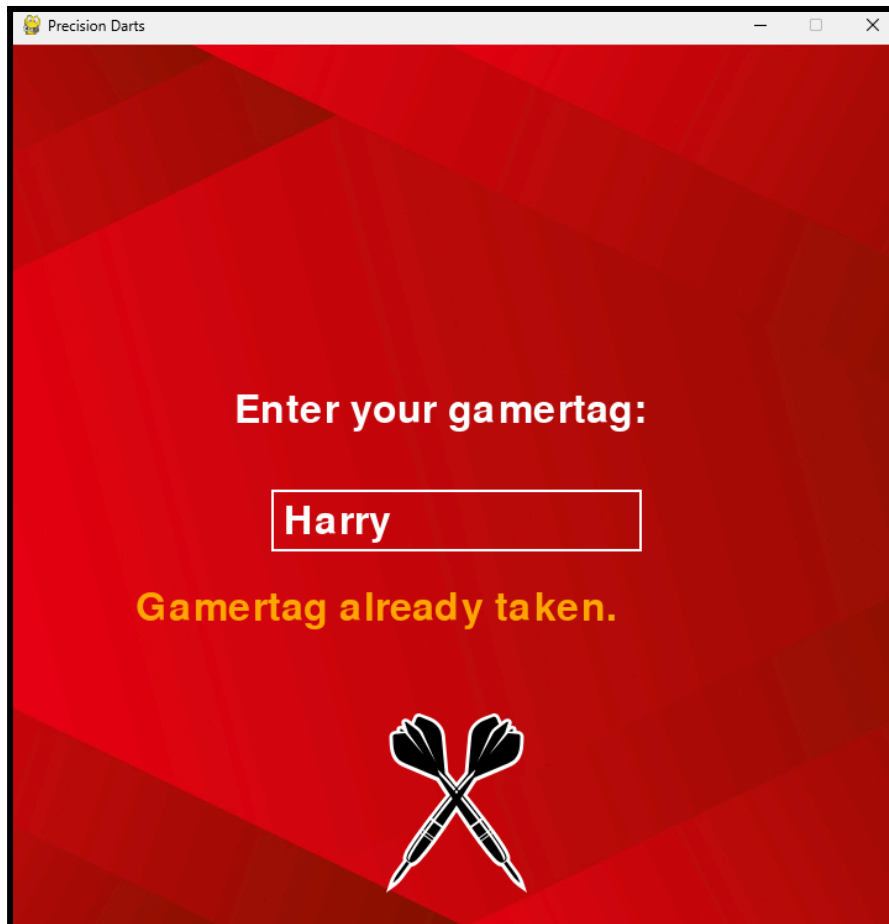
Issues and Fixes

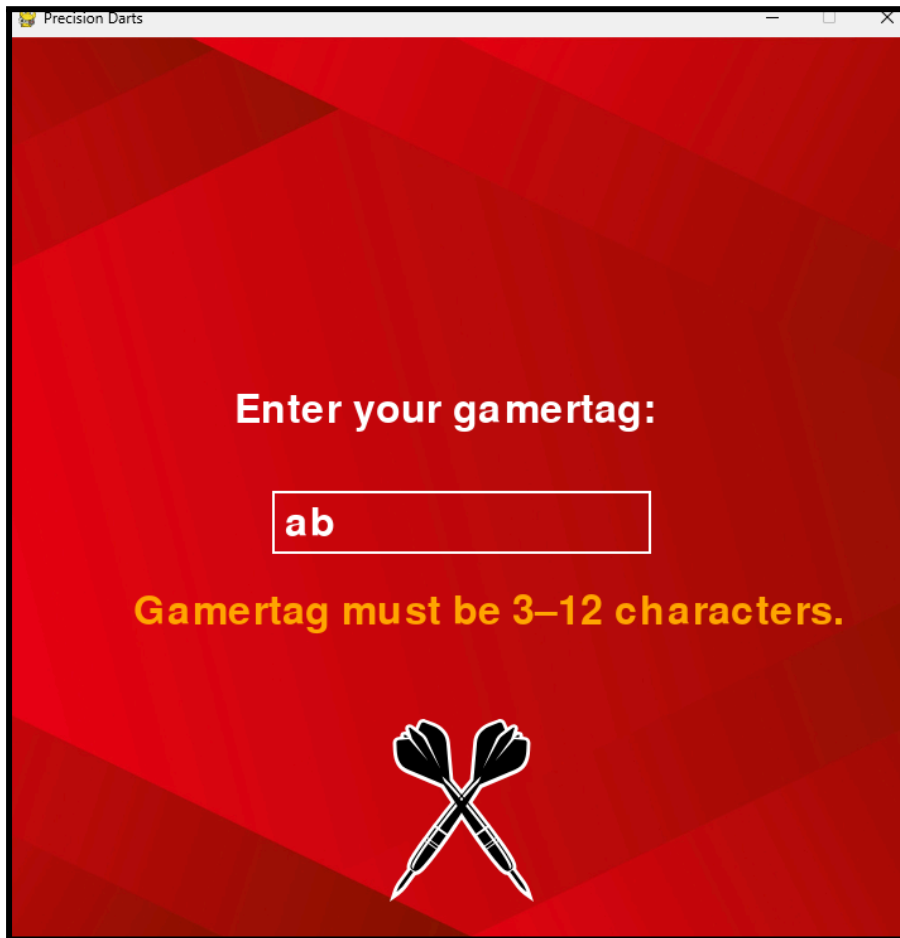
- **Issue:** Gamertag allowed special characters → **Fix:** Added regular expressions logic to ensure it was alphanumeric only
- **Issue:** Duplicate gamertags not blocked → **Fix:** Stored previous names in a JSON file and checked against them to ensure no duplicates
- **Issue:** Input screen was too plain and was not visually consistent → **Fix:** Added a background image for the gamertag screen.

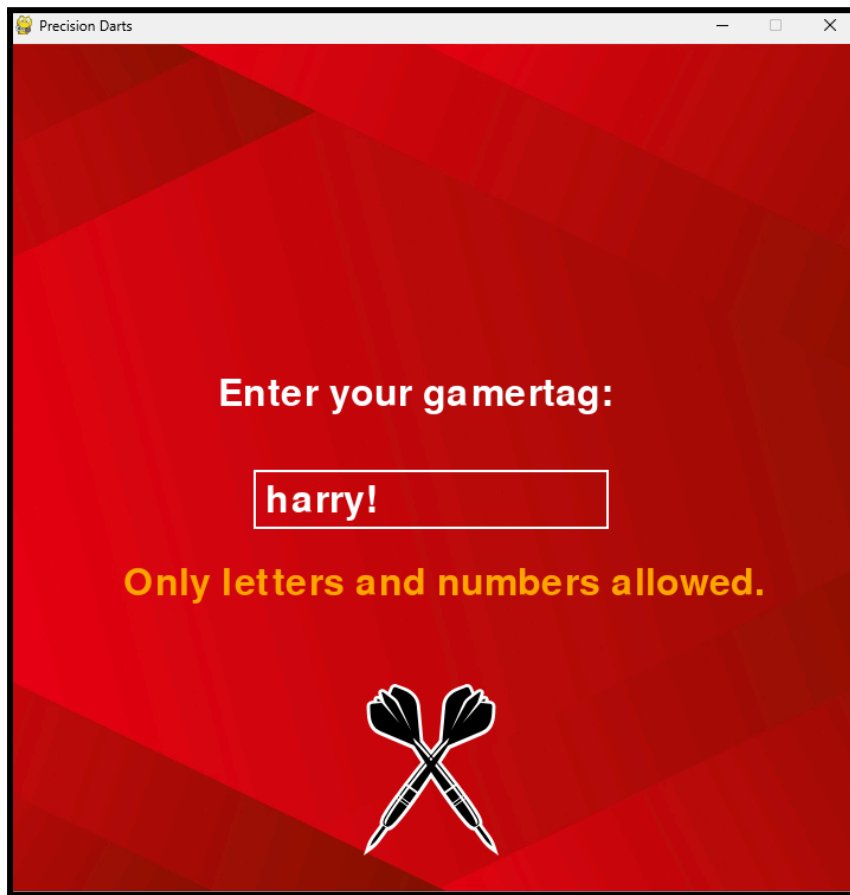
First Look at Gamertag Validation



Gamertag accepted: Harry
Welcome, Harry







Week 3 - Iteration 3 - Scoreboard + Calculation

Description

This iteration focuses on implementing the scoring feature which is responsible for subtracting score from dart throws, detecting busts (when a player overshoots beyond 0 or 1), and ensuring a win only occurs on a double-out, as well as displaying all of my scoreboard that I designed previously. It updates dynamically based on in-game performance.

Justification

Scoring is a main feature of darts gameplay. Without accurate score tracking, win

detection, and bust handling, it would not reflect real-world darts rules. Also as highlighted in both my survey and stakeholder interviews, scoring is wanted and would make the game feel catered to all players.

Code Evidence

Below is my code for the calculation of deviation and positioning based on power and how far the dart is from the center of the board (for doubles and triples). It also displays the main code for my scoreboard.

Python

[#game.py](#)

Power affects accuracy

```
if power < 20:    deviation_range = 35
elif power < 40: deviation_range = 25
elif power < 47: deviation_range = 15
elif power > 80: deviation_range = 35
elif power > 60: deviation_range = 25
elif power > 53: deviation_range = 15
else:            deviation_range = 5
```

Apply deviation

```
x += random.randint(-deviation_range, deviation_range)
y += random.randint(-deviation_range, deviation_range)
```

Calculate angle and distance from dartboard centre

```
cx, cy = 360, 272
dx, dy = x - cx, y - cy
distance = math.hypot(dx, dy)
angle = (math.degrees(math.atan2(-dy, dx)) + 360 + 90) % 360
```

```

# Find sector
sectors = [20,1,18,4,13,6,10,15,2,17,3,19,7,16,8,11,14,9,12,5]
sector_index = int((angle + 9) // 18) % 20
base_score = sectors[sector_index]

# Assign score based on ring hit
if distance < 15:
    return 50, True, (x, y) # Bullseye
elif distance < 30:
    return 25, False, (x, y) # Outer bull
elif 75 <= distance <= 105:
    return base_score * 3, False, (x, y)
elif 140 <= distance <= 165:
    return base_score * 2, True, (x, y)
elif distance < 165:
    return base_score, False, (x, y)
else:
    return 0, False, (x, y)

# Drawing helpers
def draw_scoreboard(font):
    # Draws player names, scores, legs and sets on the scoreboard
    area.

    p1, p2 = state.player1, state.player2

    # Player 1 row (name in black for contrast, numeric values in
    white)
    screen.blit(font.render(f"{p1.name}", True, (0,0,0)), (430,
648))
    screen.blit(font.render(f"{p1.score}", True, (255,255,255)),
(680, 648))

```

```

        screen.blit(font.render(f"{p1.legs}", True, (255,255,255)),
(646, 648))
        screen.blit(font.render(f"{p1.sets}", True, (255,255,255)),
(616, 648))

    # Player 2 row
        screen.blit(font.render(f"{p2.name}", True, (0,0,0)), (430,
690))
        screen.blit(font.render(f"{p2.score}", True, (255,255,255)),
(680, 690))
        screen.blit(font.render(f"{p2.legs}", True, (255,255,255)),
(646, 690))
        screen.blit(font.render(f"{p2.sets}", True, (255,255,255)),
(616, 690))

self.score = 501
self.legs = 0
self.sets = 0
self.darts_left = 3
self.turn_start_score = 501

```

Test Summary

- **TC01:** Player hits 20 → Score reduced by 20 ✓
- **TC02:** Player hits a triple → Score reduced by base value x3 ✓
- **TC03:** Player hits a double → Score reduced by base value x2 ✓
- **TC04:** Player busts score → Score resets to original score ✗

- **TC05:** Player wins a leg → legs incremented and scores reset ❌
- **TC04:** Player wins a set → sets incremented and legs reset ✅

Issues and Fixes

- **Issue:** Player data not reset after a leg win (e.g. score, darts left) → **Fix:** Ensured both a players' score and darts left reset correctly after a leg win
- **Issue:** Player name was hardcoded as "You" → Linked player gamertag to the function to ensure it was displayed correctly in-game
- **Issue:** Scores not resetting correctly after busting a score → **Fix:** I had to ensure that the player scores were being tracked individually using a class and then I was able to make sure they were reset correctly upon busting a score

First Look at Scoreboard and Scoring

	Sets	Legs	Score
Harry	0	0	501
Opponent	0	0	501

	Sets	Logs	Score
Harry	0	0	441
Opponent	0	0	501

	Sets	Logs	Score
Harry	0	1	501
Opponent	0	0	501

Week 4 - Iteration 4 - Dart Throwing System

Description

This iteration focuses on all of the dart-throwing mechanics, including a crosshair, power bar, deviation system, and hit detection. The dart follows a movement path to imitate real-life darts and I also made the dart be shown where it lands too.

Justification

Accurate throwing of a dart is essential for the game, because if it is not very accurate, the game becomes less enjoyable to play due to bugs. Implementing a crosshair and deviation system introduces an element of extra skill, and the power bar tests reflexes and ability to be accurate, adding another level of skill. Whilst doing this, I tried to make sure that the UI remained relatively simple and not

cluttered as highlighted from my survey.

Code Evidence

Below is the main code for my dart throwing system which includes the dart throwing animation and displaying the dart on screen.

Python

```
def animate_dart_throw(start_pos, end_pos):  
    # frame-by-frame animation for a dart moving from start_pos to  
    end_pos.  
    # Uses render_frame to keep the background consistent between  
    steps.  
  
    steps = 18  
    for i in range(steps):  
        t = i / (steps - 1)  
        # Straight-line animation  
        x = int(start_pos[0] * (1 - t) + end_pos[0] * t)  
        y = int(start_pos[1] * (1 - t) + end_pos[1] * t)  
  
        # While animating, show the base/ready dart position (not  
        charged)  
        ready_pos = (dart_base_x, dart_base_y)  
        render_frame(show_ready_dart=False, ready_pos=ready_pos)  
  
        # Draw the moving dart on top  
        screen.blit(dart_image, (x - 31, y - 6))  
        pygame.display.flip()  
  
        # Process events to avoid not responding and cap animation  
        FPS
```

```

pygame.event.pump()
anim_clock.tick(75)

def draw_power_bar(pwr):
    # Draws the multi-colour power bar using blocks to show
    # zones (red / orange / yellow / green). The white line
    indicates current power
    # Power range is 0-100, mapped to pixels by multiplying by 3

    bar_x, bar_y = 30, 690
    bar_width, bar_height = 300, 20

    # Colour bands (visual feedback to the player for risk/reward)
    pygame.draw.rect(screen, (255,0,0), (bar_x, bar_y, 60,
bar_height))
    pygame.draw.rect(screen, (255,120,0), (bar_x+60, bar_y, 60,
bar_height))
    pygame.draw.rect(screen, (255,200,0), (bar_x+120, bar_y, 21,
bar_height))
    pygame.draw.rect(screen, (0,255,0), (bar_x+141, bar_y, 18,
bar_height))
    pygame.draw.rect(screen, (255,200,0), (bar_x+159, bar_y, 21,
bar_height))
    pygame.draw.rect(screen, (255,120,0), (bar_x+180, bar_y, 60,
bar_height))
    pygame.draw.rect(screen, (255,0,0), (bar_x+240, bar_y, 60,
bar_height))

    # Current power indicator (white vertical line)
    pygame.draw.line(screen, (255,255,255), (bar_x + pwr * 3,
bar_y), (bar_x + pwr * 3, bar_y + bar_height), 2)

    # Outline of the whole bar

```

```

        pygame.draw.rect(screen, (255,255,255), (bar_x, bar_y,
bar_width, bar_height), 2)

def render_frame(show_ready_dart=True, ready_pos=None):
    # Centralised frame renderer.
    # - Draws board, scoreboard, power bar, crosshair and all
    # previously thrown darts.
    # - Keeps visual state rendering consistent across animations
    # and events.

    # Background & scoreboard
    screen.blit(dartboard, (0, 0))
    screen.blit(scoreboard_img, (420, 620))

    # Scores and player info
    draw_scoreboard(score_font)

    # Power bar displayed at the bottom-left
    draw_power_bar(power)

    # Crosshair
    pygame.draw.line(screen, (200,200,200), (crosshair.centerx-5,
crosshair.centery-5), (crosshair.centerx+5, crosshair.centery+5), 2)
    pygame.draw.line(screen, (200,200,200), (crosshair.centerx-5,
crosshair.centery+5), (crosshair.centerx+5, crosshair.centery-5), 2)

    # Previously-thrown darts (persist on the board until cleared)
    for pos in dart_poses:
        screen.blit(dart_image, (pos[0]-31, pos[1]-6))

    # Show the ready dart (pulled back while charging)
    if show_ready_dart and ready_pos is not None:
        screen.blit(dart_image, (ready_pos[0]-31, ready_pos[1]-6))

```

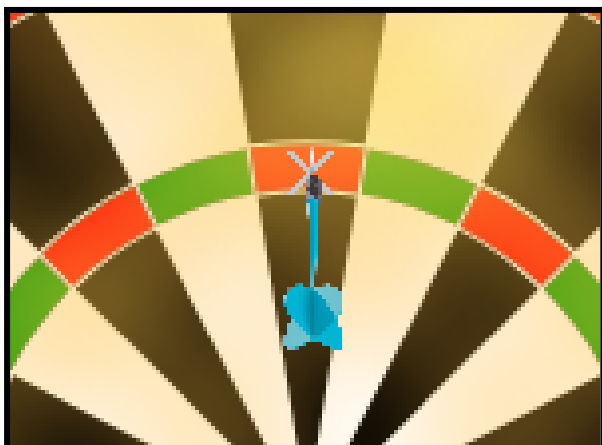
Test Summary

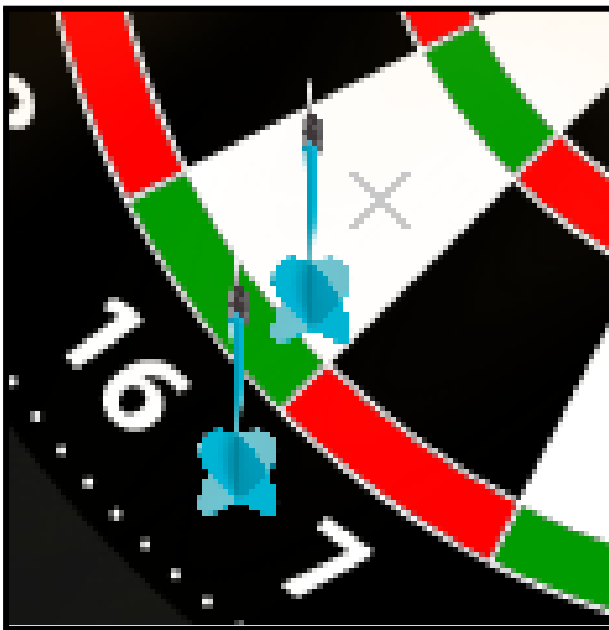
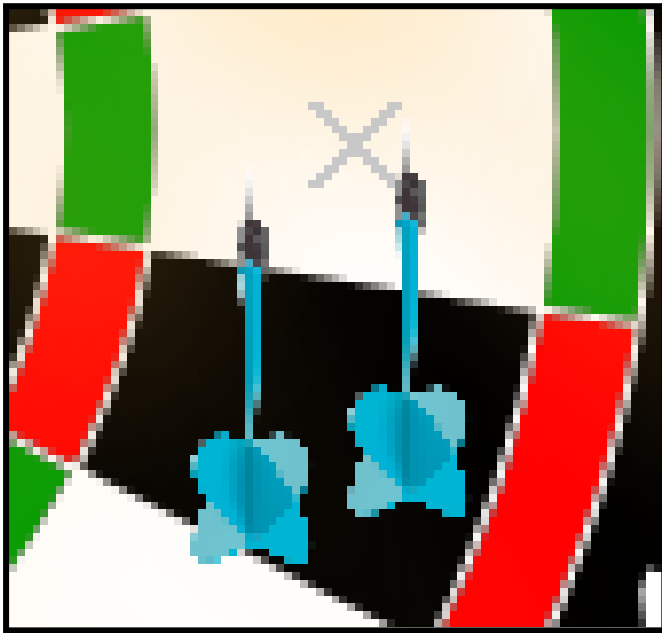
- **TC01:** Crosshair over triple 20 + perfect power → Score 50 ✓
- **TC02:** Overshoot power → Reduced accuracy, more deviation ✓
- **TC03:** Low power → Reduced accuracy, more deviation ✓

Issues and Fixes

- **Issue:** Crosshair was able to go off-screen → Added boundary checking to ensure the crosshair remained within the game window
- **Issue:** Deviation was too random with only 3 checks → Deviation now scales more smoothly and fairly based on the closeness to desired power

First look at dart throwing system





Week 5 - Iteration 5 - Game Loop & Turn System

Description

This iteration focuses on adding the complete turn system and game loop logic. Players alternate turns every 3 darts, and the game tracks legs and sets won. Turn switching happens when a player throws all 3 darts or if they bust their score. It also displays visual feedback for leg and set wins, or if a player busts their score. Also in

this iteration I added some helper functions which will help make my game more complete such as a text and flash popup that happens when winning legs or sets for example.

Justification

A complete game loop is essential to complete the game and replicate the rules of darts. Alternating turns, handling win conditions, and busts ensure the game maintains more realistic gameplay. Tracking sets and legs introduces a full match structure, necessary for tournament-style play, which I will introduce at a later stage in development.

Code Evidence

Python

```
# Player class
# Player class
class Player:
    # Represents a single player (human or AI)

    def __init__(self, player_name, player_data=None):
        self.name = player_name

    # Load player data from leaderboard if not provided
    if player_data is None:
        lb = load_leaderboard()
        player_data = lb.get(player_name)
        if player_data is None:
            # Case-insensitive lookup (so "alex" and "Alex" are
            # treated the same)
            for k, v in lb.items():
```

```

        if k.lower() == player_name.lower():
            player_data = v
            self.name = k
            break

# GameState class
class GameState:
    # Holds the current game state (two players and turn tracking).

    def __init__(self, player1_name="You", player2_name=None,
ai_difficulty="intermediate"):
        lb = load_leaderboard()
        p1_data = lb.get(player1_name)
        if player2_name is None:
            player2_name = generate_ai_name()
        p2_data = lb.get(player2_name)

        self.player1 = Player(player1_name, p1_data)
        self.player2 = Player(player2_name, p2_data)
        self.current_player = self.player1

def simulate_dart_throw(position, power):
    # Simulates the result of a dart throw

# Helper functions
def wait_with_event_pump(ms):
    # Delays for ms milliseconds while processing events (prevents
freezing)

    start = pygame.time.get_ticks()
    clock = pygame.time.Clock()
    while pygame.time.get_ticks() - start < ms:
        for ev in pygame.event.get():

```

```

        if ev.type == pygame.QUIT:
            pygame.quit()
            raise SystemExit()
    clock.tick(60)

def safe_font(path, size):
    # Loads a font safely, falling back to system default if missing

    try:
        return pygame.font.Font(path, size)
    except Exception:
        return pygame.font.SysFont(None, size)

def safe_image(path, size=None):
    # Loads an image safely, returning a placeholder if missing

    try:
        img = pygame.image.load(path)
        if size:
            img = pygame.transform.scale(img, size)
        return img
    except Exception:
        s = pygame.Surface(size if size else (50, 50))
        s.fill((120, 120, 120))
        return s

# Main match loop
def aim_and_throw_dart(screen, player1_name, player2_name=None,
    ai_difficulty="intermediate"):
    # Runs a darts match between player1 and player2 (human vs AI).
    # Handles scoring, leg/set/match progression, bust rules,
    # and animations for both AI and human throws.

```

```

# Drawing helpers

def show_flash_and_text(message, text_color=(0,0,0),
flash_color=(255,255,255), flash_time=1200):
    # overlay flash + message. Used for leg/set win messages and
    visual feedback
    # Renders the current frame first, then draws a overlay and
    message
    # wait_with_event_pump is used to avoid freezing

    # Render current frame (dartboard, scores, etc.)
    render_frame(show_ready_dart=False)

    # Make overlay
    overlay = pygame.Surface((720, 720))
    overlay.fill(flash_color)
    overlay.set_alpha(160)
    screen.blit(overlay, (0, 0))

    # Draw message centered on screen
    popup_font = safe_font(None, 48)
    popup_surf = popup_font.render(message, True, text_color)
    popup_rect = popup_surf.get_rect(center=(360, 360))
    screen.blit(popup_surf, popup_rect)

    pygame.display.flip()
    wait_with_event_pump(flash_time)

def show_bust_flash():
    # Wrapper for showing a BUST screen

    show_flash_and_text("BUST!")

```

```

def reset_power_state():
    # Resets power-related state after a throw or when changing
    turns.

    # Declared with nonlocal so nested functions can modify
    outer-scope variables.

    nonlocal power, power_dir, charging, dart_ready, animating
    power = 0
    power_dir = 1
    charging = False
    dart_ready = True
    animating = False

    # Setup before entering loop
    opponent_name = player2_name or generate_ai_name()
    state = GameState(player1_name, opponent_name, ai_difficulty)

    # Store turn start scores used for bust rollback
    state.player1.turn_start_score = state.player1.score
    state.player2.turn_start_score = state.player2.score

    # Load images (safe_image falls back to placeholders if assets are
    missing)
    dartboard = safe_image(os.path.join("assets", "background_1.png"),
(720,720))
        scoreboard_img = safe_image(os.path.join("assets",
"scoreboard.png"), (300,100))
        dart_image = safe_image(os.path.join("assets", "dart.png"),
(60,60))

    # Clocks for main loop and animations
    main_clock = pygame.time.Clock()
    anim_clock = pygame.time.Clock()

```

```

# Font used for scoreboard text
score_font = safe_font(None, 26)

# Power / input state variables
power = 0
power_dir = 1
charging = False
dart_ready = True
max_power = 100
space_released = True
animating = False

# Persistent list of thrown dart positions to display on the board
dart_poses = []

# Main match loop
while True:
    # Collect events once and iterate
    events = pygame.event.get()
    for event in events:
        if event.type == pygame.QUIT:
            return "QUIT" # signalling to quit
        if event.type == pygame.KEYDOWN and event.key ==
pygame.K_ESCAPE:
            return "MENU" # quit to main menu

# Human player turn
    # Movement of the crosshair using continuous key press
    detection
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] and crosshair.left > 0:
        crosshair.move_ip(-2, 0)

```

```

        if keys[pygame.K_RIGHT] and crosshair.right < 720:
crosshair.move_ip( 2, 0)
        if keys[pygame.K_UP] and crosshair.top > 0:
crosshair.move_ip(0, -2)
        if keys[pygame.K_DOWN] and crosshair.bottom < 720:
crosshair.move_ip(0, 2)

# Handle key events from earlier inputs
for event in events:
    # Begin charging power while spacebar is held
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_SPACE and dart_ready and not
charging and space_released and not animating:
            charging = True
            space_released = False

    # On release of spacebar perform the throw if properly
charged
        if event.type == pygame.KEYUP and event.key ==
pygame.K_SPACE:
            space_released = True
            if charging and dart_ready and not animating:
                # Lock state for the duration of the throw
                throw_power = power
                charging = False
                dart_ready = False
                animating = True

            # Save starting score at the beginning of the
player's turn for bust rollback
            if state.player1.darts_left == 3:
                state.player1.turn_start_score =
state.player1.score

```

```

        # Calculate visual pullback of the dart depending
on power

        pullback = int((throw_power / max_power) * 30)
        start_pos = (dart_base_x, dart_base_y + pullback)

        # Determine actual throw outcome (score, double
flag, landing position)

        score, is_double, dart_pos =
simulate_dart_throw(crosshair.center, throw_power)

        # Animate dart travel and clear events afterwards
to avoid accidental input

        animate_dart_throw(start_pos, dart_pos)
        pygame.event.clear()
        animating = False

        # Record and apply score
        dart_poses.append(dart_pos)
        p = state.current_player
        p.score -= score

        # Bust handling
        if p.score < 0 or p.score == 1:
            # Invalid finish - turn is forfeit and score
rolled back

            show_bust_flash()
            p.score = p.turn_start_score
            p.darts_left = 3
            # Switch to opponent (AI)
            state.current_player = state.player2
            state.player2.turn_start_score =
state.player2.score

```

```

        dart_poses.clear()
        reset_power_state()

    # Exact 0 score
    elif p.score == 0:
        if is_double:
            # Valid finish -> award leg and possibly
set/match

            p.legs += 1
            p.legs_won_total += 1
            p.save_stats()

            # Set handling
            if p.legs >= 3:
                p.sets += 1
                p.legs = 0
                (state.player1 if p is state.player2
else state.player2).legs = 0

                show_flash_and_text(f"{p.name} wins the
set!")

            # Match win condition
            if p.sets == 3:
                update_player_stats(state.player1)
                update_player_stats(state.player2)
                show_flash_and_text(f"{p.name} wins
the match!")

                return p.name
            else:
                # Leg won but match continues
                show_flash_and_text(f"{p.name} wins the
leg!")

```

```

        # Reset for next leg
        state.player1.score = 501
        state.player2.score = 501
        state.player1.darts_left = 3
        state.player2.darts_left = 3
        state.current_player = state.player1
        dart_poses.clear()
        reset_power_state()

        # Must finish on a double - otherwise treated
as a bust

    else:
        show_bust_flash()
        p.darts_left = 3
        state.current_player = state.player2
        state.player2.turn_start_score =
state.player2.score

        dart_poses.clear()
        reset_power_state()

        # Non-finishing result
    else:
        # Decrement remaining darts for this player's
turn

        p.darts_left -= 1

        # If out of darts, switch to opponent
    if p.darts_left == 0:
        p.darts_left = 3
        state.current_player = state.player2
        state.player2.turn_start_score =
state.player2.score

        dart_poses.clear()

```

```

        # Prepare visual/power state for next dart
        (either same player or opponent)
        reset_power_state()

    # power charge oscillation
    if charging:
        if power_dir == 1:
            power += 2
            if power >= max_power:
                power_dir = -1
        else:
            power -= 2
            if power <= 0:
                power_dir = 1

    # Pullback visual for the ready dart (makes power visible as
    movement)
    pullback = int((power / max_power) * 30) if charging else 0
    ready_dart_pos = (dart_base_x, dart_base_y + pullback)

    # Final render
    render_frame(show_ready_dart=True, ready_pos=ready_dart_pos)
    pygame.display.flip()
    main_clock.tick(60)

```

Test Summary

- **TC01:** Player throws 3 darts → Turn switches ✓
- **TC02:** Player busts score by overscoring → Score resets to start of turn and

the turn switches **✗ (Turn not switched)**

- **TC03:** Player busts score by not finishing on a double → Score resets to start of turn and the turn switches **✗ (Turn not switched)**

Issues and Fixes

- **Issue:** Player bust reset score to wrong value → **Fix:** Added turn_start_score per played and updated properly on turn start
- **Issue:** Darts didn't reset after a leg win → **Fix:** Explicitly set darts left to 3 for both players after leg
- **Issue:** Game didn't alternate correctly after busts → **Fix:** Ensured bust triggers turn switch and resets dart visual and power bar

Week 6 - Iteration 6 - AI Opponent Game Logic

Description

This iteration focuses on the implementation of an AI with beginner, intermediate, and advanced difficulty levels that adapt based on the score remaining and mappings, and then incorporating the AI into the turn system, with the aim being to simulate a realistic game of darts. Beginner difficulty has randomised and inaccurate throwing to simulate a beginner in a real game of darts. Intermediate has more precise accuracy, and advanced even more so. I also added input locking during AI throws to prevent a buildup of input causing bugs when it came to the next player turn.

This iteration also includes clean-up and fixes from previous iterations that were highlighted during the development and testing of the current iteration such as not

being able to throw again after an AI bust, player not being able to throw after a leg win, power bar not resetting between human turns, and visual glitches with the dart rendering on-screen.

On top of all this, I felt like reading through previous iterations of the development process. I wanted to clean-up my program and comment more precisely on what was happening, as comments from my previous iterations may have been unclear to anyone trying to follow the code. I also cleaned up some duplicate code that ended up arising, such as power bar and scoreboard redrawing, power bar resetting and a couple more things. This will be visibly updated in the final code at the end of the development process.

Justification

AI functionality is critical for my game to meet its aims and without it, the game would be very much limited to local multiplayer, and so by building a scalable AI, it added challenge and made the game more suitable for a single-player mode. Multiple difficulties allows players of all skill levels to play the game, whether it be casually or more competitively at a higher skill level.

Also highlighted from my survey and also my second interview with (Anonymous), people would rather play a game with multiple difficulty levels to cater to all types of players, which I have implemented.

Code Evidence

Below is the main AI logic that I made for my game. It is based around the principle of selected targeting based on whatever score is left with added deviation depending on the difficulty. I implemented a checkout map that includes every

possible darts finish so that the AI can dynamically change its throw location depending on this, if a finish is possible.

Python

```
import pygame
import os
import random

# AI name generation
def generate_ai_name():
    return random.choice(["Alex", "Taylor", "Jordan", "Jamie", "Sam",
                          "Riley", "Drew", "Morgan", "Casey", "Eric", "Grace", "Billy", "Mike",
                          "Mason", "Luke"])

# Utility functions
def update_player_stats(player):
    # Updates leaderboard entry for a given player with latest stats

    leaderboard = load_leaderboard()
    if player.name in leaderboard:
        leaderboard[player.name]["legs_won_total"] =
player.legs_won_total
        leaderboard[player.name]["tournaments_won"] =
player.tournaments_won
        leaderboard[player.name]["achievements_unlocked"] =
player.achievements_unlocked
        save_leaderboard(leaderboard)

# AI target selection
def ai_select_target(score, difficulty):
    # Determines AI aiming position and throw power
```

```

# Fixed dartboard sector order (clockwise from top)
SECTOR_ORDER = [20, 1, 18, 4, 13, 6, 10, 15, 2, 17,
                 3, 19, 7, 16, 8, 11, 14, 9, 12, 5]

def get_angle(sector_number):
    # Returns radian angle of a given sector number

    index = SECTOR_ORDER.index(sector_number)
    angle_deg = index * 18
    return math.radians(angle_deg - 90)

def aim_for_ring(sector_number, ring):
    # Returns pixel coordinate for aiming at a sector ring

    ring_radai = {'S': 120, 'T': 90, 'D': 155}
    radius = ring_radai[ring]
    angle_rad = get_angle(sector_number)
    x = int(360 + radius * math.cos(angle_rad))
    y = int(272 + radius * math.sin(angle_rad))
    return (x, y)

# Difficulty settings
difficulty_settings = {
    'beginner': {'deviation': 45, 'power_range': (30, 70)},
    'intermediate': {'deviation': 25, 'power_range': (45, 65)},
    'advanced': {'deviation': 16, 'power_range': (50, 58)},
}

settings = difficulty_settings.get(difficulty,
difficulty_settings['intermediate'])

def apply_jitter(pos):
    """Adds inaccuracy to simulate realistic dart throws."""

```

```

dev = settings['deviation']

    return (pos[0] + random.randint(-dev, dev), pos[1] +
random.randint(-dev, dev))

# Checkout map for AI selection
checkout_map = {
    170:[(20, 'T')], 167:[(20, 'T')], 164:[(20, 'T')], 161:[(20, 'T')],
    160:[(20, 'T')], 158:[(20, 'T')], 157:[(20, 'T')], 156:[(20, 'T')],
    155:[(20, 'T')], 154:[(20, 'T')], 153:[(20, 'T')], 152:[(20, 'T')],
    151:[(20, 'T')], 150:[(20, 'T')], 149:[(20, 'T')], 148:[(20, 'T')],
    147:[(20, 'T')], 146:[(20, 'T')], 145:[(20, 'T')], 144:[(20, 'T')],
    143:[(20, 'T')], 142:[(20, 'T')], 141:[(20, 'T')], 140:[(20, 'T')],
    139:[(20, 'T')], 138:[(20, 'T')], 137:[(20, 'T')], 136:[(20, 'T')],
    135:[(20, 'T')], 134:[(20, 'T')], 133:[(20, 'T')], 132:[(20, 'T')],
    131:[(20, 'T')], 130:[(20, 'T')], 129:[(19, 'T')], 128:[(18, 'T')],
    127:[(20, 'T')], 126:[(19, 'T')], 125:[(18, 'T')], 124:[(20, 'T')],
    123:[(19, 'T')], 122:[(18, 'T')], 121:[(17, 'T')], 120:[(20, 'T')],
    119:[(19, 'T')], 118:[(20, 'T')], 117:[(20, 'T')], 116:[(20, 'T')],
    115:[(20, 'T')], 114:[(20, 'T')], 113:[(20, 'T')], 112:[(20, 'T')],
    111:[(20, 'T')], 110:[(20, 'T')], 109:[(20, 'T')], 108:[(20, 'T')],
    107:[(19, 'T')], 106:[(20, 'T')], 105:[(19, 'T')], 104:[(18, 'T')],
    103:[(17, 'T')], 102:[(16, 'T')], 101:[(15, 'T')], 100:[(20, 'T')],
    99:[(19, 'T')], 98:[(20, 'T')], 97:[(19, 'T')], 96:[(20, 'T')],
    95:[(19, 'T')], 94:[(18, 'T')], 93:[(19, 'T')], 92:[(20, 'T')],
    91:[(17, 'T')], 90:[(18, 'T')], 89:[(19, 'T')], 88:[(16, 'T')],
    87:[(17, 'T')], 86:[(18, 'T')], 85:[(15, 'T')], 84:[(20, 'T')],
    83:[(17, 'T')], 82:[(14, 'T')], 81:[(15, 'T')], 80:[(20, 'T')],
    79:[(13, 'T')], 78:[(18, 'T')], 77:[(19, 'T')], 76:[(20, 'T')],
    75:[(17, 'T')], 74:[(14, 'T')], 73:[(19, 'T')], 72:[(16, 'T')],
    71:[(13, 'T')], 70:[(18, 'T')], 69:[(19, 'T')], 68:[(20, 'T')],
    67:[(17, 'T')], 66:[(10, 'T')], 65:[(19, 'T')], 64:[(16, 'T')],
    63:[(13, 'T')], 62:[(10, 'T')], 61:[(15, 'T')], 60:[(20, 'S')],
    59:[(19, 'S')], 58:[(18, 'S')], 57:[(17, 'S')], 56:[(16, 'S')],

```

```

55:[(15, 'S')], 54:[(14, 'S')], 53:[(13, 'S')], 52:[(12, 'S')],
51:[(11, 'S')], 50:[('BULL', 'D')], 49:[(9, 'S')], 48:[(8, 'S')],
47:[(7, 'S')], 46:[(6, 'S')], 45:[(5, 'S')], 44:[(4, 'S')],
43:[(3, 'S')], 42:[(2, 'S')], 41:[(1, 'S')], 40:[(20, 'D')],
39:[(7, 'S')], 38:[(19, 'D')], 37:[(5, 'S')], 36:[(18, 'D')],
35:[(3, 'S')], 34:[(17, 'D')], 33:[(1, 'S')], 32:[(16, 'D')],
31:[(7, 'S')], 30:[(15, 'D')], 29:[(5, 'S')], 28:[(14, 'D')],
27:[(3, 'S')], 26:[(13, 'D')], 25:[(1, 'S')], 24:[(12, 'D')],
23:[(3, 'S')], 22:[(11, 'D')], 21:[(1, 'S')], 20:[(10, 'D')],
19:[(3, 'S')], 18:[(9, 'D')], 17:[(1, 'S')], 16:[(8, 'D')],
15:[(3, 'S')], 14:[(7, 'D')], 13:[(1, 'S')], 12:[(6, 'D')],
11:[(3, 'S')], 10:[(5, 'D')], 9:[(1, 'S')], 8:[(4, 'D')],
7:[(1, 'S')], 6:[(3, 'D')], 5:[(1, 'S')], 4:[(2, 'D')],
3:[(1, 'S')], 2:[(1, 'D')],
}

# Checkout logic
if score in checkout_map:
    seg, ring = checkout_map[score][0]
    if seg == 'BULL':
        return apply_jitter((360, 272)),
random.randint(*settings['power_range'])
    pos = aim_for_ring(seg, ring)
    return apply_jitter(pos),
random.randint(*settings['power_range'])

# General scoring logic
if score > 100:
    return apply_jitter(aim_for_ring(20, 'T')),
random.randint(*settings['power_range'])
if score == 50:
    return apply_jitter((360, 272)),
random.randint(*settings['power_range'])

```

```

        return apply_jitter(aim_for_ring(20, 'T')),
        random.randint(*settings['power_range'])

# Dart throw simulation

# AI turn
if state.current_player == state.player2:
    # Save starting score for bust rollback
    state.player2.turn_start_score = state.player2.score

    # AI gets up to 3 darts (may finish early)
    for _ in range(3):
        # Wait a short, randomised time while pumping events
        (makes AI feel realistic)
        wait_with_event_pump(random.randint(900,1500))

        # Decide where to aim and how hard to throw
        ai_target, ai_power =
ai_select_target(state.player2.score, state.ai_difficulty)

        # Simulate the accuracy to get the actual result
        score, is_double, dart_pos =
simulate_dart_throw(ai_target, ai_power)

        # animate from base to result
        start_pos = (dart_base_x, dart_base_y)
        animate_dart_throw(start_pos, dart_pos)

        # Brief pause after the animation to make readable
        wait_with_event_pump(250)

        # Record landed dart visually and update score
        dart_poses.append(dart_pos)

```

```

        state.player2.darts_used_in_leg += 1
        state.player2.score -= score

        # check throw-based achievements (ton, 180)
        new_unlocked = check_achievements(p, "throw",
{"points": score})
        for new_achievement in new_unlocked:
            show_flash_and_text(f"Achievement Unlocked:
{ACHIEVEMENTS[new_achievement]['name']}", text_color=(255,215,0))

        # bust rules: <0 or exactly 1 are invalid finishes ->
end turn

if state.player2.score < 0 or state.player2.score == 1:
    show_bust_flash()

                                state.player2.score =
state.player2.turn_start_score
    dart_poses.clear() # remove thrown darts from board
    state.current_player = state.player1
    break

# If exact zero: must be double to count as leg win
if state.player2.score == 0:
    if is_double:
        # Valid leg finish
        state.player2.legs += 1
        state.player2.legs_won_total += 1

                                unlocked = check_achievements(p, "leg",
meta={"is_double": is_double, "finish_points": score})
        if unlocked:
            for aid in unlocked:
                print(f"Achievement unlocked: {aid}")

```

```

        # leg-based achievements (first leg, leg
hunter, checkout king)

        new_unlocked = check_achievements(p, "leg",
{"finish_points": score, "is_double": is_double})
        for new_achievement in new_unlocked:
            show_flash_and_text(f"Achievement Unlocked:
{ACHIEVEMENTS[new_achievement]['name']}", text_color=(255,215,0))

        state.player2.save_stats() # persist cumulative
legs

        # If leg-win leads to set-win
        if state.player2.legs >= 3:
            state.player2.sets += 1

            new_unlocked = check_achievements(p, "set",
{})

            for new_achievement in new_unlocked:
                show_flash_and_text(f"Achievement
Unlocked:
{ACHIEVEMENTS[new_achievement]['name']}",
text_color=(255,215,0))

            state.player2.legs = 0
            state.player1.legs = 0
            show_flash_and_text(f"{state.player2.name}
wins the set!")

        # If set-win completes match
        if state.player2.sets == 3:
            update_player_stats(state.player1)
            update_player_stats(state.player2)

        show_flash_and_text(f"{state.player2.name} wins the match!")

```

```

        return state.player2.name
    else:
        # Leg won but match not over
        show_flash_and_text(f"{state.player2.name}
wins the leg!")

        # Reset scores for next leg and give next turn
to player1

        state.player2.darts_used_in_leg = 0
        state.player1.score = 501
        state.player2.score = 501
        dart_poses.clear()
        state.current_player = state.player1
        break
    else:
        # If finishing on non-double, count as bust
(rule enforcement)

        show_bust_flash()

        state.player2.score =
state.player2.turn_start_score
        dart_poses.clear()
        state.current_player = state.player1
        break

        turn_points = state.current_player.turn_start_score -
state.current_player.score
        unlocked = check_achievements(state.current_player,
"turn_end", meta={"turn_points": turn_points})
        if unlocked:
            for aid in unlocked:
                print(f"Achievement unlocked: {aid}")

```

```

        # After AI's up-to-3-darts loop: ensure control passes back
to player
    if state.current_player == state.player2:
        # If AI still marked as current (didn't bust or win),
switch to human
        state.current_player = state.player1
        state.player1.darts_left = 3
        dart_poses.clear()

        # Reset power visuals/state in case the player was
mid-charge
        reset_power_state()

```

Test Summary

This iteration came with a lot of testing and issues that needed to be fixed and below highlights the main tests that came with the AI building as well as the issues and fixes which follow.

- **TC01:** All AI difficulties throw towards a set target based on logic → Beginner AI did not follow same logic ❌
- **TC02:** Beginner AI throws with lower accuracy → More deviation and less likely to hit higher score ✔
- **TC03:** Intermediate AI throws with improved accuracy → Still quite a bit of deviation but slightly more likely to high higher scores ✔
- **TC04:** Advanced AI throws with high accuracy → Less deviation and more accuracy on higher scores but can still miss occasionally ✔

- **TC05:** Player Bust resets turn correct → Score reset, AI throws ✗ (AI did not throw)
- **TC06:** AI bust resets turn correctly → Score reset, Player throws ✗ (Power bar was not reset after AI turn)
- **TC07:** AI throws are animated visually → Dart flies with delay for realism ✓
- **TC08:** After leg win, both players reset → Scores set to 501, turns continue ✗ (Player could not throw after a leg win)

Issues and Fixes

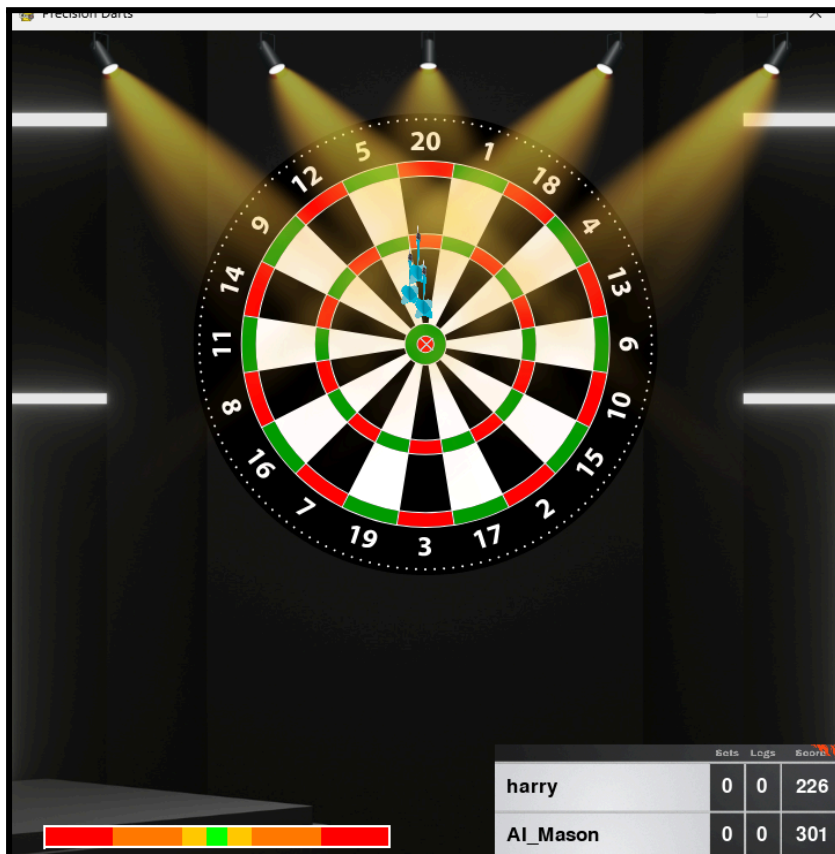
- **Issue:** AI did not throw after human bust → **Cause:** Turn state not switched correctly → **Fix:** Added proper score reset and turn assignment logic
- **Issue:** Power bar did not reset after AI turn → **Cause:** Power variable not reset → **Fix:** set power=0 after AI throws
- **Issue:** Human could throw during an AI's turn → **Cause:** Inputs were not locked during the AI's turn → **Fix:** cleared pygame events during the AI's turn
- **Issue:** Player couldn't throw after a leg win → **Cause:** darts_left was not reset → **Fix:** Added the reset after every leg and set win
- **Issue:** AI always aimed for the bullseye → **Cause:** No original target selection logic → **Fix:** I created the AI select target function with target logic
- **Issue:** AI aiming for incorrect finishes → **Cause:** Wrong checkout logic. I was checking for doubles under 50//2 which meant that it checked for double 22 for example and I faced an error → **Fix:** Added logic to aim for doubles up to 40//2 at maximum and a special case for the bullseye
- **Issue:** Dart positions flickered on the AI throw → **Cause:** The dart animation overlapped the UI drawing → **Fix:** Updated the animate dart throw function to

update the UI at each frame to prevent overlap

Overall this iteration was very challenging to implement, having never programmed an AI before, albeit a relatively simplistic AI but it turned out to be quite successful and handled edge cases quite well in the end upon testing and bug fixing. As far as I know there are no known bugs remaining but I will continue to test and fix at later iterations should an issue arise.

First look at AI

	Sets	Legs	Score
harry	0	0	326
AI_Mason	0	0	376



Week 7 - Iteration 7 - Leaderboard System

Description

This iteration focuses on the leaderboard system which is used to track and display player data based on their performances. I have included 3 different categories, which are: total legs won, tournaments won, and achievements unlocked.

The leaderboard option is able to be selected from the main menu and by default, when clicking the leaderboard, total legs won is shown. The player can use arrow keys to switch between the categories, displaying the data in the `leaderboard_data.json` file. The stats are live updated so even if a match is not fully completed, legs won in the match will count towards the tally.

Also in this iteration, I made some more changes to earlier versions of the program which I found bugs with, or needed improvement, most significantly an issue with how the gamertag validation worked. I decided to revamp the system and add a login-styled system instead so that an account and stats can be carried across sessions. Finally, I improved the commenting of the code within all files to make it easier to understand.

Justification

A leaderboard system adds another competitive element to the game, alongside the achievements system and tournament system coming in a later iteration of the program, and gives the game better replayability and encourages players to improve their performance and try to unlock achievements.

Having separate categories makes the data more meaningful, as a player may have won a lot of legs but not tournaments, or may have lots of achievements but not many legs won. By doing this, it means that players can feel a sense of accomplishment by being on the leaderboard for a different type of performance. By storing data in a JSON file, the leaderboard remains between sessions, which ensures long-term tracking, as well as quick loading and saving of data without needing a database system.

I decided to update the gamertag validation because it did not work very well with the leaderboard once it was implemented. I found that a player could not use the same gamertag across different sessions, making the leaderboard only live for the current session essentially. By adding a password system, it means that a player can login to their account across different sessions and continue to gain progress from where they left off.

This iteration also sets the foundation for the coming iterations, where the tournaments won category of the leaderboard will be directly updated in iteration 8's tournament mode, and the achievements unlocked category will sync with iteration 9's achievement system.

Code Evidence

Python

```
from leaderboard_data import load_leaderboard, save_leaderboard

#Player Class updated
    # Assign stored stats, or initialise defaults
    if player_data:
        self.legs_won_total = player_data.get("legs_won_total", 0)
        self.tournaments_won = player_data.get("tournaments_won",
0)

                                self.achievements_unlocked =
player_data.get("achievements_unlocked", 0) # Placeholder for my
achievements iteration
    else:
        self.legs_won_total = 0
        self.tournaments_won = 0
        self.achievements_unlocked = 0

def save_stats(self):
    # Updates the leaderboard file with the player's persistent
stats.

    # Ensures the entry exists before updating.

    lb = load_leaderboard()
    existing = lb.get(self.name, {})
    passwd = existing.get("password_hash", "")
```

```

lb[self.name] = {
    "password_hash": passwd,
    "legs_won_total": self.legs_won_total,
    "tournaments_won": self.tournaments_won,
    "achievements_unlocked": self.achievements_unlocked,
    "achievements": list(self.achievements),
    "achievements_unlocked": len(self.achievements)
}
save_leaderboard(lb)

self.ai_difficulty = ai_difficulty

```

Python

[#leaderboard.py](#)

```

# Responsible for displaying the leaderboard screen.
# Supports multiple categories (legs won, tournaments won,
# achievements unlocked) and sorts players accordingly.

```

```

import pygame
import sys
import os
from leaderboard_data import load_leaderboard

```

```

# Global font & colours used for consistency

```

```

FONT_NAME = "freesansbold.ttf"

```

```

BLACK = (0, 0, 0)

```

```

WHITE = (255, 255, 255)

```

```

RED = (255, 0, 0)

```

```

# Categories available in the leaderboard screen

```

```

CATEGORIES = ["legs", "tournaments", "achievements"]

```

```

# Text Rendering Helpers
def draw_text(surface, text, size, x, y, color=BLACK):
    # Draws plain text at (x,y) with given font size
    font = pygame.font.Font(FONT_NAME, size)
    text_surface = font.render(text, True, color)
    text_rect = text_surface.get_rect(center=(x, y))
    surface.blit(text_surface, text_rect)

def highlight_text(surface, text, size, x, y, selected=False):
    # Draws text, highlighted in red if currently selected
    font = pygame.font.Font(FONT_NAME, size)
    color = RED if selected else BLACK
    text_surface = font.render(text, True, color)
    text_rect = text_surface.get_rect(center=(x, y))
    surface.blit(text_surface, text_rect)

# Main Leaderboard Display Function
def display_leaderboard(screen):
    # Shows the leaderboard screen.
    # Allows the player to switch categories with arrow keys.
    # Displays top 5 players sorted by selected stat.
    # Backspace returns to main menu.

    # Attempt to load background image, else fallback is a plain
    surface
    bg_path = os.path.join("assets", "leaderboard.png")
    try:
        background = pygame.image.load(bg_path)
        background = pygame.transform.scale(background, (720, 720))
    except Exception:
        background = pygame.Surface((720, 720))
        background.fill((235, 235, 235))

```

```

current_category_index = 0
clock = pygame.time.Clock()

players_data = load_leaderboard()

# Transform JSON data into objects for easier sorting
players = [
    type("P", (), {"name": name, **stats})
    for name, stats in players_data.items()
]

running = True
while running:
    screen.blit(background, (0, 0))

    # Draw category selection row (legs / tournaments /
achievements)
    current_category = CATEGORIES[current_category_index]
    x_positions = [120, 360, 600]
    for i, category in enumerate(CATEGORIES):
        highlight_text(screen, category.capitalize(), 26,
x_positions[i], 142, selected=(i == current_category_index))

    # Sort players by chosen category
    if current_category == "legs":
        sorted_players = sorted(players, key=lambda p: getattr(p,
"legs_won_total", 0), reverse=True)
    elif current_category == "tournaments":
        sorted_players = sorted(players, key=lambda p: getattr(p,
"tournaments_won", 0), reverse=True)
    else:

```

```

        sorted_players = sorted(players, key=lambda p: getattr(p,
"achievements_unlocked", 0), reverse=True)

# Draw top 5 players
x_rank = 150
x_name = 360
x_value = 570
y = 230
for rank, p in enumerate(sorted_players[:5], start=1):
    if current_category == "legs":
        stat = f"{getattr(p, 'legs_won_total', 0)}"
    elif current_category == "tournaments":
        stat = f"{getattr(p, 'tournaments_won', 0)}"
    else:
        stat = f"{getattr(p, 'achievements_unlocked', 0)}"

# Draw each column separately
draw_text(screen, f"{rank}.", 30, x_rank, y)
draw_text(screen, p.name, 30, x_name, y)
draw_text(screen, stat, 30, x_value, y)
y += 98 # vertical spacing

# Navigation instructions
draw_text(screen, "Arrow Keys Change Category | Backspace to
Return", 20, 360, 680, BLACK)
pygame.display.flip()

# Handle input
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    if event.type == pygame.KEYDOWN:

```

```

        if event.key == pygame.K_LEFT:
            current_category_index = (current_category_index -
1) % len(CATEGORIES)
        elif event.key == pygame.K_RIGHT:
            current_category_index = (current_category_index +
1) % len(CATEGORIES)
        elif event.key == pygame.K_BACKSPACE:
            running = False

    clock.tick(60)

```

Python

#leaderboard_data.py

```

# Handles all interactions with the leaderboard.json file.
# Responsible for saving player stats, creating new players,
# password hashing, and authentication.

```

```

import json
import os
import hashlib

```

```

LEADERBOARD_FILE = "leaderboard.json"

```

```

# Password Handling

```

```

def hash_password(password):
    # Hash the password for security
    return hashlib.sha256(password.encode()).hexdigest()

```

```

# File I/O Functions

```

```

def load_leaderboard():
    # Loads leaderboard data from JSON

```

```

# Returns empty dictionary if file not found
if os.path.exists(LEADERBOARD_FILE):
    with open(LEADERBOARD_FILE, "r", encoding="utf-8") as f:
        return json.load(f)
return {}

def save_leaderboard(data):
    # Saves leaderboard dictionary back to JSON
    with open(LEADERBOARD_FILE, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=4)

# Player creation and authentication
def create_player(gamertag, password):
    # Creates a new player account if gamertag not already taken
    lb = load_leaderboard()
    if gamertag in lb:
        return None # Prevents overwriting existing account
    lb[gamertag] = {
        "password_hash": hash_password(password),
        "legs_won_total": 0,
        "tournaments_won": 0,
    }
    save_leaderboard(lb)
    return lb[gamertag]

def authenticate_player(gamertag, password):
    # Verifies gamertag + password combination
    lb = load_leaderboard()
    entry = lb.get(gamertag)
    if not entry:
        return None
    return entry if entry.get("password_hash") == hash_password(password) else None

```

```

def ensure_player_exists(gamertag):
    # Ensures that a player entry exists in the leaderboard
    # Used for AI players that don't require passwords
    lb = load_leaderboard()
    if gamertag not in lb:
        lb[gamertag] = {
            "password_hash": "",
            "legs_won_total": 0,
            "tournaments_won": 0,
            "achievements_unlocked": 0,
            "achievements": []
        }
        save_leaderboard(lb)
    return lb[gamertag]

# Update Stats
def update_tournaments_won(player_name):
    # Increments the tournament win count for the given player
    data = load_leaderboard()
    if player_name not in data:
        # Create entry if it doesn't exist
        data[player_name] = {
            "password_hash": "",
            "legs_won_total": 0,
            "tournaments_won": 0,
            "achievements_unlocked": 0
        }
        data[player_name]["tournaments_won"] =
data[player_name].get("tournaments_won", 0) + 1
        save_leaderboard(data)

```

Test Summary

- **TC01:** Opened leaderboard upon opening the game → Leaderboard loads with all 3 categories available ✓
- **TC02:** Played several games to win legs and test stat updating → Leaderboard shows updated legs during the game and after ✗ **(Only upon resetting the game)**
- **TC03:** Used left and right arrow keys to switch between leaderboard categories → Category changes with the highlighted option, and the data correctly loads ✗ **(Returns user to menu)**
- **TC04:** Pressed backspace on the leaderboard screen → Returns to main menu without exiting the program ✓
- **TC05:** Closed game and reopened → Previous leaderboard data still displays correctly
- **TC06:** Tried to use the same gamertag to see if stats could be progressed by the user → Able to enter the same gamertag and progress stats in the game ✗ **(Unable to enter same gamertag due to old system not having a login feature)**

Issues and Fixes

- **Issue:** Any key press returned the user to the menu → **Fix:** I modified the event handling so that only the backspace key would trigger the return to the main menu and the arrow keys would switch between the categories
- **Issue:** No background for the leaderboard → **Fix:** The early version of the leaderboard had no background image, which made it inconsistent with the rest of the game. I added a scaled leaderboard image and consistent highlight

styling.

- **Issue:** Data not updating in real time → **Fix:** Adjusted `update_leaderboard()` in `leaderboard_data.py` to save and reload the JSON file each time the leaderboard was opened
- **Issue:** Unable to login using the same gamertag on a different session → **Fix:** I revamped the gamertag creation and validation system to include a password that could be used to login on a different session

First look at leaderboard system



Leaderboard

Legs**Tournaments**Achievements

1.	harry	0
2.	AI_Sam	0
3.	AI_Billy	0
4.	AI_Mike	0
5.	AI_Casey	0

Arrow Keys Change Category | Backspace to Return

Leaderboard

Legs**Tournaments****Achievements**

1.	harry	0
2.	AI_Sam	0
3.	AI_Billy	0
4.	AI_Mike	0
5.	AI_Casey	0

Arrow Keys Change Category | Backspace to Return

Week 8-11 - Iteration 8 - Tournament Mode

Description

This iteration took significantly longer due to the nature of the logic and focussed on implementing a knockout tournament system and completing the integration of the gameplay loop between the human player and AI opponents. It is a 16-player structure that automatically seeds AI players alongside the human, and simulates AI vs AI matches where required. The match gameplay loop was refined to handle both AI and human turns, scoring, busts, leg/set wins, and overall match win. I also had to polish up some fixes and documentation of code to make the commenting better.

This iteration therefore completes the core gameplay cycle and logic

Justification

The main goal of this iteration was to make the game feel like a complete product. Adding tournament play creates a proper progression system that reflects how darts is played competitively. The knockout tournament format is simple for players to understand and works well with a single-elimination structure. Tournament seeding and AI generation includes unique AI names that are generated and stored in the leaderboard so that stats remain consistent. This avoids duplicate opponents and creates replayability. Player and AI integration means that the same player and gamestate classes are used for both human and AI, making the code easier to maintain and reducing duplication. Animated dart throws and overlays make the game more engaging, and the overlays provide clear feedback (important for usability testing). Event pumping during waits ensures that the game never freezes and remains responsive, which was an issue in earlier iterations. Stat tracking is persisted in the JSON leaderboard file. This ensures long-term progression and allows the leaderboard screen to remain meaningful.

Code Evidence

Below is the code for my tournament mode which brings together all of the key game loops to create a 16 player tournament structure.

Python

```
# Implements the Tournament class
# Seeds human player and AI opponents
# Displays matchups and results
# Updates player stats on leaderboard

import random
import pygame
import sys
import os

from leaderboard_data import ensure_player_exists, load_leaderboard
from game import aim_and_throw_dart, Player, generate_ai_name
from achievements import check_achievements, ACHIEVEMENTS

FONT = "freesansbold.ttf"

# Utility Functions
def wait_with_event_pump(ms):
    # Delays for ms milliseconds while still processing pygame events
    (prevents freeze)

    start = pygame.time.get_ticks()
    clock = pygame.time.Clock()
    while pygame.time.get_ticks() - start < ms:
        for ev in pygame.event.get():
            if ev.type == pygame.QUIT:
                pygame.quit()
                sys.exit()
        clock.tick(60)

def show_message(screen, text, size=36, color=(255,255,255), ms=1500):
    # Displays a temporary message in the middle of the screen
```

```

font = pygame.font.Font(FONT, size)
screen.fill((0,0,0))
surf = font.render(text, True, color)
rect = surf.get_rect(center=(360, 360))
screen.blit(surf, rect)
pygame.display.flip()
wait_with_event_pump(ms)

def show_round_pairings(screen, label, pairs):
    # Displays the pairings for a tournament round

    bg = pygame.image.load("assets/blank_background.png")
    bg = pygame.transform.scale(bg, (720, 720))

    title_f = pygame.font.Font(FONT, 40)
    row_f    = pygame.font.Font(FONT, 28)

    screen.blit(bg, (0, 0)) # draw background
    title = title_f.render(label, True, (255,255,255))
    screen.blit(title, title.get_rect(center=(360, 80)))

    y = 150
    for p1, p2 in pairs:
        line = row_f.render(f"{p1.name} vs {p2.name}", True,
(255,255,255))
        screen.blit(line, (190, y))
        y += 36

    pygame.display.flip()
    wait_with_event_pump(4000)

# Tournament Class

```

```

class Tournament:
    # Represents a knockout-style tournament with 16 players
    (adjustable however).

    # Seeds a human player alongside AI players, then runs matches
    until a winner is found.

    def __init__(self, screen, human_name,
ai_difficulty="intermediate", num_players=16):
        if num_players not in (2,4,8,16,32,64):
            raise ValueError("num_players must be power of two
(2,4,8,16,...)")
        self.screen = screen
        self.human_name = human_name
        self.ai_difficulty = ai_difficulty
        self.num_players = num_players
        self.players = self._seed_players()

    def _reset_for_next_round(self, player):
        # Resets player's in-match stats for the next round

        player.score = 501
        player.darts_left = 3
        player.legs = 0
        player.sets = 0

    def _seed_players(self):
        # Human player is added first
        # AI opponents generated with unique names
        # All opponents ensured in leaderboard

        roster = [Player(self.human_name,
self._get_player_data(self.human_name))]

```

```

        used = {self.human_name.lower()} # names in this tournament

        while len(roster) < self.num_players:
            ai_name = "AI_" + generate_ai_name()

            # keep regenerating name until it's unique in the
tournament
            while ai_name.lower() in used:
                ai_name = "AI_" + generate_ai_name()

            # ensure it's in leaderboard
            ensure_player_exists(ai_name)
            roster.append(Player(ai_name,
self._get_player_data(ai_name)))
            used.add(ai_name.lower())

        random.shuffle(roster)
        return roster

    def _get_player_data(self, name):
        # Fetches leaderboard entry for a given player name

        return load_leaderboard().get(name)

    def run(self, simulate=False):
        # Executes the tournament until a champion is found

        round_num = 1
        current = self.players[:]
        round_names = {16: "Round of 16", 8: "Quarterfinal", 4:
"Semifinal", 2: "Final"}

        while len(current) > 1:

```

```

        label = round_names.get(len(current), f"Round {round_num}")
        winners = []
        pairs = [(current[i], current[i+1]) for i in range(0,
len(current), 2)]
        show_round_pairings(self.screen, label, pairs)

    for p1, p2 in pairs:
        if simulate:
            # Always pick a random winner for testing
            winner_name = random.choice([p1.name, p2.name])
            # Human-involved match
            elif p1.name == self.human_name or p2.name ==
self.human_name:
                if p1.name == self.human_name:
                    winner_name = aim_and_throw_dart(self.screen,
p1.name, p2.name, self.ai_difficulty)
                else:
                    winner_name = aim_and_throw_dart(self.screen,
p2.name, p1.name, self.ai_difficulty)
            else:
                # AI vs AI -> random winner
                winner_name = random.choice([p1.name, p2.name])

            # Convert winner_name -> actual Player object
            if winner_name in ("MENU", "EXIT"):
                return winner_name
            if winner_name == p1.name:
                winner = p1
            else:
                winner = p2

        self._reset_for_next_round(winner)
        winners.append(winner)

```

```

        current = winners
        round_num += 1

    # Champion found
    champion = current[0]
    ensure_player_exists(champion.name) # make sure entry exists
    champion.tournaments_won += 1
    champion.save_stats()

    new_unlocked = check_achievements(champion, "tournament", {})
    for new_achievement in new_unlocked:
        show_message(self.screen, f"Achievement Unlocked:
{ACHIEVEMENTS[new_achievement]['name']}!", size=28, color=(255,215,0),
ms=1500)

    # Display winner
    bg_path = os.path.join("assets", "blank_background.png")
    try:
        bg_img = pygame.image.load(bg_path)
        bg_img = pygame.transform.scale(bg_img, (720, 720))
    except:
        bg_img = None

    # Show winner screen
    font_big = pygame.font.Font(FONT, 56)
    font_small = pygame.font.Font(FONT, 32)

    self.screen.fill((0, 0, 0))
    if bg_img:
        self.screen.blit(bg_img, (0, 0))

```

```

        text = font_big.render(f"Tournament Winner!", True, (255, 215,
0))

    rect = text.get_rect(center=(360, 260))
    self.screen.blit(text, rect)

    champ_text = font_small.render(f"{champion.name}", True, (255,
255, 255))
    champ_rect = champ_text.get_rect(center=(360, 360))
    self.screen.blit(champ_text, champ_rect)

    pygame.display.flip()
    wait_with_event_pump(4000) # pause for 4 seconds

```

Test Summary

- **TC01:** 16-player tournament starts → All slots filled with unique names ✓
- **TC02:** Human vs AI match plays → Turns alternate correctly ✓
- **TC03:** AI vs AI matches simulated → Random winner chosen, tournament progresses ✓
- **TC04:** Player busts by overscoring → Score resets to turn_start_score and switches turn ✓
- **TC05:** Player busts by not finishing on a double → Score resets to turn_start_score and switches turn ✓
- **TC06:** Player wins leg on a double → Leg increments, scores reset, darts reset ✓
- **TC07:** Player wins set after 3 legs → Set increments, legs reset for both players ✓

- **TC08:** Player wins match after 3 sets → Match ends, correct winner returned ✓
- **TC09:** Tournament champion recorded → tournaments_won increments in leaderboard ✓

Issues and Fixes

- **Issue:** Game froze after login → **Fix:** Removed invalid load_player_data argument in Player.__init__.
- **Issue:** Tournament generated duplicate AI names → **Fix:** Added uniqueness check with used set in _seed_players().
- **Issue:** Bust logic inconsistent → **Fix:** Updated turn_start_score at start of every turn and restored on bust.
- **Issue:** Human input carried over during AI turns → **Fix:** Added pygame.event.clear() after AI throws.
- **Issue:** Game froze during delays (unresponsive window) → **Fix:** Replaced pygame.time.delay() with wait_with_event_pump() to pump events.
- **Issue:** Leaderboard stats not persisting correctly → **Fix:** Ensured .save_stats() called after leg/set/match wins.

First look at tournament system

Round of 16

Al_Mason vs Al_Jordan
Al_Drew vs Al_Taylor
Al_Mike vs Al_Morgan
Al_Riley vs Al_Grace
Al_Eric vs Al_Jamie
Al_Casey vs Al_Sam
Al_Luke vs harry
Al_Alex vs Al_Billy



Quarterfinal

Al_Grace vs harry
Al_Alex vs Al_Mike
Al_Luke vs Al_Casey
Al_Eric vs Al_Mason



Semifinal

AI_Grace vs AI_Morgan
AI_Drew vs AI_Jordan



Final

AI_Luke vs AI_Alex





Week 12-13 - Iteration 9 - Achievements System

Description

This iteration focuses all around the achievements system and display within the game. I have allowed for players to unlock achievements upon meeting specific conditions, whether it may be winning their first leg, winning a tournament or even hitting a perfect 9-dart leg. Achievements are stored persistently so they can track across multiple sessions and it remains a global leaderboard.

I also added repeatable achievement tracking so that if a player has won 3 tournaments it would show they have completed that achievement 3 times on the display screen, except for first-time achievements such as the first leg one. I created an achievements menu within the main menu, allowing users to scroll using the arrow keys through the achievements to see what they have, and haven't unlocked, with grey trophies being locked ones, and gold trophies being unlocked ones.

Justification

The main reasoning behind this iteration was to add a different kind of reward into the game, as winning tournaments was good but could become boring over time. By implementing achievements, it adds an extra sense of motivation for players to unlock all the achievements and encourages replayability of the game.

As well as that, the scrolling menu provides a clear, user-friendly interface to view progress, which is much better than cramming all of the achievements onto one static page. The counters on achievements make the system more rewarding as even if you unlock all achievements, you can still increase your number of tournament wins for example. Achievements were restricted to only human players, to reward the user's progress rather than automated AI performance, preventing the leaderboard from being overwhelmed by AI.

Code Evidence

Python

```
# New function in the player class in game.py

def unlock_achievement(self, aid):
    if self.name.startswith("AI_"):
        return False # Do not save achievements for AI players

    lb = load_leaderboard() # Load current leaderboard data
    player_data = lb.get(self.name, {})

    # make sure achievements list exists
    unlocked = player_data.get("achievements", [])
    counts = player_data.get("achievement_counts", {}) # Store repeat
counts of achievements such as tournaments won

    if aid not in unlocked: # (aid is achievement id)
        unlocked.append(aid) # Add achievement if first time completing

    # increment achievement counter for repeatable unlocks
    counts[aid] = counts.get(aid, 0) + 1

    # Save updated stats back into leaderboard
```

```

player_data["achievements"] = unlocked
player_data["achievement_counts"] = counts
player_data["achievements_unlocked"] = len(unlocked)

lb[self.name] = player_data
save_leaderboard(lb)
return True # Always return True so game knows achievement was unlocked

```

Python

```

import pygame
import os
from achievements import ACHIEVEMENTS
from leaderboard_data import load_leaderboard

FONT = "freesansbold.ttf"

def display_achievements(screen, gamertag):
    clock = pygame.time.Clock()

    # Load player data from leaderboard
    lb = load_leaderboard()
    pdata = lb.get(gamertag, {})

    unlocked = pdata.get("achievements", []) # Which achievements have been
    unlocked

    counts = pdata.get("achievement_counts", {}) # Track how many times an
    achievement has been earned

    achievement_ids = list(ACHIEVEMENTS.keys())
    current_index = 0

    # Load trophy images for locked/unlocked
    gold_path = os.path.join("assets", "gold_trophy.png")
    grey_path = os.path.join("assets", "grey_trophy.png")
    try:
        trophy_gold = pygame.image.load(gold_path)

```

```

    trophy_gold = pygame.transform.scale(trophy_gold, (720, 720))
    trophy_grey = pygame.image.load(grey_path)
    trophy_grey = pygame.transform.scale(trophy_grey, (720, 720))
except Exception:
    # Fallback if images can't load
    trophy_gold = pygame.Surface((720, 720)); trophy_gold.fill((255,215,0))
    trophy_grey = pygame.Surface((720, 720));
trophy_grey.fill((120,120,120))

while True:

    # Title
    title_font = pygame.font.Font(FONT, 60)
    title = title_font.render("Achievements", True, (255,255,255))
    screen.blit(title, title.get_rect(center=(360, 80)))

    # Current achievement info
    aid = achievement_ids[current_index]
    info = ACHIEVEMENTS[aid]

    # Draw trophy
    unlocked_status = aid in unlocked
    trophy = trophy_gold if unlocked_status else trophy_grey
    screen.blit(trophy, (0, 0))

    # Name + description of achievements
    name_font = pygame.font.Font(FONT, 36)
    desc_font = pygame.font.Font(FONT, 24)

    name_text = name_font.render(info["name"], True, (255,255,255))
    desc_text = desc_font.render(info["description"], True, (255,255,255))

    screen.blit(name_text, name_text.get_rect(center=(360, 550)))
    screen.blit(desc_text, desc_text.get_rect(center=(360, 600)))

    # Draw the counter number on the trophy if more than 1

```

```

count = counts.get(aid, 0)
if count > 1:
    counter_font = pygame.font.Font(FONT, 48)
    counter_surf = counter_font.render(str(count), True, (0,0,0))
    counter_rect = counter_surf.get_rect(center=(360, 300)) # center
of trophy
    screen.blit(counter_surf, counter_rect)

# Navigation Instructions
hint_font = pygame.font.Font(FONT, 20)
hint = hint_font.render("Arrow Keys to scroll, Backspace to return",
True, (255,255,255))
screen.blit(hint, hint.get_rect(center=(360, 680)))

pygame.display.flip()

# Handle events
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        return
    if event.type == pygame.KEYDOWN:
        if event.key == pygame.K_LEFT:
            current_index = (current_index - 1) % len(achievement_ids)
        elif event.key == pygame.K_RIGHT:
            current_index = (current_index + 1) % len(achievement_ids)
        elif event.key == pygame.K_BACKSPACE:
            return # back to menu

clock.tick(60)

```

Python

updated function in leaderboard_data.py

```
def ensure_player_exists(gamertag):
```

```

# Ensures that a player entry exists in the leaderboard
# Used for AI players that don't require passwords
lb = load_leaderboard()
if gamertag not in lb:
    lb[gamertag] = {
        "password_hash": "",
        "legs_won_total": 0,
        "tournaments_won": 0,
        "achievements_unlocked": 0,
        "achievements": []
    }
    save_leaderboard(lb)
return lb[gamertag]

```

Python

```

# example of an achievement check I implemented: check throw-based achievements
(ton, 180)
new_unlocked = check_achievements(p, "throw", {"points": score})
    for new_achievement in new_unlocked:
        show_flash_and_text(f"Achievement Unlocked:
{ACHIEVEMENTS[new_achievement]['name']}", text_color=(255,215,0))

```

Python

achievements.py

This file defines all possible achievements in the game.

```

ACHIEVEMENTS = {
    "first_leg": {
        "name": "First Leg", # Unlocked once a player has won their first ever
leg
        "description": "Win your first leg",
        "trigger": "leg",
        "condition": lambda player, meta: player.legs_won_total >= 1,

```

```

    },
    "leg_hunter": {
        "name": "Leg Hunter", # Unlocked once a player has won 50 total legs
        "description": "Win 50 legs (total)",
        "trigger": "leg",
        "condition": lambda player, meta: player.legs_won_total >= 50,
    },
    "set_master": {
        "name": "Set Master", # Unlocked once a player has won their first ever
leg
        "description": "Win 25 sets (total)",
        "trigger": "set",
        "condition": lambda player, meta: player.sets >= 25,
    },
    "champion": {
        "name": "Champion", # Unlocked once a player has won a tournament
        "description": "Win a tournament",
        "trigger": "tournament",
        "condition": lambda player, meta: player.tournaments_won >= 1,
    },
    "ton_scorer": {
        "name": "Ton Scorer", # Unlocked once a player has won scored over 100
with 3 darts
        "description": "Score 100+ points in a single turn",
        "trigger": "turn_end",
        "condition": lambda player, meta: (meta or {}).get("turn_points", 0) >=
100,
    },
    "one_eighty": {
        "name": "180!", # Unlocked once a player has won scored a perfect 180
with 3 darts
        "description": "Hit a perfect 180 in a single turn",
        "trigger": "turn_end",
        "condition": lambda player, meta: (meta or {}).get("turn_points", 0) ==
180,
    },

```

```

    "checkout_king": {
        "name": "Checkout King", # Unlocked once a player has won a leg
finishing on double 20
        "description": "Win a leg by double 20 (D20)",
        "trigger": "leg",
        "condition": lambda player, meta: (meta or {}).get("is_double", False)
and (meta or {}).get("finish_points", 0) == 40,
    },
    "nine_darter": {
        "name": "Nine Darter", # Unlocked once a player has scored a perfect leg in
only 9 darts (the minimum possible)
        "description": "Complete a leg in just 9 darts",
        "trigger": "leg",
        "condition": lambda player, meta: player.darts_used_in_leg == 9,
    },
}

```

```

def check_achievements(player, event, meta=None):
    # Check all achievements that trigger on 'event' for 'player'.
    # If any new ones are unlocked, call player's unlock_achievement and return
list of unlocked achievement ids.

    meta = meta or {}
    unlocked = []
    for aid, info in ACHIEVEMENTS.items():
        if info.get("trigger") != event:
            continue # Only check achievements that match this event
        try:
            if info["condition"](player, meta): # Check if achievement
condition is met
                if player.unlock_achievement(aid): # Save to leaderboard if
newly unlocked
                    unlocked.append(aid) # Add to list of new unlocks
        except Exception as e:

```

```
print(f"[achievements] error checking {aid}: {e}") # I added this
to print the error and not crash the game
return unlocked
```

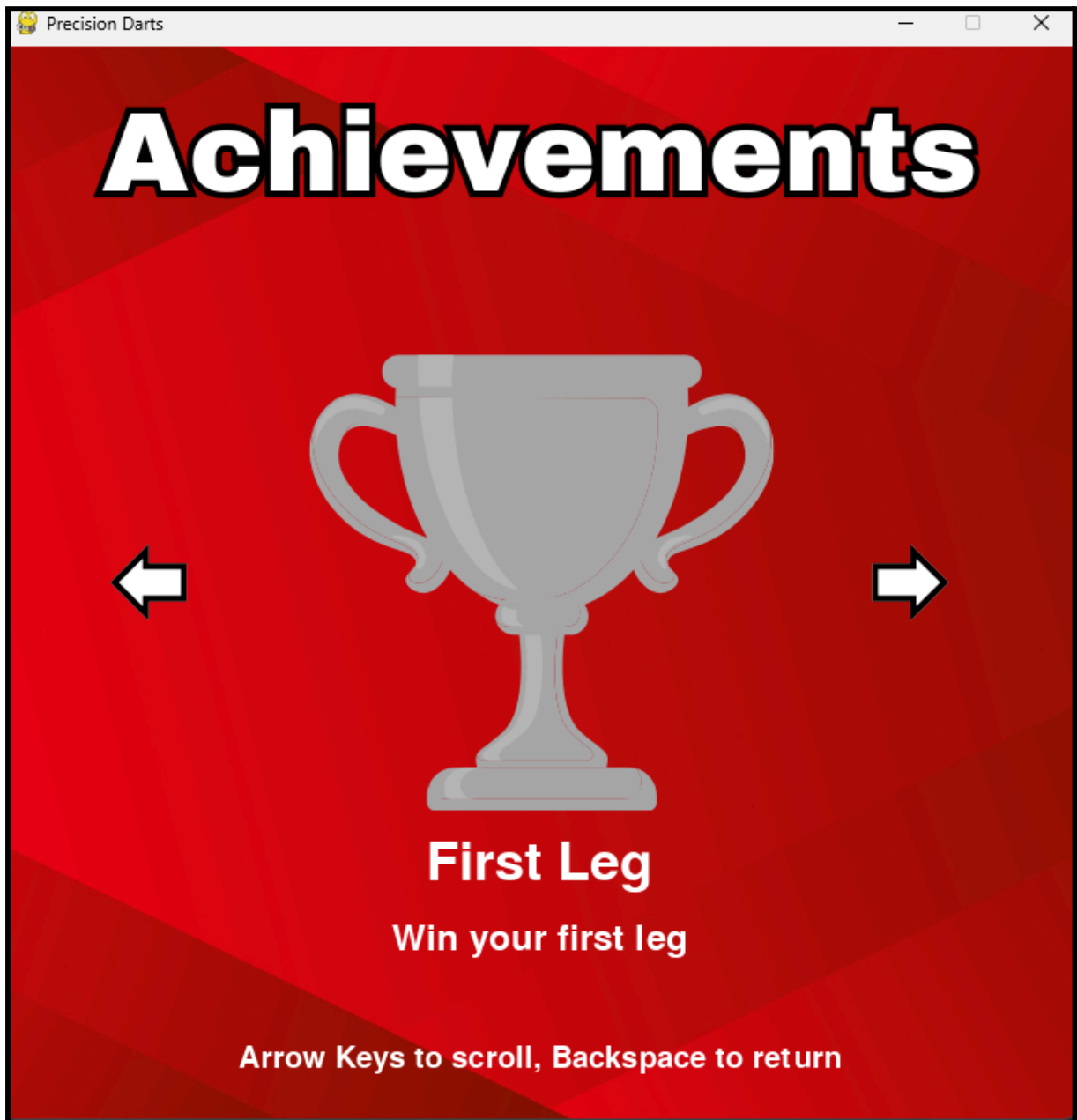
Test Summary

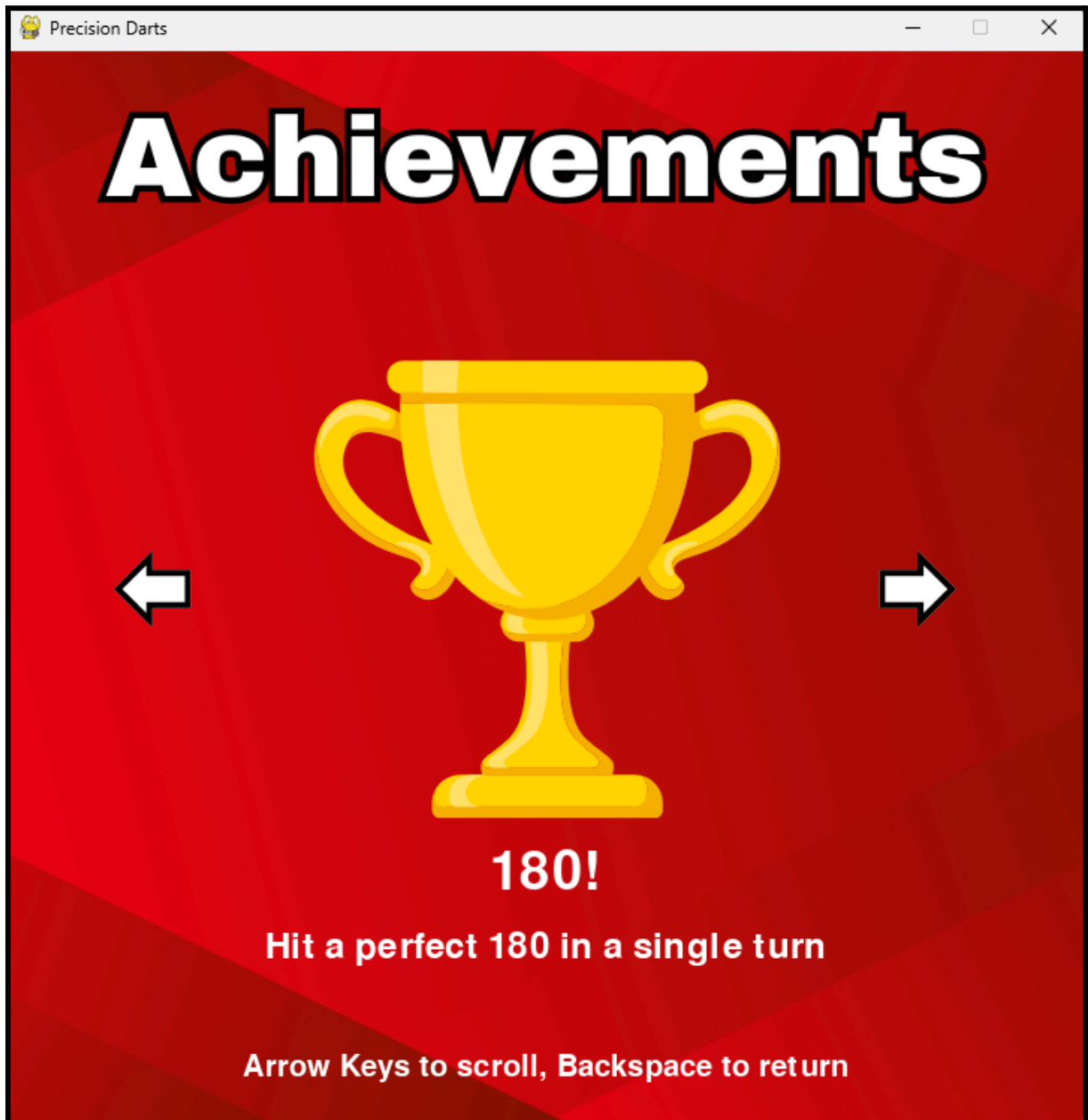
- **TC01:** Player wins first ever leg → Achievement 'First Leg' unlocked ✓
- **TC02:** Player scores 100+ points → Achievement 'Ton Scorer' unlocked ✓
- **TC03:** Player scores exactly 180 points → Achievement "180!" unlocked ✓
- **TC04:** Winning a tournament → Achievement "Champion" unlocked ✓
- **TC05:** Winning multiple tournaments → Counter number shown on trophy ✓
- **TC06:** Achievements menu shows correct trophy colour and description ✓
- **TC07:** All players do not unlock or save achievements. ✓

Issues and Fixes

- **Issue:** Achievements were unlocked but not displayed in the menu → **Fix:** I standardised to use achievements and achievement_counts fields in the leaderboard data.
- **Issue:** Achievements were not counting repeats → **Fix:** Added achievement_counts dictionary to leaderboard data

First Look at Achievements System







Week 14 - Iteration 10 - Pausing and Instructions

Description

This iteration was a shorter one but focussed on the pause menu as well as instructions screen of the game. Players are able to press the escape key once in game in order to bring up the pause menu. I have made the pause menu with 3 options: resume, instructions, and quit to the main menu. The user can navigate the options like all other menu screens by using the arrow keys and then pressing enter to select their choice. The resume button continues gameplay, the instructions button

brings up the how to play screen and the quit to main menu returns the user to the main menu.

Justification

The reasoning behind this iteration was to enable the user to be able to pause gameplay at any time and then continue when they return, as well as quit the game if they needed to, or to enable newer and less familiar players with the rules and instructions of the game. This creates a more user friendly experience within the game and is very beneficial to have.

Code Evidence

```
Python
# pause_menu.py
# pause_menu.py
import pygame
import os
from instructions import display_instructions

FONT = "freesansbold.ttf"

def pause_menu(screen):
    clock = pygame.time.Clock()

    # Load background image
    bg_path = os.path.join("assets", "pause_menu.png")
    try:
        background = pygame.image.load(bg_path)
        background = pygame.transform.scale(background, (720, 720))
    except Exception:
        background = pygame.Surface((720, 720))
        background.fill((30, 30, 30))

    options = ["Resume", "Instructions", "Quit to Main Menu"]
    selected_index = 0
```

```

while True:
    screen.blit(background, (0, 0))

    # Draw menu options
    font = pygame.font.Font(FONT, 40)
    for i, option in enumerate(options):
        color = (255, 0, 0) if i == selected_index else (255, 255, 255)
        surf = font.render(option, True, color)
        screen.blit(surf, surf.get_rect(center=(360, 250 + i * 100)))

    pygame.display.flip()

    # Handle input
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            return "quit"
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_UP:
                selected_index = (selected_index - 1) % len(options)
            elif event.key == pygame.K_DOWN:
                selected_index = (selected_index + 1) % len(options)
            elif event.key == pygame.K_RETURN:
                selected = options[selected_index]
                if selected == "Resume":
                    return None
                elif selected == "Instructions":
                    display_instructions(screen)
                elif selected == "Quit to Main Menu":
                    return "MENU"

    clock.tick(60)

```

Python

```
# instructions.py

# Displays an instructions/help screen with a background image.

import pygame
import os

def display_instructions(screen):
    clock = pygame.time.Clock()

    # Load background
    bg_path = os.path.join("assets", "how_to_play.png")
    try:
        background = pygame.image.load(bg_path)
        background = pygame.transform.scale(background, (720, 720))
    except Exception:
        background = pygame.Surface((720, 720))
        background.fill((30, 30, 30))

    while True:
        screen.blit(background, (0, 0))
        pygame.display.flip()

        # Handle events
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                pygame.quit()
                return
            if event.type == pygame.KEYDOWN:
                if event.key in (pygame.K_BACKSPACE, pygame.K_ESCAPE):
                    return # back to pause menu

        clock.tick(60)
```

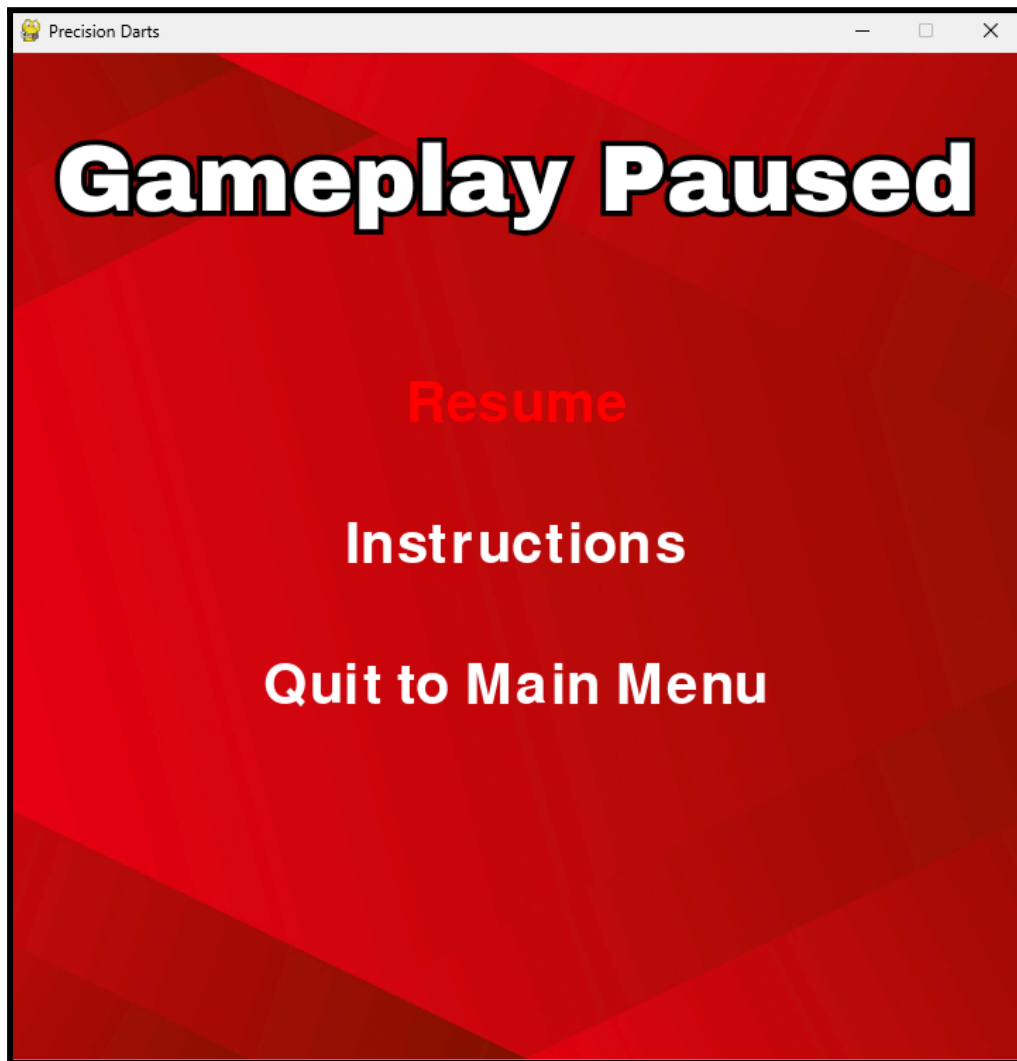
Test Summary

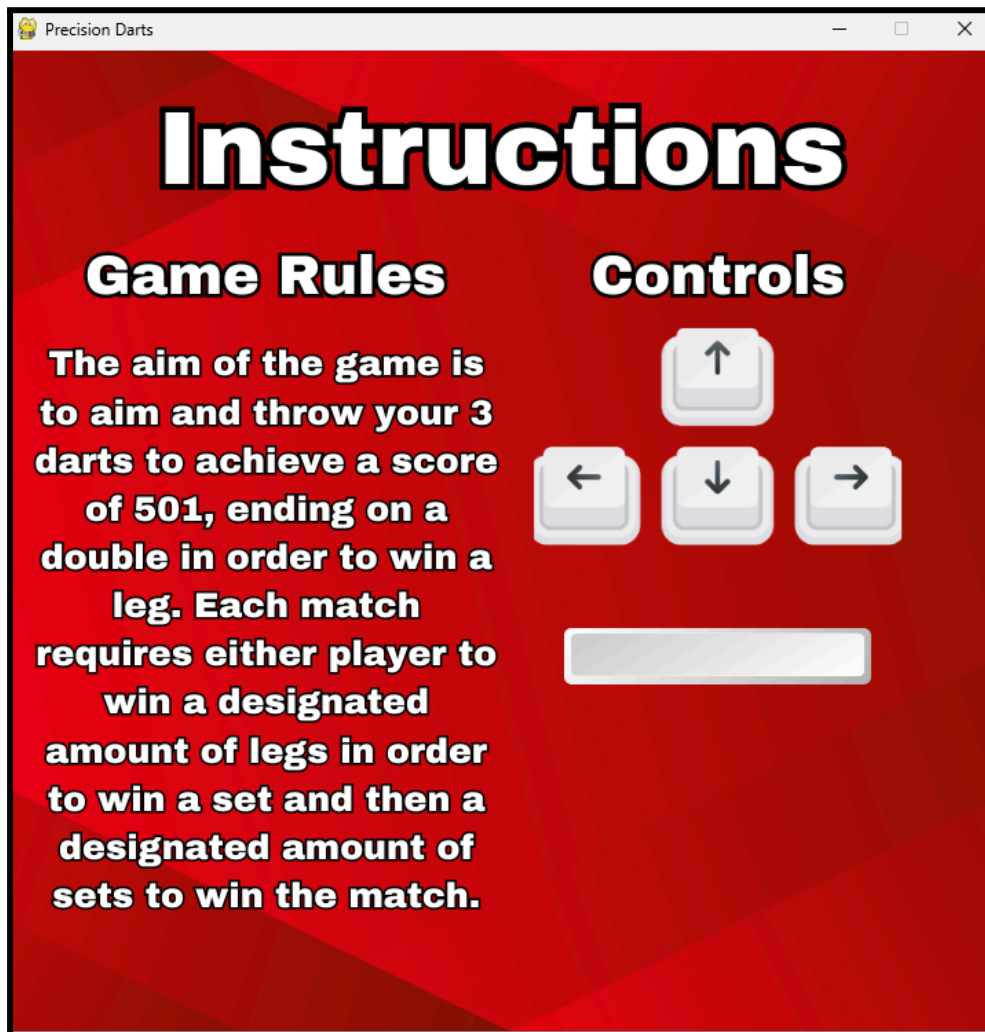
- **TC01:** Player presses 'esc' key → Pause menu is displayed ✓
- **TC02:** Player presses resume on pause menu → Gameplay is resumed where it was ✓
- **TC03:** Player presses instructions on pause menu → Instructions screen is displayed to the user ✓
- **TC04:** Player presses quit to main menu → Game exits to the main menu ✗

Issues and Fixes

Issue: Player presses quit to main menu on the pause screen, but it returns the user to gameplay → **Fix:** I had a duplicated piece of code which was doing two different things. My flag to return to the main menu wasn't being run because the duplicate code was instead returning a different flag value and so it returned the user to gameplay. I removed this and it worked as expected.

First look at pause menu and instructions





Week 15 - Iteration 11 - Sound Effects

Description

This iteration I focussed on all of the sound effects and audio tracks within the game. This includes the dart hit sound, a leg, set or match win sound, a bust sound, as well as menu navigation sound, and background audio tracks for both the menus and gameplay. There is an audio range that will be able to be adjusted through the settings in the next iteration.

Justification

The main reason for this iteration is to provide a more immersive experience to the user by adding audio to experience different elements of the game, which means that the game does not become as boring in silence. This is evidenced through my

survey and stakeholder interviews in which both my stakeholder and 65% of the survey respondents said that sound and audio would be beneficial to the game.

Code Evidence

Below is the main code for my sound manager as well as example usage to play or fade sounds or music

```
Python
# This file manages all sound effects used throughout the game.

import pygame
import os

pygame.mixer.init()

# Folder containing all sound assets
SOUND_DIR = os.path.join("assets", "sounds")

# Short sounds
SOUNDS = {}

def init_sounds():
    #] Initializes pygame mixer
    import pygame
    pygame.mixer.init()
    print("Sound system initialized.")

def find_audio_file(base_name):
    # Looks for either .mp3 or .wav file with given name
    for ext in [".wav", ".mp3"]:
        path = os.path.join(SOUND_DIR, base_name + ext)
        if os.path.exists(path):
            return path
    print(f"[sound_manager] Missing audio: {base_name}")
    return None

# Sound effects
```

```

def load_sound(name):
    # Loads a short sound effect
    path = find_audio_file(name)
    if not path:
        return None
    sound = pygame.mixer.Sound(path)
    SOUNDS[name] = sound
    return sound

def play(name, volume=0.4):
    # Play a short sound effect like hit or bust
    if name not in SOUNDS:
        snd = load_sound(name)
        if snd is None:
            return
    SOUNDS[name].set_volume(volume)
    SOUNDS[name].play()

# Music
def play_music(track, volume=0.05, loop=True):
    # Play looping background music with correct volume
    path = find_audio_file(track)
    if not path:
        print(f"[sound_manager] Could not find music track: {track}")
        return
    pygame.mixer.music.stop() # Stop any current music
    pygame.mixer.music.load(path)
    pygame.mixer.music.set_volume(volume)
    pygame.mixer.music.play(-1 if loop else 0)
    print(f"[sound_manager] Now playing {track} at volume {volume}")

def stop_music():
    # Stops background music immediately
    pygame.mixer.music.stop()

def pause_music():

```

```

    # Pauses background music
    pygame.mixer.music.pause()

def resume_music():
    # Resumes paused music
    pygame.mixer.music.unpause()

def fade_to_music(track, fade_ms=1500, volume=0.05, loop=True):
    # Fades out current music, then fades in new track
    pygame.mixer.music.fadeout(fade_ms)
    path = find_audio_file(track)
    if not path:
        return
    pygame.time.delay(fade_ms)
    pygame.mixer.music.load(path)
    pygame.mixer.music.set_volume(volume)
    pygame.mixer.music.play(-1 if loop else 0)

def set_music_volume_percent(percent):
    # Sets music volume using a 0-100% slider value
    # Limits actual Pygame volume to a safe range (0.0-0.4).
    percent = max(0, min(100, percent))
    safe_max = 0.4
    volume = (percent / 100) * safe_max
    pygame.mixer.music.set_volume(volume)

```

```

Python
play("hit")
play("leg_win")

```

```

Python
fade_to_music("menu_music", fade_ms=1500)

```

Test Summary

- **TC01:** Play sound effect 'hit' after dart throw → Short sound effect plays once ✓
- **TC02:** Play 'bust' sound after invalid finish → Bust sound plays ✓
- **TC03:** Background music in the main menu plays → Music loops ✓
- **TC04:** Transition music using the fade to music function → current track fades out, new track fades in ✓
- **TC05:** Attempt to play a sound file that isn't there → Console outputs error, no crash ✓

Issues and Fixes

- **Issue:** Background music was far too loud → **Fix:** Added volume scaling and made the maximum 0.4
- **Issue:** Set volume didn't change the volume → **Fix:** There was a flag in the code I used to set volume that overwrote the volume each time, so i removed this

Week 16 - Iteration 12 - Settings and Background Switching

Description

This iteration focuses on finalising my game. It includes the settings menu which has been implemented, which includes volume control for both the music volume and sound effects such as the menu navigation and darts hitting the board, etc , All difficulty control, as well as the ability to switch gameplay backgrounds.

Justification

The main reason for this iteration was to add a few accessibility settings such as the volume control which I feel is beneficial to any game of application, as some people like to have audio on, and some don't. As well as that, some people also may have difficulties with hearing, and this could cater towards them. In terms of background switching, I feel it is a good idea to include as it adds some variation to the game's look so you aren't always having to play on the same dartboard. As mentioned in

previous iterations about the AI, the ability to switch between difficulties allows for better user experience for different skilled players, increasing the longevity and replayability of the game

Code Evidence

Below is the main code that was used to implement both the settings menu.

Python

```
# background_manager.py

import os
import pygame

# Gameplay backgrounds
BACKGROUND_OPTIONS = [
    os.path.join("assets", "background_classic.png"),
    os.path.join("assets", "background_brick.png"),
    os.path.join("assets", "background_wood.png"),
]

# Persistent global state
active_background = BACKGROUND_OPTIONS[0] # Used in gameplay
selected_background = BACKGROUND_OPTIONS[0] # Previewed in settings
current_ai_difficulty = "intermediate" # Global AI difficulty

# Manage backgrounds

def get_active_background():
    # Returns a loaded pygame.Surface for the active gameplay background
    try:
        img = pygame.image.load(active_background)
        return pygame.transform.scale(img, (720, 720))
    except Exception as e:
        print(f"[background_manager] Error loading background: {e}")
        fallback = pygame.Surface((720, 720))
        fallback.fill((80, 80, 80))
```

```

        return fallback

def get_selected_background_name():
    # Returns the name of the currently selected background
    base = os.path.basename(selected_background).replace("background_",
    "").replace(".png", "")
    return base.capitalize()

def cycle_background(direction=1):
    # Cycles between background options
    global selected_background
    index = BACKGROUND_OPTIONS.index(selected_background)
    index = (index + direction) % len(BACKGROUND_OPTIONS)
    selected_background = BACKGROUND_OPTIONS[index]

def apply_selected_background():
    # Applies the selected background to gameplay
    global active_background
    active_background = selected_background

# AI management

AI_LEVELS = ["beginner", "intermediate", "advanced"]

def get_ai_difficulty():
    # Returns the current AI difficulty level
    return current_ai_difficulty

def cycle_ai_difficulty(direction=1):
    # Cycles AI difficulty between beginner/intermediate/advanced
    global current_ai_difficulty
    index = AI_LEVELS.index(current_ai_difficulty)
    index = (index + direction) % len(AI_LEVELS)
    current_ai_difficulty = AI_LEVELS[index]

```

Python

```
# settings_menu.py

import pygame
import os
from sound_manager import (
    set_music_volume,
    play_music,
    stop_music,
    set_muted,
    muted,          # Import the global mute flag
    music_volume,   # Import global music volume
    sfx_volume,     # Import global sfx volume
)
from background_manager import (
    cycle_background,
    apply_selected_background,
    get_selected_background_name,
    get_ai_difficulty,
    cycle_ai_difficulty
)

FONT = "freesansbold.ttf"

# Track live adjustments during the settings session
music_volume = music_volume
sfx_volume = sfx_volume
muted = muted

def settings_menu(screen):
    global music_volume, sfx_volume, muted

    clock = pygame.time.Clock()
    selected_option = 0
    options = ["Background", "AI Difficulty", "Music Volume", "SFX Volume",
               "Mute / Unmute", "Back"]
```

```

# Load settings background
bg_path = os.path.join("assets", "settings.png")
try:
    background = pygame.image.load(bg_path)
    background = pygame.transform.scale(background, (720, 720))
except Exception:
    background = pygame.Surface((720, 720))
    background.fill((200, 0, 0))

while True:
    screen.blit(background, (0, 0))
    option_font = pygame.font.Font(FONT, 34)

    # Text labels
    bg_text = f"Background: {get_selected_background_name()}"
    ai_text = f"AI Difficulty: {get_ai_difficulty().capitalize()}"
    music_text = f"Music Volume: {int(music_volume * 100)}%"
    sfx_text = f"SFX Volume: {int(sfx_volume * 100)}%"
    mute_text = "Muted" if muted else "Unmuted"

    labels = [bg_text, ai_text, music_text, sfx_text, mute_text, "Back"]

    for i, label in enumerate(labels):
        color = (255, 0, 0) if i == selected_option else (0, 0, 0)
        surf = option_font.render(label, True, color)
        screen.blit(surf, surf.get_rect(center=(360, 170 + i * 95)))

    pygame.display.flip()

    # Handle input
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_UP:

```

```

        selected_option = (selected_option - 1) % len(options)
elif event.key == pygame.K_DOWN:
    selected_option = (selected_option + 1) % len(options)

elif event.key == pygame.K_LEFT:
    if options[selected_option] == "Background":
        cycle_background(-1)
    elif options[selected_option] == "AI Difficulty":
        cycle_ai_difficulty(-1)
    elif options[selected_option] == "Music Volume":
        music_volume = max(0.0, music_volume - 0.01)
        set_music_volume(music_volume)
    elif options[selected_option] == "SFX Volume":
        sfx_volume = max(0.0, sfx_volume - 0.05)

elif event.key == pygame.K_RIGHT:
    if options[selected_option] == "Background":
        cycle_background(1)
    elif options[selected_option] == "AI Difficulty":
        cycle_ai_difficulty(1)
    elif options[selected_option] == "Music Volume":
        music_volume = min(1.0, music_volume + 0.01)
        set_music_volume(music_volume)
    elif options[selected_option] == "SFX Volume":
        sfx_volume = min(1.0, sfx_volume + 0.05)

elif event.key == pygame.K_RETURN:
    if options[selected_option] == "Mute / Unmute":
        muted = not muted
        set_muted(muted) # Sync global mute flag

    if muted:
        stop_music()
    else:
        # Resume the menu music only if not muted
        play_music("menu_music")

```

```

        elif options[selected_option] == "Background":
            apply_selected_background()
        elif options[selected_option] == "Back":
            return
    elif event.key == pygame.K_BACKSPACE:
        return

    clock.tick(60)

```

Python

updated sound_manager.py

```
import pygame
```

```
import os
```

```
pygame.mixer.init()
```

Folder for audio assets

```
SOUND_DIR = os.path.join("assets", "sounds")
```

Loaded short sounds

```
SOUNDS = {}
```

Global states

```
sfx_volume = 0.6 # Default SFX volume (0-1)
```

```
music_volume = 0.05 # Default music volume (0-1)
```

```
muted = False # Global mute flag
```

```
current_track = None # Track currently playing
```

```
def set_muted(state: bool):
```

```
    # Globally mute/unmute all sounds
```

```
    global muted
```

```
    muted = state
```

```
    if state:
```

```
        pygame.mixer.music.set_volume(0)
```

```

else:
    pygame.mixer.music.set_volume(music_volume)

def find_audio_file(base_name):
    # Looks for a valid audio file (.wav, .mp3, .m4a) in SOUND_DIR.
    # Returns the full path or None if missing.
    for ext in [".wav", ".mp3", ".m4a"]:
        path = os.path.join(SOUND_DIR, base_name + ext)
        if os.path.exists(path):
            return path
    print(f"[sound_manager] Missing audio: {base_name}")
    return None

def init_sounds():
    # Initialises the sound system
    pygame.mixer.init()
    print("Sound system initialised.")

# Volume control

def set_sfx_volume_percent(percent):
    # Adjusts the volume for all sound effects (0-100%).
    global sfx_volume
    sfx_volume = max(0, min(1, percent / 100))
    for snd in SOUNDS.values():
        snd.set_volume(sfx_volume)
    print(f"[sound_manager] SFX volume set to {round(sfx_volume, 2)}")

def set_music_volume_percent(percent):
    # Adjusts the background music volume (0-100%).
    global music_volume
    music_volume = max(0, min(1, percent / 100))
    pygame.mixer.music.set_volume(music_volume)

```

```

    print(f"[sound_manager] Music volume set to {round(music_volume, 2)}")

def mute_all():
    # Mutes all sounds
    global muted
    muted = True
    pygame.mixer.music.set_volume(0)
    for s in SOUNDS.values():
        s.set_volume(0)
    print("[sound_manager] Audio muted.")

def unmute_all():
    # Unmutes all sounds
    global muted
    muted = False
    pygame.mixer.music.set_volume(music_volume)
    for s in SOUNDS.values():
        s.set_volume(sfx_volume)
    print("[sound_manager] Audio unmuted.")

def is_muted():
    # Returns True if audio is currently muted
    return muted

# Sound effects

def load_sound(name):
    # Loads a short sound effect into memory and stores it in SOUNDS.
    path = find_audio_file(name)
    if not path:
        return None
    sound = pygame.mixer.Sound(path)

```

```

    sound.set_volume(0 if muted else sfx_volume)
    SOUNDS[name] = sound
    return sound

def play(name):
    # Plays a short sound effect such as 'hit', 'bust', or menu navigation.
    if muted:
        return
    if name not in SOUNDS:
        snd = load_sound(name)
        if snd is None:
            return
    SOUNDS[name].play()

# Music

def play_music(track, volume=0.05, loop=True):
    # Plays a music track with adjustable volume.
    if muted:
        return
    path = find_audio_file(track)
    if not path:
        print(f"[sound_manager] Could not find music track: {track}")
        return

    pygame.mixer.music.load(path)
    pygame.mixer.music.set_volume(volume)
    pygame.mixer.music.play(-1 if loop else 0)

def stop_music():
    # Stops all background music immediately
    pygame.mixer.music.stop()

```

```

def fade_to_music(track, fade_ms=1500, loop=True):
    # Smoothly fades from the current music track to a new one
    global current_track
    if muted:
        return
    pygame.mixer.music.fadeout(fade_ms)
    path = find_audio_file(track)
    if not path:
        return
    pygame.time.delay(fade_ms)
    pygame.mixer.music.load(path)
    pygame.mixer.music.set_volume(music_volume)
    pygame.mixer.music.play(-1 if loop else 0)
    current_track = track
    print(f"[sound_manager] Faded to new track: {track}")

def pause_music():
    # Temporarily pauses background music
    pygame.mixer.music.pause()

def resume_music():
    # Resumes paused background music
    if not muted:
        pygame.mixer.music.unpause()

def set_music_volume(vol):
    # Sets background music volume (direct float 0-1)
    global music_volume
    music_volume = max(0, min(1, vol))
    pygame.mixer.music.set_volume(music_volume)

```

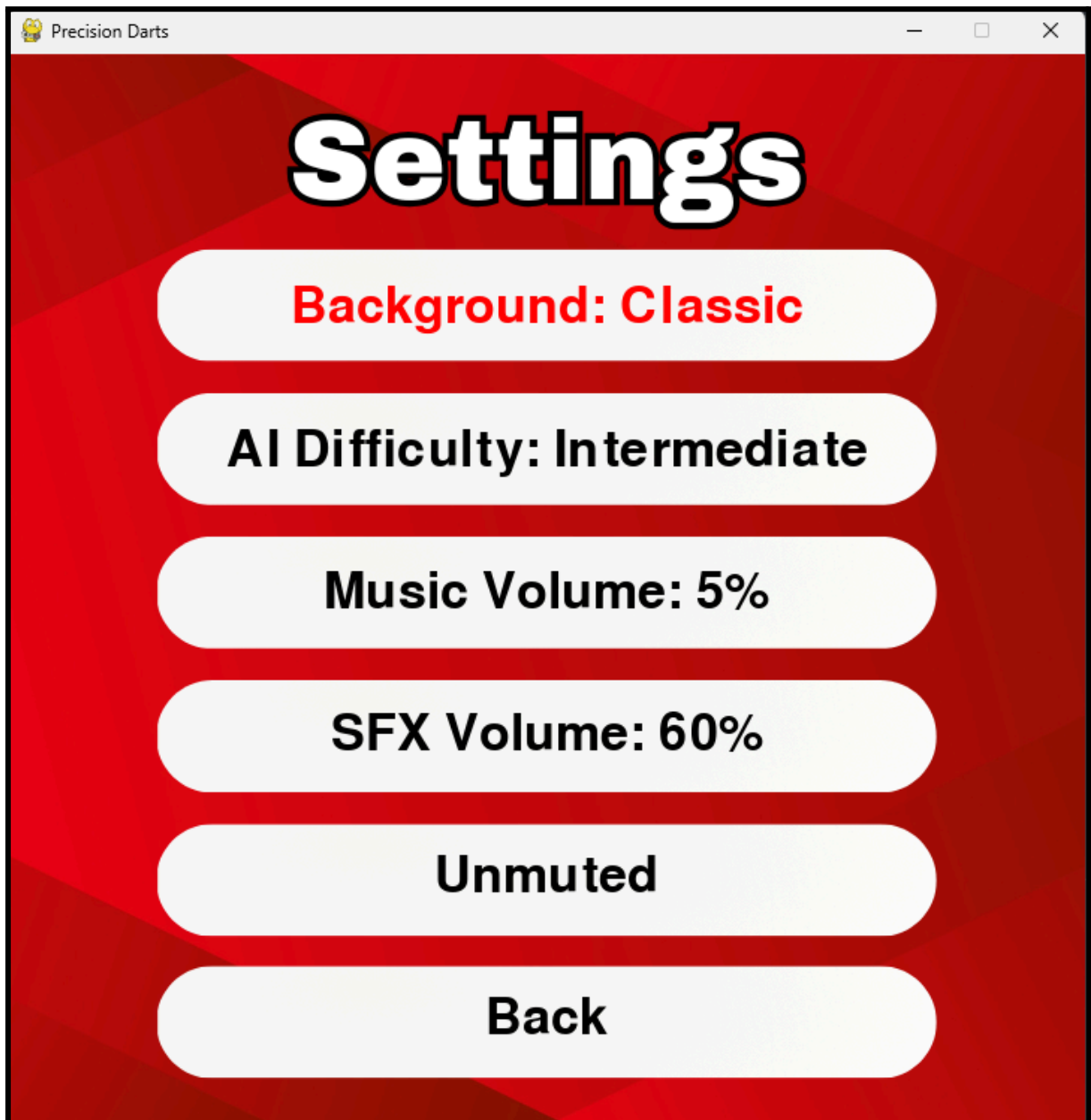
Test Summary

- **TC01:** Background switched to background 2 & 3 → Background switched upon game start ✓
- **TC02:** Music volume adjusted to 20% → Music volume decreased to 20% ✓
- **TC03:** Sfx volume adjusted to 20% → Sfx volume decreased to 20% ✓
- **TC04:** Toggled audio to mute → All audio within the game is muted ✓
- **TC05:** Difficulty Adjusted to Beginner → More deviation in the AI in the game (beginner AI toggled) ✓

Issues and Fixes

During this iteration, I was not faced with many issues that needed to be addressed, however one major issue was that upon changing volume levels for both the music and sound effects, when you started gameplay, these changes would not save. To fix this, I added global flags and variables for the audio levels so that it would transfer into gameplay. I was also faced with an issue that caused muting and then unmuting audio to play the wrong music track. To fix that, I again created a global muted flag to ensure that audio was muting correctly.

First Look at Settings



Stakeholder Follow-up

Following the development of my game, I decided to hold a short follow-up meeting with my main stakeholder, (Anonymous), to present the solution to him. I showed him the program and all features within it to ensure maximum evaluation. Below is a summary of our conversation that we had (Harry, (Anonymous)):

1. *What are your thoughts on the design of both the menu screens and gameplay screens?*

“I like the colour scheme of the menu screens and the general tidiness of them. I think it would be good if the pause menu had the same theme as the main menu screen and if the tournament screen was a bit tidier. I like the gameplay screens.”

2. What are your thoughts on the audio and sound within the game?

“I think the audio is perfectly balanced within the game, it adds immersiveness but not too much distraction.”

3. Are you satisfied with the features within the game?

“Yes I like that the AI changes based on the score that it has left and the tournament feature is done well. I think some quality of life changes to enhance the game further would be nice.”

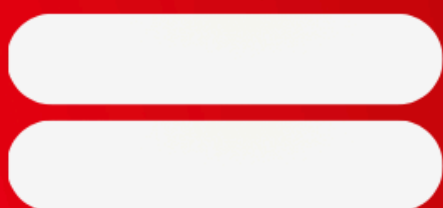
Week 17 - Iteration 13 - QOL Improvements

This is simply a final iteration of the game which includes a couple quality of life improvements such as the pause menu screen matching the other menu screens and tidying up the tournament matchup displays. Below are the new and improved versions of the changes that were made to improve the QOL of the game.

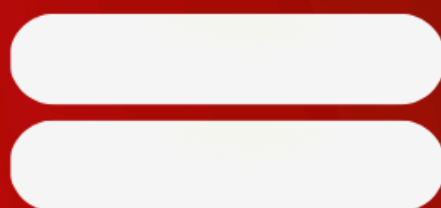
Final



Semi Finals



VS



Quarter Finals

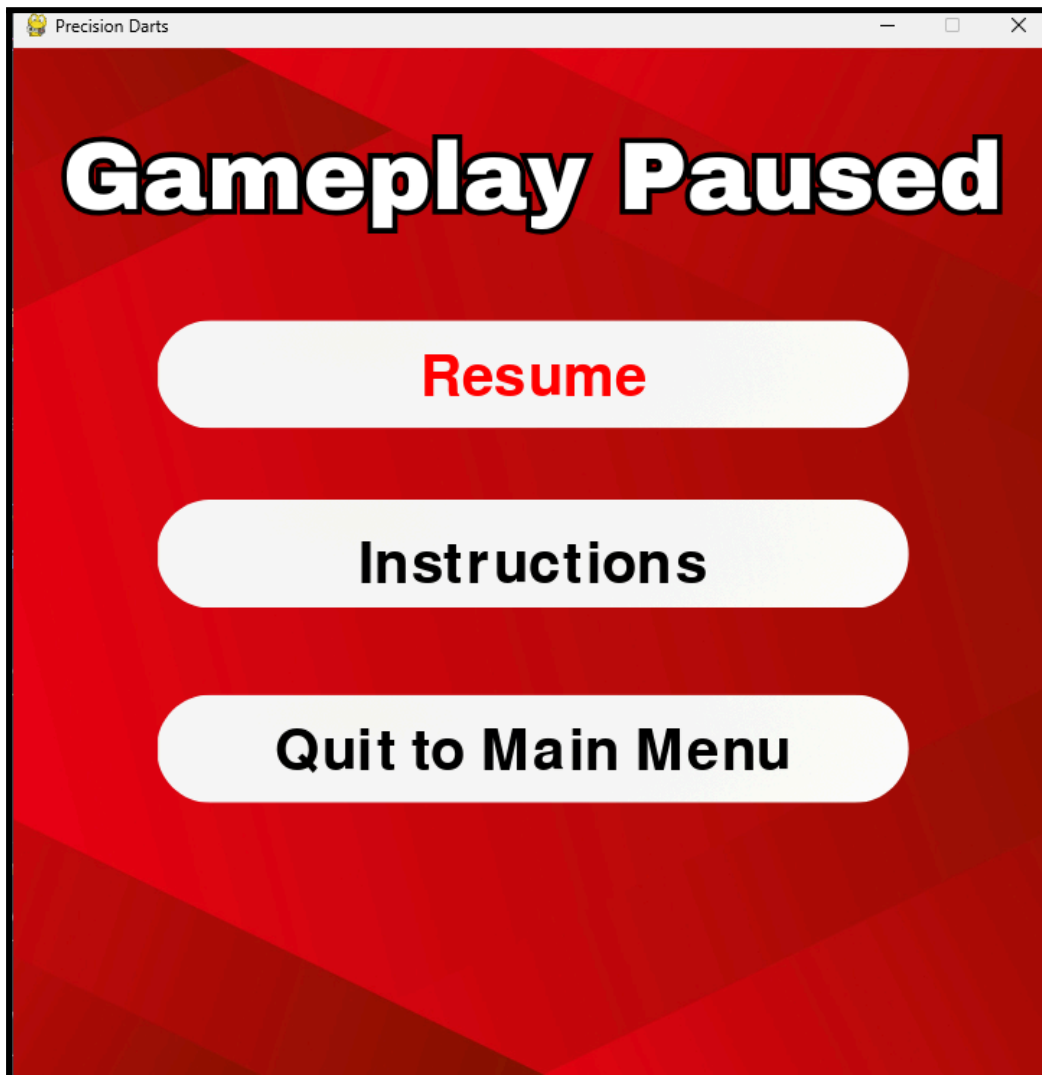
VS



Round of 16

VS





Evaluation

Post Developing Testing

Testing for Function and Robustness

Following the development of my game, I have carried out some additional testing for function and robustness to ensure that all components of the game functioned correctly under realistic conditions that had been set. The table below summarises the key test cases carried out to ensure this.

Test ID	Description	Expected Result	Actual Result	Pass/Fail
Testing Gameplay				
TC01	Player throws 3 darts	Turn switches to AI	Turn switched correctly	Pass
TC02	Player busts by overscoring	Score resets to turn start	Reset occurred, correct turn switch	Pass
TC03	Player busts on non-double finish	Score resets to turn start	Reset occurred, correct turn switch	Pass
Testing AI				
TC04	AI throws 3 darts	Control returns to human player	Works as intended	Pass
TC05	AI finishes leg on double	Leg correctly awarded and scores reset	Works consistently	Pass
Testing Tournament Completion				
TC06	Tournament Completes	Winner displayed and saved to leaderboard	Works as intended	Pass
TC07	Tournament count increases by 1 for the winner	Winner of the tournament stats increases by 1 on leaderboard	Works as intended	Pass
Testing Leaderboard				
TC08	Leaderboard persists data	Player stats remain after a restart	Data persists correctly	Pass

Testing Login				
TC09	Incorrect Password entered	Login rejected with message	Works as intended	Pass
TC10	Password stored in file	File contains hash, not plain-text	Securely hashed	Pass
Testing for Missing Assets				
TC11	Missing asset file	Game displays fallback surface	No crash	Pass
Testing Settings				
TC12	Audio volume changed	Audio within game changes to match the user change	Works as intended, volume adjusted	Pass

This confirms that the game operates reliably across all tested conditions, with data persistence, scoring accuracy, and user input validation performing correctly and in line with the success criteria outlined in the Analysis section of my project.

Usability Testing

Usability testing aimed to assess how intuitive and accessible the game was to my stakeholder and thinking about the broader target stakeholder group as a whole.

I carried out testing across 3 players, each with different experience levels with darts and games, to ensure I got a wider perspective for testing.

- **My stakeholder, (Anonymous):** Familiar with darts rules, typically plays some videogames too
- **User 2:** Intermediate player familiar with dart scoring, often played some games

- **User 3:** Unfamiliar with darts scoring and rules, but interested in learning.
Doesn't often play video games

I tested each of them across the features that were identified in the analysis section 'features of the proposed solution', as well as the key usability features in the design section. I have included whether or not the feature has been fully met, partially met, or not met at all, as per the success criteria outlined previously, and explained as to why.

Proposed Feature	Description & Justification	Met/Unmet	Additional comments
Instructions	Displays the instructions and rules of the game, alongside controls	Met fully: back button is included, not physically but via key input, users can navigate menus.	I showed images of the usability features proposed in the design section, as well as allowing them to test the game. All testers could navigate the menu and create accounts without external help.
Pause Screen	Pauses gameplay temporarily	Met fully: Pauses gameplay with ability to unpause and resume	The power bar mechanic was described as 'clear' and 'satisfying to use'.
Game over	Displays a game over screen, and provides the user with a key prompt to return to the menu	Partially Met: Includes a screen when the tournament finishes showing the tournament winner, but not a game over screen as designed previously.	The crosshair provided a strong sense of control, though some users suggested slower movement speed. The visual feedback from 'BUST!' and 'WIN!' overlays were well received.
Main menu	The game will have a structured menu	Fully met: as described, contains a	The AI was viewed as challenging but fair across different difficulties.

	allowing users to navigate between different game modes, settings, achievements, and leaderboards. This ensures a clear and user-friendly experience.	structured menu for users to navigate easily.	One tester suggested adding key remapping to the game. All testers suggested that more gamemodes be added into the game, and not just a tournament mode. All testers identified that the original game over screen presented to them, was different to the one in the game when winning a tournament, however they preferred it to the original.
Scoring Decrementation and Leg/Set Incrementation	The game will use the official darts scoring system, decrementing from 501 with proper validation for checkouts. This ensures an authentic experience for darts players. Upon reaching 0, a leg will be given to the player, and sets will be played as follows: 3 legs → 1 set 3 sets → win the match This format means that it is not too long or short of a format,	Fully Met: Game works as described with the scoring system and in the success criteria, however could be changed to be shorter.	All testers said that having 3 legs to win per set, and then 3 sets to win the match was quite long to win, so perhaps shortening would be better for engagement and replayability.

	remaining enjoyable to players.		
Scoreboard and Live updating	Displays player scores, throws, and required checkouts dynamically. Ensures players can track progress and strategy without many external calculations.	Fully Met: as described to the left, matching the success criteria	
AI Opponent (Multiple Difficulty Levels)	AI opponents simulate real darts players with varying skill levels: beginner (randomised errors), intermediate (strategic mistakes) and advanced (high accuracy). This ensures long-term engagement for all skill levels.	Fully met: The game provides 3 varying difficulties for users to be able to switch between to match their skill level and needs as outlined in the AI section of the success criteria.	
Gamertag System & Player Identification	Each player has a custom name displayed on leaderboards, improving clarity within the game	Fully Met: Each player has their own unique gamertag as outlined in the success criteria	
Keyboard-Ba	Players can	Not met: I did	

sed Controls (With Custom Keybinds)	customise their controls for aiming and throwing, improving accessibility and comfort.	not have sufficient time to implement custom keyboard systems.	
Leg/Set Win Screens	When a leg or set is won, the game displays a transition screen with animations to enhance immersion.	Partially Met: The game displays a flash and text popup when a leg is completed but not a full transition screen.	
Tournament Mode (16-Player Knockout)	A structured knockout-style tournament mode where the player competes against AI opponents. Winning matches progresses them through rounds until they reach the final. This adds long-term gameplay value. The tournament mode follows a 16-player knockout structure, ensuring a balanced progression	Fully Met: As described in the left panel, following guidelines of the success criteria.	

	system while keeping the experience engaging. • A smaller tournament (e.g., 8 players) would result in fewer matches, limiting competitive depth. • A larger tournament (e.g., 32 players) would make matches take too long, reducing accessibility for casual players. • 16 players allow for four rounds (First Round, Quarter-finals		
--	--	--	--

	, Semi-finals, Final), maintaining challenge while preventing fatigue.		
Leaderboard & Tournament Tracking	A leaderboard tracks match wins and trophies earned in tournaments. Competitive players are encouraged to play more and improve their ranking.	Fully Met: Players can track different stats such as total leg wins, tournament wins, and amount of achievements unlocked.	
Achievements & Trophy System	Players unlock trophies for winning tournaments or achieving high checkouts. This encourages replayability.	Fully Met: As described, players unlock achievements through playing the game such as hitting 9 darters or their first leg wins.	
Basic Sound Effects for Immersion	Includes dart impact sounds, and a sound for leg or set wins. These small touches enhance the gaming atmosphere.	Fully Met: Simple sounds for legs won, busting the score, or darts hitting the board have been included as per the success	

		criteria.	
Customisable Backgrounds & Dartboards	Players can choose from different background themes This adds aesthetic variety without affecting gameplay.	Fully Met: 3 different backgrounds available to use as designed.	
Game Settings & Accessibility Options	Includes volume control, key remapping, and difficulty adjustments to cater to different player preferences and accessibility needs.	Partially Met: All is included as described, except from the key remapping as I did not have time to implement this into the game.	

Overall, usability testing has helped to confirm that my game is functional, visually understandable, and enjoyable to interact with. Minor improvements such as key remapping and additional gamemodes could further enhance the experience, but the overall usability is strong and effective, and well received. I think that the usability features provided, based on this testing, is quite well rounded for my target stakeholder group and for my main stakeholder, as it provides a range of difficulties that, although are difficult, remain fair across the board. As well as this, the menus are easy to navigate and rules are easy to follow, so it would cater to all skill levels.

Evaluating the Success Criteria

How the Criteria has and hasn't been met

Evaluating the success criteria is a crucial point in the making of a successful game and project as a whole. I have decided to check against each point within my

success criteria outlined in the analysis section in order to determine whether it was met, partially met, or not met at all, giving elaboration where necessary.

Number	Requirement	Justification	Met or Unmet?
1	Startup Screen		
1.1	The game must have a startup screen with options to proceed.	The user will be able to navigate around to get to their desired game state. This provides a clear entry point.	Met
1.2	Users must be able to navigate menus using wasd keys	Provides Intuitive navigation for all users.	Partially Met: The game does not use wasd but instead the arrow keys to navigate menus.
1.3	A back button must be included in submenus.	Ensures users can return to previous menus without restarting the game.	Partially Met: No physical back button visually, however you can press backspace key to go back.
2	Gameplay Mechanics		
2.1	Players must be able to aim the darts using a controlled input method.	Allows for precision in gameplay.	Met: Uses arrow keys to aim the dart.

	2.2	A power meter must determine the strength of the dart throw.	Introduces skill-based mechanics.	Met: Power bar fluctuates at a set speed.
	2.3	The dart must follow a logical path based on user input such as where they aim, and the amount of power.	Ensures realistic gameplay.	Met: The dart lands in a realistic position based on the power you put on the throw and where you aim the crosshair, using deviation, to adjust path.
	2.4	User should be able to pause game during gameplay	Allows users to take a break if needed	Met: Users can pause by pressing the escape key
3	Scoring			
	3.1	The game must follow official 501 darts scoring, decrementing scores correctly.	Ensures accuracy and realism.	Met
	3.2	The player must finish on a double to win the game.	Follows official darts ruling and ensures accuracy.	Met
	3.3	A scoreboard must display the current score for the leg, as well as number of legs	Enhances user experience by providing necessary feedback.	Met

		and sets won for each player.		
4	AI Opponent and Difficulty Levels			
	4.1	The AI opponent must follow logical decision-making when throwing darts.	Ensures fair and competitive gameplay.	Met: Uses a checkout map to determine where to throw if on a finish, and if the AI is not on a finish, it will go typically for the highest scoring value until it is on a finish.
	4.2	The game must include multiple AI difficulty levels (Beginner, Intermediate, Advanced).	Allows players to choose a level of challenge that suits them, encouraging replayability.	Met
	4.3	The AI should make more frequent errors at lower difficulty levels.	Simulates realistic AI behaviour based on difficulty.	Met: The beginner AI has a greater range between the power than it can land on, the intermediate less of a range, and the advanced, even less again. In terms of deviation when throwing, there is less deviation as you

				increase the difficulty to ensure more realistic gameplay.
5	Game Flow and Win Conditions			
	5.1	The game must detect when a valid checkout has been achieved.	Ensures accurate match results and fair gameplay.	Met
	5.2	A victory screen must be displayed when a player wins.	Provides clear feedback on match results and adds an element of achievement.	Met: Flash, sound effect, and text popup displayed after each match win. Upon tournament win, a winner screen is displayed.
6	Achievement and Leaderboard System			
	6.1	Players must earn trophies for completing particular achievements or winning tournaments.	Encourages long-term engagement and adds competitiveness.	Met: Players receive achievements (trophies) for different tasks such as hitting a 9-darter, winning a tournament, or hitting 180's.
	6.2	A leaderboard must	Adds competitiveness	Met: Although met,

		display the top players based on different categories such as total match wins or tournament wins.	to the game, encouraging players to keep playing to become the best.	the categories vary slightly, containing total legs won, tournament wins, and number of achievements unlocked.
	6.3	Each player must have an individual gamertag that will be displayed on the leaderboard.	Individualises each player and gives players' a sense of achievement that is unique to them.	Met
7	Audio Feedback			
	7.1	Sound effects must be included for dart hits, menu navigation and game events.	Improves game immersion.	Met: Menu music added, and crowd ambience has been added to gameplay too.
8	Game Settings and Customisation			
	8.1	Players must be able to adjust audio settings.	Ensures accessibility for different player preferences.	Met: Users can change the audio levels or mute audio in the settings menu.
	8.2	AI difficulty must be adjustable	Provides a tailored experience for different skill levels.	Met: Difficulty can be changed in the settings menu.

	8.3	Background can be changed	Provides wider experience and customisation options	Met: Users can change the background in the settings menu
	8.4	Keybinds can be changed	Provides wider accessibility and option with how you play the game	Not Met
9	Performance and Stability			
	9.1	The game must be run at a minimum of 30 FPS	Ensures smooth gameplay on supported hardware.	Met
	9.2	The game must not crash during standard gameplay.	Provides a reliable user experience.	Met

Unmet Criteria and Further Development

Following on from evaluation of the success criteria points, I have picked out the points which were partially met, or not met at all in order to provide comment on how they could be addressed in further development.

Number	Requirement	Met or Unmet?	Further Development

1	Startup Screen			
	1.2	Users must be able to navigate menus using wasd keys	Partially Met: The game does not use wasd but instead the arrow keys to navigate menus.	The only reason that this is partially met is that I felt it was more accessible and intuitive to use arrow keys as opposed to wasd, as the arrow keys would also show direction, making it easier to follow. In order to fully meet this, I would simply have to change the keybinds within the code, however I feel it is not necessary.
	1.3	A back button must be included in submenus.	Partially Met: No physical back button visually, however you can press backspace key to go back.	This is partially met, as it does not contain a physical back button to press due to arrow key movement, as opposed to clicking with a mouse. If I were to develop the program further, I would include a visual back button

				for clarity, by editing my drawings of each menu screen, or by adding a drawing overlay. If i decided to add mouse movement later on, I would create a button which you could click on physically to take users back to the previous menu.
8	Game Settings and Customisation			
	8.4	Keybinds can be changed	Not Met	This point was never met during development due to time constraints in development. I had to delay the development of my AI due to difficulties that arose and therefore had less spare time towards the back end of the development. Further developing, I would create a screen on the

				settings menu where it took a user's key input and then I would feed that to the variable that stores the movement and throwing of darts and navigation of menus, in order to map the keys to the function.
--	--	--	--	--

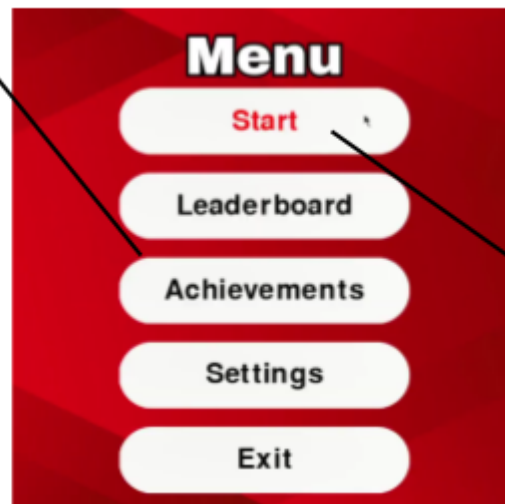
Evaluating Usability Features

Following development of my game, I have evaluated the usability features included within the game, annotating them with an explanation, followed by a short comment from my stakeholder, (Anonymous).

Evidence of Usability Features

Main Menu

Pressing the up and down arrow keys scroll through the menu options.



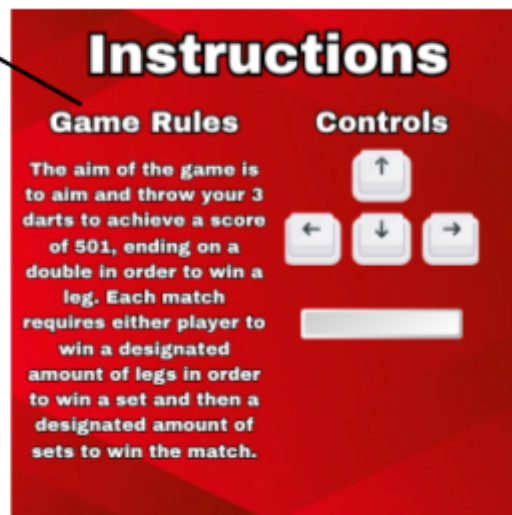
Pressing the enter key on the highlighted option, selects that option and navigates the user.

Me: “As we discussed, the main menu screen should allow user to navigate through the menu options using keyboard keys which were outlined as ‘wasd’, however due to the feel of greater intuitive navigation, I decided to go with using arrow keys as opposed to ‘wasd’

(Anonymous): “I like the fact that you have decided to use arrow key navigation in the game, over ‘wasd’ navigation, however as a final product, this would not be acceptable as it lacks clarity in saying what keybinds are to control the navigation.”

Instructions

This menu displays the game rules and controls for the user to play the game.



Me: “As discussed and designed, this is the instructions menu that was outlined, however with arrow key navigation instead of ‘wasd’. The instructions menu can be accessed via the pause menu in-game.”

(Anonymous): “This is good, however it is missing the enter key from the controls and clarity of what each key does. Also being able to access the instructions from the main menu would be good. Apart from that, it is good.”

Leaderboard

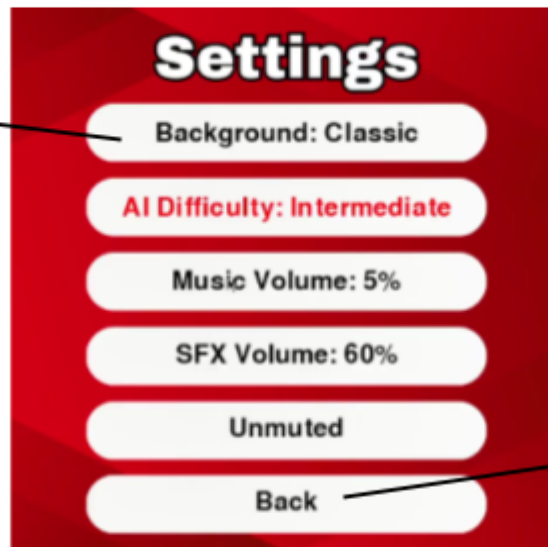


Me: "This is the leaderboard screen as discussed previously, showcasing the top 5 players in each of the categories displayed. I chose these 3 categories because tournaments offer competitive nature, legs won also encourage casual gameplay, and achievements target the collectionists of the game. This offers a variety to the users."

(Anonymous): "This is perfect, as it is clear to know the controls displayed on the screen, it is well presented and not cluttered, and offers variety within the game with different leaderboard types."

Settings

There are 5 settings to alter, 3 relating to audio control, 1 changing the AI difficulty, and 1 for background changing (3 available)



There is a back button, shown clearly at the bottom, to return to main menu.

Me: “The purpose of the settings menu is to allow for different accessibility within my game. The choice of including different difficulty levels is a choice made to cater towards different skilled players of my game, so that it can be played by a broader range of players.”

(Anonymous): “This is perfect. The back button included is very clear and I like that you have included different accessibility settings to users. No issue with this.”

Game Over

There is a text popup displaying both “Tournament Winner!” and the name of the winner beneath.



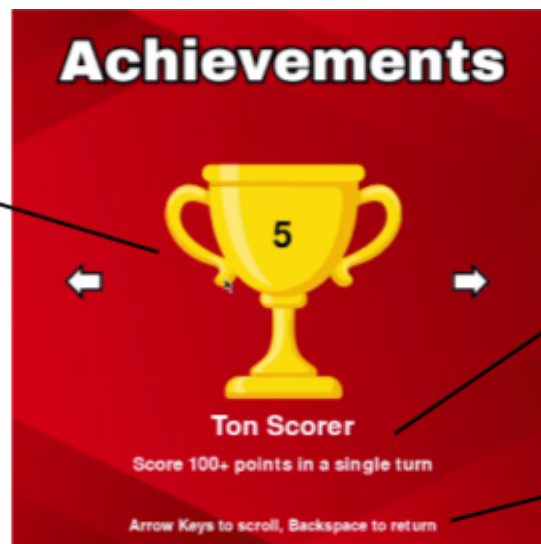
My game logo is showcased at the bottom of the screen.

Me: “The purpose of this screen is to add a sense of achievement to the winner of each tournament. Originally the game over screen that we discussed before, simply said ‘game over’, however I decided to take this approach as it added a better sense of achievement within the game and was more fitting.”

(Anonymous): “The choice to switch to a game over screen that adds this sense of achievement by winning the tournament is a good choice I think as it is clear, to the point, and also adds that rewarding feeling. I like it.”

Achievements

Achievements are displayed via trophy cups. If unlocked they are gold with the number of times won displayed in the center. If locked, they are displayed grey.



The achievement name and description is displayed beneath for clarity.

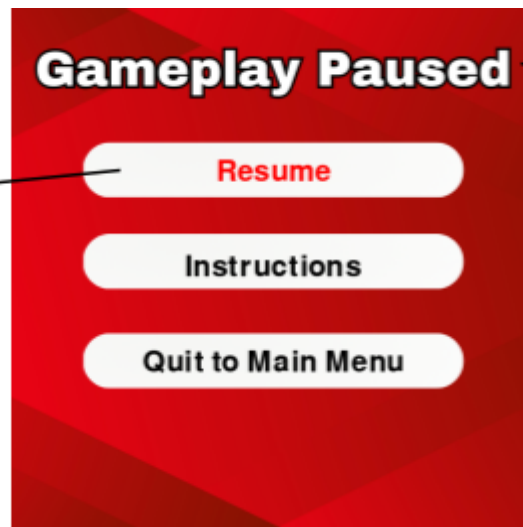
Navigation controls are showed at the bottom.

Me: “The purpose of this achievement screen was to add that extra sense of reward within the game for completing certain tasks, but also to add another element to play for within the game in order to encourage replayability. The choice of gold for unlocked, and grey for locked was made as these are typically colours associated with trophy unlocks.”

(Anonymous): “This is perfect. The design and look of this is very sharp, it offers support for navigation and what the achievement requirements are, no issue whatsoever with this.”

Pause Menu

There are 3 options: resume, instructions, and, quit to main menu which can be navigated to using arrow keys and pressing enter on the highlighted option.

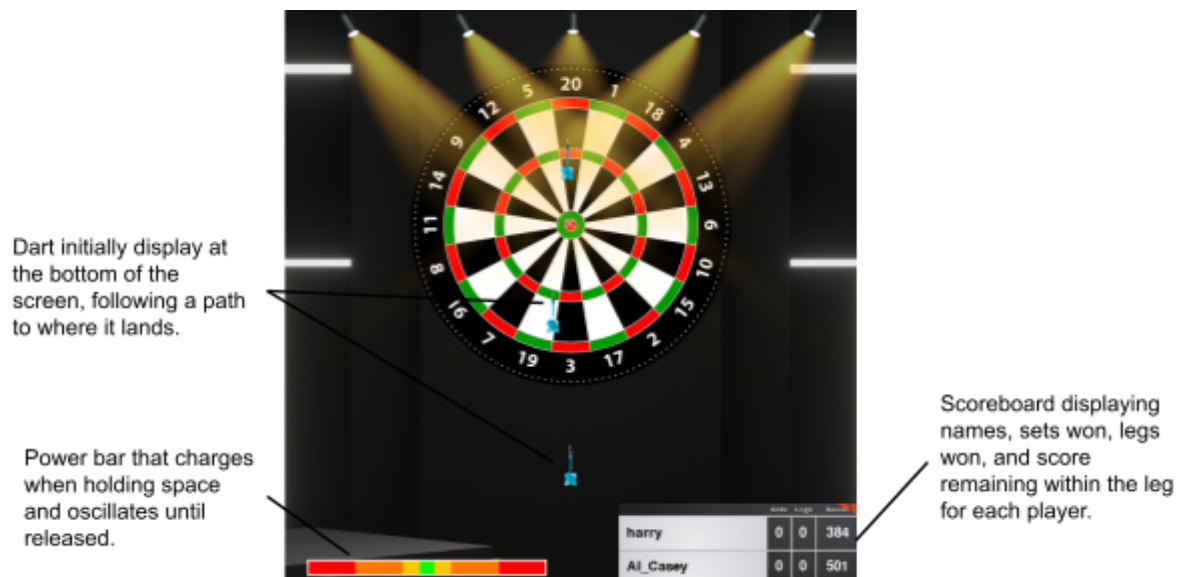


Menu name displayed clearly at the top.

Me: “The purpose of adding a pause menu to gameplay was to allow users to go and carry out other day-to-day tasks or take a short break but then be able to return to gameplay, which was a good choice. Also the choice to add instructions to the pause menu also helps players learn the rules and controls of gameplay before returning to the match which is very helpful.”

(Anonymous): “I like the idea of this pause menu as it is clear to the user, but it also provides feedback, through help to the user, giving them the rules and controls of the game should they need it, very nice.”

Playing the Game



Me: "The main features I wanted to get across when actually playing the game was to have a clear and working scoreboard and power bar that was easy to use to be accessible to all players no matter the skill level. I wanted the game to feel realistic, much like darts as a sport. Pressing the escape key brings up the pause menu."

(Anonymous): "I really like this, as it is clean-looking, vibrant, and is well developed. However, as a final product, it is unclear that there is a pause menu accessible, and users don't know how to access it."

Unmet Features and Further Development

Following on from the statements from my stakeholder, I have picked out the areas which he thought were missing, and have created a table below which outlines these and how I would go about altering them should there be further development.

Missing Feature	Further Development
Main menu keybinds lack clarity	If I were to carry out further development, in order to deal with this issue, I would take one

	of two approaches: the first being to add a screen that pops-up upon launching the key displaying the navigation keys, or simpler, I would add text overlays to display the keybinds on the main menu. This way, I can ensure that users are clearly informed how to navigate the main menu, leading to less frustration by users trying to figure out what the controls are.
Instructions menu does not include the enter key or escape key, and lacks clarity to what each key does	In order to fix this, I would edit my drawing for the instructions menu to include both the enter key and the escape key. I would then add a short section to explain what each key does, for example the arrow keys are for menu navigation and dart movement, and the escape key is to pause during gameplay. This provides more clarity and improves the replayability for perhaps people who need a bit more help figuring out what the controls of the game are.
Instructions menu not included on the main menu	In further development, I would choose to add a tab for the instructions menu within the main menu so that you don't have to go into gameplay and then onto the pause menu in order to access the instructions of the game, as this seems quite irrational.
Pause menu unclear within gameplay	A simple fix for this one: I would simply add a text overlay on the screen that displays to the user the key that they need to press to

	pause the game, but I would make it clear so that it is accessible for a range of users.
--	--

Improvements and Limitations

Maintenance Issues and Limitations

As of now I haven't implemented any additional features into my game through maintenance, however if this game was to be released professionally, then I would need to add these features to keep the game running smoothly and to prevent bugs. Although my game is lacking some depth, only having the option to play against AI in a tournament, I would add features such as local and online multiplayer through frequent patches to improve my game for the users, as well as a button where users can manually check for updates within the game. I think in order to gain feedback from them, I would also add a form within the game or section where players can feedback any bugs they encounter or features they would like to be added. By doing this, I, or a team of developers would be able to work towards fixing these bugs and also expanding the features within the game, in order to be more of a complete and successful game, with longevity and entertainment.

Improvements and Dealing with Limitations

As outlined in my 'Unmet Features and Further Development' there are certainly issues to be fixed and improvements to be made within the game. Should further development happen, it explains how I would go about fixing and dealing with these limitations in order to provide a much more complete and well-rounded game. I should also add that due to the game lacking depth, I would certainly choose to add more game modes to deal with that limitation of the game, as this would increase the replayability of my game, and if this was professionally distributed, I would want this to be the case.

Final Look at the Game

Attached is a series of videos that showcase the full coded solution, a walkthrough of the key programming, as well a match that was played, and a simulated tournament for showcase purposes of the full project.

Credits and Attribution

Asset Type: Background menu music (game loop)

Title: “Game music - action, uplifting house anthem, fast paced retro game melody leading into pumping synth chords” (from Zapsplat)

Source: Zapsplat (<https://www.zapsplat.com>)

Date Obtained: 22 October 2025

Usage Rights: Free to use under Zapsplat Free License for non-commercial / educational projects; attribution required

Edits Made: None

Credit Line: “Game music - action, uplifting house anthem, fast paced retro game melody leading into pumping synth chords” - provided by Zapsplat, used under Zapsplat Free License for non-commercial educational use

Asset Type: Background music (game loop), victory sound effect, bust sound effect

Source: Zapsplat (<https://www.zapsplat.com>)

Date Obtained: 22 October 2025

Usage Rights: Free to use under Zapsplat Free License for non-commercial / educational projects; attribution required

Edits Made: None

Credit Line: Provided by Zapsplat, used under Zapsplat Free License for non-commercial educational use

Asset Type: All images in the game

Created with: Canva (Free elements under Canva Free Media License)

Date Created: 18 August 2025

Usage Rights: Elements used are allowed under Canva Free Media License for personal / non-commercial projects

Credit Line: Created by Harry Marsh using Canva (Free elements under Canva Free Media License for non-commercial use.

Asset Type: Background music (win sound)

Title: Orchestral Win

Source: Pixabay (<https://pixabay.com/music>)

Date Obtained: 22 October 2025

Usage Rights: Free to use under Pixabay License; attribution recommended for educational / non-commercial projects

Edits Made: None

Credit Line: "Orchestral Win" - provided by Pixabay, used under Pixabay License

Asset Type: All other sound effects

Source: Pixabay (<https://pixabay.com/music>)

Date Obtained: 22 October 2025

Usage Rights: Free to use under Pixabay License; attribution recommended for educational / non-commercial projects

Edits Made: None

Credit Line: Provided by Pixabay, used under Pixabay License